Escuela Técnica Superior de Ingeniería Informática

Trabajo de fín de máster




# Aplicación de Computación Cuántica en el Procesamiento del Lenguaje Natural

*Autor:*

José Manuel Ramírez García

*Tutor:*

José Francisco Quesada Moreno

Noviembre, 2025

# Resumen

El procesamiento del lenguaje natural cuántico ofrece nuevos enfoques para superar las limitaciones de las técnicas clásicas. A lo largo de este trabajo se desarrollan los conceptos fundamentales necesarios para comprender el enfoque categórico, composicional y distribucional del modelo DisCoCat, el cual integra la representación distribucional del significado con las gramáticas formales y permite su implementación en dispositivos cuánticos. Posteriormente, se presenta una implementación del modelo en un caso práctico de clasificación multiclase de intenciones utilizando el conjunto de datos SNIPS y la librería Lambeq, que se compara con una versión simplificada del mismo. Finalmente, se estudian algunas propiedades relevantes para futuras implementaciones.



Palabras clave: Procesamiento del lenguaje natural, Procesamiento del lenguaje natural cuántico, Computación cuántica, DisCoCat, Lambeq.



# Abstract

Quantum natural language processing offers new approaches to address the limitations of classical techniques. This work develops the fundamental concepts necessary to understand the categorical, compositional, and distributional approach of the DisCoCat model, which integrates the distributional representation of meaning with formal grammars and enables its implementation on quantum devices. Afterward, the model is applied in a practical case of multi-class intent classification using the SNIPS dataset and the Lambeq library, and compared with a simplified version of the model. Finally, some relevant properties are examined for future implementations.



Keywords: Natural Language Processing, Quantum Natural Language Processing, Quantum Computing, DisCoCat, Lambeq

# Índice general

**1. Introducción** **7**

1.1. Modelos del lenguaje en el Procesamiento del Lenguaje Natural . . . . . 7

1.2. Procesamiento del Lenguaje Natural Cuántico . . . . . 9

1.3. Objetivos del trabajo . . . . . 11

1.4. Estructura del trabajo . . . . . 12

**2. Fundamentos teóricos** **14**

2.1. Introducción a la computación cuántica . . . . . 14

2.2. Aprendizaje automático cuántico . . . . . 23

2.3. Teoría de categorías . . . . . 24

2.4. Modelo del significado: DisCoCat . . . . . 31

**3. Clasificación de intenciones con Lambeq** **37**

3.1. Descripción del dataset . . . . . 37

3.2. Flujo de trabajo de Lambeq . . . . . 39

3.3. Modelo DisCoCat . . . . . 41

3.4. Modelo Box of Words . . . . . 52

3.5. Comparativa . . . . . 58

**4. Consideraciones en la implementación de los modelos** **61**

4.1. Efecto del embedding en el modelo Box of Words . . . . . 62

4.2. Resistencia al ruido del modelo Box of Words . . . . . 64

4.3. One-versus-rest en clasificación cuántica . . . . . 66

**5. Conclusiones** **68**

5.1. Futuras líneas de trabajo . . . . . 69

**A. Gráficas de entrenamiento y evaluación de los modelos** **71**

**Bibliografía** **80**

# Índice de figuras


1.1. Arquitecturas CBOW y Skip-gram de Word2Vec . . . . . 8

2.1. Visualización de los estados más representativos de un qubit sobre la esfera de Bloch. . . . . 15
2.2. Representación gráfica de las puertas AND, OR y NOT. . . . . 16
2.3. Representación gráfica de las puertas NAND y NOR como combinación de puertas AND, OR y NOT. . . . . 16
2.4. Diagrama de la operación $|\psi'\rangle = \hat{U}\,|\psi\rangle$. . . . . 17
2.5. Diagrama de la operación $HX\,|0\rangle = H\,|1\rangle = |-\rangle$. . . . . 19
2.6. Diagrama de la operación $(H \otimes X)(|\psi_1\rangle \otimes |\psi_2\rangle)$. . . . . 20
2.7. Diagrama y expresión matricial de la puerta Controlled-U. . . . . 21
2.8. Diagrama y expresión matricial de la puerta CNOT. . . . . 21
2.9. Esquema general de un circuito cuántico. . . . . 22
2.10. Arquitectura general de un circuito variacional. . . . . 23
2.11. Representación gráfica de los morfismos característicos en una categoría. . 26
2.12. Representación gráfica de las propiedades del producto tensorial en la categoría monoidal. . . . . 28
2.13. Representación gráfica de estados y efectos. . . . . 29
2.14. Representación gráfica de las propiedades de la dualidad en las categorías monoidales. . . . . 30
2.15. Diagrama de la oración *"Bob hates John"*. . . . . 34
2.16. Diagrama de la contracción $f = \epsilon_V \otimes \mathrm{Id}_S \otimes \epsilon_W : \overrightarrow{Bob} \otimes \overrightarrow{hates} \otimes \overrightarrow{John} \to \vec{s}$. 36

3.1. Palabras mas frecuentes en el conjunto de entrenamiento. . . . . 38
3.2. Flujo de trabajo de Lambeq para modelos DisCoCat con circuitos variacionales. . . . . 40

3.3. Ejemplo de diagrama generado a partir del análisis CCG de una oración del conjunto de entrenamiento. . . . . 42
3.4. Diagrama generado por el Reader *cups_reader* de Lambeq. . . . . 43
3.5. Diagrama simplificado de la figura 3.3 después aplicar las transformaciones de la fase de reescritura. . . . . 46
3.6. Circuito de la Fig. 3.5 con el *ansatz* `IQPCircuit` . . . . . 48
3.7. Comparación de la función de pérdida y el *accuracy* durante el entrenamiento y la validación a lo largo de 1,000 *epochs* para los modelos DisCoCat 51
3.8. Ejemplo de diagrama generado por `BoxReader` de una oración del conjunto de entrenamiento. . . . . 53
3.9. Distribución de las distintas estructuras generadas por el modelo *Box of Words* separadas por categoría. . . . . 54
3.10. Diagrama final de la Fig. 3.8 tras la fase de reescritura. . . . . 55
3.11. Circuito de la Fig. 3.10 con el *ansatz* `IQPCircuit` . . . . . 56
3.12. Comparación de la función de pérdida y el *accuracy* durante el entrenamiento y la validación de los modelos Box . . . . . 57
3.13. Desglose de tiempo de creación de los diagramas, su reescritura y transformación en circuitos para los diferentes modelos. . . . . 60

4.1. Representación de las palabras exclusivas y repetidas en la esfera de Bloch, diferenciadas por categoría. . . . . 62
4.2. Comparación de la función de pérdida y el *accuracy* durante el entrenamiento y la validación para los modelos entrenados sin *embedding* condicionado y con él. . . . . 63
4.3. Representación de las palabras exclusivas y repetidas en la esfera de Bloch, diferenciadas por categorías antes y después del entrenamiento con *embedding* condicionado. . . . . 64
4.4. Matrices de confusión del modelo $\text{Box}_{1,3,5}$ simulado clásicamente y ejecutado en una simulación cuántica en presencia de ruido. . . . . 65
4.5. Matriz de confusión del modelo $\text{Box}_{1,3,3}$ entrenado con One-versus-rest . . . 67

A.1. Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$. . . . . 71
A.2. Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_2$. . . . . 72

A.3. Gráfica de entrenamiento y validación del modelo $Box_{1,3,1}$. . . . . 72
A.4. Gráfica de entrenamiento y validación del modelo $Box_{2,3,1}$. . . . . 73
A.5. Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$. . . . . 73
A.6. Gráfica de entrenamiento y validación del modelo $Box_{1,3,5}$. . . . . 74
A.7. Gráfica de entrenamiento y validación del modelo $Box_{1,5,3}$. . . . . 74
A.8. Gráfica de entrenamiento y validación del modelo $Box_{1,3,5}$ con *embedding* condicional. . . . . 75
A.9. Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *add_to_playlist*. . . . . 75
A.10. Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *book_restaurant*. . . . . 76
A.11. Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *get_weather*. . . . . 76
A.12. Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *search_screening_event*. . . . . 77
A.13. Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *add_to_playlist*. . . . . 77
A.14. Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *book_restaurant*. . . . . 78
A.15. Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *get_weather*. . . . . 78
A.16. Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *search_screening_event*. . . . . 79

# Índice de tablas


3.1. Distribución de oraciones del subconjunto de datos de SNIPS. . . . . 37
3.2. Distribución de palabras exclusivas por cada categoría y la media de número de tokens en el conjunto de entrenamiento. . . . . 39
3.3. Número de estructuras únicas e índice de dominancia del conjunto de entrenamiento a nivel global y por categorías en el modelo DisCoCat. . . . . 44
3.4. Número de estructuras únicas e índice de dominancia del conjunto de entrenamiento a nivel global y por categorías después de la fase de reescritura. 47
3.5. Comparativa de los datos de entrenamiento de los modelos DisCoCat . . . 51
3.6. Resultados de los modelos DisCoCat aplicados al conjunto de prueba . . . 52
3.7. Número de estructuras únicas e índice de dominancia del conjunto de entrenamiento a nivel global y por categorías en el modelo *Box of Words*. . . 53
3.8. Comparativa de los datos de entrenamiento de los modelos Box . . . . . 58
3.9. Resultados de los modelos Box aplicados al conjunto de prueba . . . . . 58
3.10. Comparativa de los datos de entrenamiento de los modelos $\mathrm{Box}_{\mathtt{N},r,l}$ . . . . . 59
3.11. Resultados de los modelos $\mathrm{Box}_{\mathtt{N},r,l}$ aplicados al conjunto de prueba . . . . . 60

4.1. Datos de validación y prueba de los modelos con y sin *embedding* condicionado. . . . . 63
4.2. Comparativa del resultado del modelo $\mathrm{Box}_{1,3,5}$ simulado clásicamente y ejecutado en una simulación cuántica en presencia de ruido. . . . . 65
4.3. Comparativa de los mejores modelos DisCoCat y *Box of Words* respecto a sus análogos entrenados con One-versus-rest. . . . . 67

# Capítulo 1

# Introducción

## 1.1. Modelos del lenguaje en el Procesamiento del Lenguaje Natural

En la última década, la evolución de los modelos del lenguaje ha marcado una gran diferencia en cómo interactúan las personas con los ordenadores. En 2017, la publicación *Attention is all you need* [1] presentó los denominados *transformers*, marcando un antes y un después, y dando lugar a arquitecturas como BERT (*Bidirectional Encoder Representations from Transformers*) [2] de Google o GPT (*Generative Pre-trained Transformer*) [3] de OpenAI, que supusieron un gran avance en la comprensión automática de texto y sentaron las bases para los actuales *Large Language Models* (LLMs), como Gemini o ChatGPT.

En el campo del Procesamiento del Lenguaje Natural (NLP), un modelo de lenguaje es un sistema estadístico o de aprendizaje automático que predice las probabilidades de secuencias de palabras. Algunos de ellos, como los basados en *transformers*, generan *embeddings*, representaciones vectoriales del significado de las palabras o *tokens* [4]. Estas representaciones se apoyan en la hipótesis distribucional, formulada inicialmente por Z.S. Harris [5] y popularizada por J.R. Firth, quien resumió esta idea con la frase: *"You shall know a word by the company it keeps"* [6, p. 11]. La hipótesis propone que las palabras que tienden a aparecer en contextos similares poseen significados similares. Esto significa que los *embeddings* pueden aprenderse a partir de la distribución de las palabras en el corpus, de manera que el significado de las palabras se proyecte en el espacio vectorial donde las

relaciones semánticas pueden expresarse como distancias o direcciones [7, Cap. 5].

En 2003, el trabajo de Bengio et al. [8] propuso una forma eficiente de utilizar modelos del lenguaje basados en redes neuronales (*Neural Network Language Models*, NNLMs) para aprender los *embeddings* y mitigar el problema de la dimensionalidad que surge al construirlos a partir de datos estadísticos. Su idea consistía en que el NNLM aprendiera simultáneamente a asociar cada palabra a un vector de características denso y continuo —una representación distribuida del significado—, y a modelar la función de probabilidad de las secuencias de palabras en función de dichos vectores.

Posteriormente, este enfoque sería refinado y optimizado por Mikolov et al. [9] en su trabajo con *Word2Vec*. En él, se utilizan dos arquitecturas (Fig. 1.1) que reducen la complejidad computacional del aprendizaje de representaciones distribuidas en las NNLMs, simplificando el modelo al prescindir de las capas ocultas no lineales. La primera, denominada *Continuous Bag-of-Words Model* (CBOW), predice la palabra objetivo a partir de su contexto, sustituyendo las capas no lineales por una capa proyectora que promedia los vectores que representan sus palabras vecinas y un clasificador log-linear. La segunda arquitectura, conocida como *Skip-gram*, realiza el proceso inverso: toma una palabra como entrada y, mediante la capa de proyección y un clasificador log-linear, predice las palabras que la preceden y la siguen dentro de un rango.

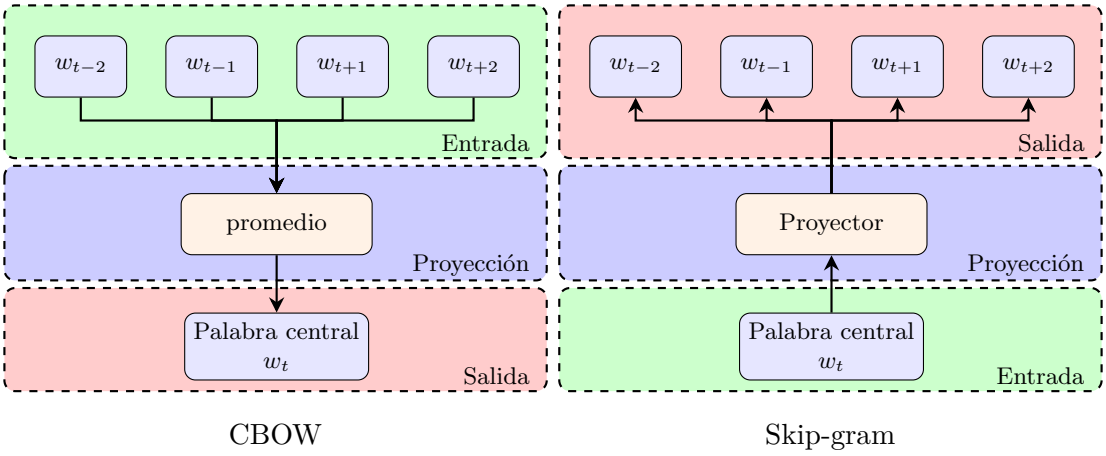



**Figura 1.1:** Arquitecturas CBOW y Skip-gram basadas en las ilustraciones del artículo original de Mikolov et al. sobre *Word2Vec* [9].

Estos *embeddings* pertenecen al conjunto de *embeddings* estáticos, lo que significa que

cada palabra está representada por un único vector, de dimensiones en un rango de 50 a 1000 [7, Cap. 5], dependiente únicamente de las palabras que aparecen en el vocabulario y manteniendo una representación fija. La llegada de los *transformers* permitió paralelizar enormemente el entrenamiento sobre enormes volúmenes de texto y aportó los mecanismos de atención, que permiten generar representaciones vectoriales para una misma palabra dependiendo del contexto, los denominados *embeddings* contextuales [7, Cap. 8]. Modelos como BERT o el primer GPT trabajan con este tipo de representaciones, mientras que los LLMs aprovechan esta idea a gran escala, manejando representaciones vectoriales de dimensiones muy altas. Sin embargo, estos modelos presentan limitaciones importantes en cuanto a la composicionalidad del lenguaje: las relaciones entre palabras no tienen una representación explícita, sino que quedan encapsuladas en estructuras muy complejas difíciles de interpretar. Para aportar mayor transparencia y comprensión, se busca incorporar la información estructural y gramatical a los modelos distribucionales [10].

Tanto el problema de la alta dimensionalidad como la necesidad de representar relaciones composicionales, junto a la búsqueda de nuevos algoritmos más eficientes y la alta exigencia de recursos por parte de las técnicas actuales, han motivado explorar enfoques alternativos, entre ellos la computación cuántica.

## 1.2. Procesamiento del Lenguaje Natural Cuántico

El Procesamiento del Lenguaje Natural Cuántico (*Quantum Natural Language Processing*, QNLP) es la disciplina emergente que combina los principios de la mecánica cuántica con las técnicas del procesamiento del lenguaje natural. Su objetivo es mejorar la eficiencia de los modelos, optimizar la complejidad computacional y proponer nuevas formas de representación lingüística, aprovechando las ventajas que ofrece el poder computacional de la computación cuántica [11].

La computación cuántica se basa en el uso de *qubits*, unidades de información cuántica que pueden encontrarse en estados de superposición y entrelazados entre sí. Estas propiedades permiten realizar operaciones sobre múltiples estados de forma simultánea, posibilitando resolver ciertos problemas con una complejidad menor a la requerida por

los algoritmos clásicos. Por ejemplo, el algoritmo de Grover [12] ofrece una aceleración cuadrática respecto al mejor algoritmo clásico para la búsqueda de un un elemento en una base de datos no ordenada de tamaño $N$, reduciendo el número de iteracciones de $O(N)$ a $O(\sqrt{N})$.

Actualmente, la capacidad tecnológica para construir ordenadores cuánticos nos sitúa en la era NISQ (*Noisy Intermediate-Scale Quantum*), caracterizada por dispositivos con un número limitado de *qubits*, sensibles al ruido provocado por la interacción con el entorno, a su limitación en la conectividad y a los errores acumulativos en las operaciones realizadas sobre ellos. Aun con estas limitaciones, estos sistemas permiten implementar algunos de los algoritmos que resultan ser mas eficientes que sus equivalentes clásicos y también permiten explorar su aplicabilidad en otros dominios, como en tareas de clasificación de texto o análisis de sentimientos en NLP [11].

A partir de estas ventajas, y considerando los problemas y limitaciones del procesamiento del lenguaje natural clásico mencionados en la sección anterior, podemos clasificar las técnicas de QNLP en dos enfoques principales: el distribucional y el distribucional-composicional.

Por un lado, el enfoque puramente distribucional se centra en adaptar versiones cuánticas de las técnicas habituales en el procesamiento del lenguaje natural. La capacidad de codificar de forma compacta las representaciones en estados cuánticos, permitiendo operar con múltiples dimensiones de manera simultanea, ha inspirado nuevas técnicas de *embedding* como *Word2ket* [13]; y arquitecturas cuánticas análogas a los modelos clásicos, como, por ejemplo, en tareas para clasificación de textos:

- *Quantum Support Vector Machine* (QSVM) [14].
- *Quantum Recurrent Neural Network* (QRNN) [15].
- *Quantum Self-Attention Neural Networks* (QSANN) [16].

Por el otro, el enfoque distribucional-composicional (DisCo) une la hipótesis distribucional con el principio de composicionalidad, que establece la idea de que el significado de una oración no depende únicamente del significado de las palabras que la compone,

sino que se construye de acuerdo con la estructura sintáctica que las combina, concepto atribuido a G. Frege [17].

En 1957, N. Chomsky publicó *Syntactic Structures* [18], donde formalizó las estructuras subyacentes del lenguaje natural mediante modelos gramaticales generativos [7, Cap. 18], un trabajo que sigue siendo relevante en la actualidad. Siguiendo esta linea, B. Coecke et al. desarrollaron el modelo del significado DisCoCat (*Categorical Compositional Distributional Model of Meaning*) [19], que integra las gramáticas formales de Chomsky con la representación distribucional del significado mediante la teoría de categorías, permitiendo su implementación tanto en sistemas clásicos como cuánticos.

## 1.3. Objetivos del trabajo

En este trabajo exploramos la aplicación de la computación cuántica al procesamiento del lenguaje natural siguiendo el modelo DisCoCat de B. Coecke. El objetivo principal es evaluar la capacidad de este enfoque en una tarea práctica, la clasificación multiclase de intenciones, utilizando el conjunto de datos SNIPS [20], ampliamente utilizado en problemas de comprensión del lenguaje y sistemas conversacionales. Específicamente, los objetivo serán:

1. Desarrollar los conceptos clave para la comprensión del modelo, incluyendo fundamentos de la computación cuántica, aprendizaje automático cuántico, teoría de categorías y la teoría subyacente al propio modelo DisCoCat.

2. Familiarizarse con el flujo de trabajo de Lambeq [21], la librería diseñada para trabajar con modelos categóricos dentro del NLP y del QNLP.

3. Implementar el modelo DisCoCat y compararlo con una versión simplificada del mismo para evaluar diferencias en rendimiento y complejidad.

4. Explorar propiedades relevantes que deben considerarse en futuras implementaciones y extensiones del modelo.

El código desarollado en este trabajo es una modificación y ampliación del publicado en la documentación de la librería Lambeq [22]. Dicho código se encuentra disponible en

el repositorio de GitHub [23] y se adjunta junto a este trabajo, en formato digital, con el nombre `Anexo.rar`. Todas las operaciones fueron ejecutadas en Google Colab, en un entorno con CPU AMD EPYC 7B12 (1 núcleo físico, 2 hilos, 64-bit) y 12.7 GB de RAM. Las peculiaridades del trabajo no hicieron posible utilizar las GPUs disponibles.

## 1.4. Estructura del trabajo

La estructura de este trabajo está formada por 4 capítulos, que siguen a la introducción, y un apéndice, los cuales dividen el contenido de la siguiente manera:

- Fundamentos teóricos: Este capítulo presenta los conocimientos básicos necesarios para comprender la naturaleza de la computación cuántica y el funcionamiento de los circuitos cuánticos. Se aborda cómo se implementan las tareas de aprendizaje automático cuántico mediante circuitos variacionales, así como los algoritmos de optimización más relevantes para el trabajo. Finalmente, se desarrollan los fundamentos de la teoría de categorías requeridos para introducir formalmente el modelo del significado DisCoCat.

- Clasificación de intenciones con Lambeq: Analiza el conjunto de datos empleado en la parte práctica, un subconjunto del *dataset* SNIPS utilizado en tareas de clasificación de intenciones. Se introduce la librería Lambeq [21], indicando el flujo de trabajo que emplea para llevar a cabo implementaciones de los modelos basados en DisCoCat. Posteriormente, cada parte del proceso se explica de manera detallada a medida que se lleva a la práctica, implementando el modelo paso a paso, mostrando los resultados obtenidos en cada fase y analizándolos para evaluar su comportamiento. Finalmente, siguiendo la idea de simplificación de *Word2Vec*, se implementa un modelo simplificado que se compara con el modelo anterior.

- Consideraciones en la implementación de los modelos: Se estudian aspectos relevantes a tener en cuenta al implementar los modelos desarrollados en el trabajo, enfocados en tareas de clasificación multiclase: En particular, se analiza la representación gráfica de las palabras y el efecto de condicionar la inicialización de los parámetros para obtener un *embedding* inicial antes del entrenamiento. Además, se evalúa la resistencia al ruido cuántico del mejor modelo entrenado en el capítulo

anterior y se comprueba el uso de la estrategia *One-versus-rest*.

- Conclusiones: Se expone de manera detallada lo abordado a lo largo del trabajo y se interpretan los resultados obtenidos, destacando las ventajas e inconvenientes del enfoque propuesto. Además, se plantean futuras lineas de ampliación, para futuras implementaciones o investigaciones.

- Apéndice A: Se presentan las gráficas de entrenamiento y evaluación de cada modelo por separado, para una interpretación más clara.

# Capítulo 2

# Fundamentos teóricos

## 2.1. Introducción a la computación cuántica

### 2.1.1. La unidad básica de información cuántica: El qubit

La computación cuántica es el campo que lleva a la práctica la teoría de la información cuántica, fundamentada en los principios de la mecánica cuántica, para procesar la información utilizando una serie de propiedades imposibles de replicar en ordenadores clásicos. A diferencia de la teoría de la información clásica introducida por Shannon [24], donde la unidad básica de información es el bit, cuyos valores posibles son 0 y 1, su análogo cuántico emplea qubits: vectores unitarios en el espacio $\mathbb{C}^2$ cuya base está descrita como

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{y} \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

y que representan estados del sistema que en su forma general se pueden escribir como una combinación lineal

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \tag{2.1}$$

donde los coeficientes $\alpha$ y $\beta$ se denominan amplitudes de probabilidad y deben cumplir la condición normalización de un estado cuántico, es decir,

$$|\alpha|^2 + |\beta|^2 = 1. \tag{2.2}$$

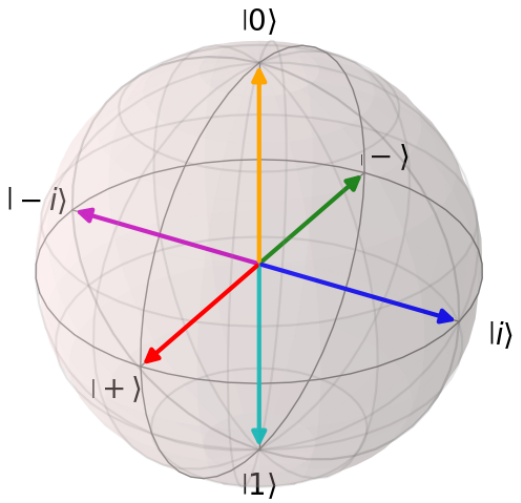
Un estado $|\psi\rangle$ que es combinación lineal de otros estados se denomina estado en su-

perposición, concepto fundamental en la mecánica cuántica. El módulo del cuadrado de cada amplitud, $|\alpha|^2$ y $|\beta|^2$, representa la probabilidad de observar el sistema en el estado $|0\rangle$ o $|1\rangle$, respectivamente. Cuando se realiza una medición, el sistema pasa a estar en el estado observado, perdiendo la superposición y la información contenida en ella. A este fenómeno se le denomina colapso de la función de onda [25]. Dado que una medición solo determina en qué estado se encuentra el sistema, para estimar las amplitudes de probabilidad es necesario realizar un muestreo estadístico, preparando el sistema en el mismo estado $|\psi\rangle$ una multitud de veces y midiendo los resultados.

Existe una representación geométrica tridimensional de los estados de un qubit al parametrizar la ecuación 2.1, de la forma

$$|\psi(\theta,\phi)\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2})|1\rangle\,, \tag{2.3}$$

donde $0 < \theta < \pi$ y $0 < \phi < 2\pi$. Esta expresión sitúa un estado $|\psi(\theta,\phi)\rangle$ sobre la superficie de una esfera unitaria, denominada Esfera de Bloch [26]:




$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \left|\psi\left(\frac{\pi}{2}, 0\right)\right\rangle$$
$$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \left|\psi\left(\frac{\pi}{2}, \pi\right)\right\rangle$$
$$|+i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle) = \left|\psi\left(\frac{\pi}{2}, \frac{\pi}{2}\right)\right\rangle$$
$$|-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i\,|1\rangle) = \left|\psi\left(\frac{\pi}{2}, \frac{3\pi}{2}\right)\right\rangle$$

**Figura 2.1:** Visualización de los estados más representativos de un qubit sobre la esfera de Bloch.

La Fig. 2.1 muestra esta representación, la cual resulta útil para entender la evolución temporal de un qubit tras la acción de las operaciones que intervienen en la computación cuántica.

### 2.1.2. Operaciones con qubits: Puertas cuánticas

Todo sistema de computación está diseñado para procesar información con el objetivo de realizar una serie de tareas. En los sistemas clásicos, el carácter binario de los bits exige que las operaciones sigan las reglas del álgebra booleana, en la que las operaciones fundamentales son conocidas como puertas lógicas. Las puertas lógicas son los bloques elementales de la computación clásica; las más comunes son las puertas AND, OR y NOT, las cuales se consideran un conjunto completo de puertas lógicas, es decir, que cualquier función lógica puede expresarse como una combinación de ellas.

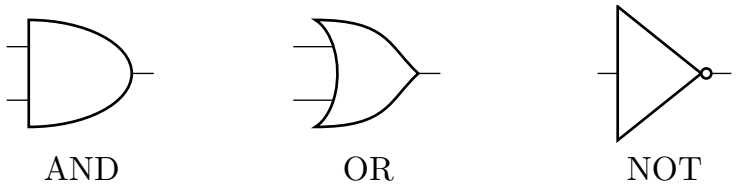



**Figura 2.2:** Representación gráfica de las puertas AND, OR y NOT.
Conjunto completo de puertas lógicas.

Las puertas lógicas tienen una representación gráfica que indica como fluye la información a través de ellas (Fig. 2.2). La combinación de estas representaciones, formando operaciones más complejas, se denominan circuitos lógicos o diagramas de circuito (Fig. 2.3). Esta visualización facilita tanto el diseño de sistemas digitales como la comprensión de las interacciones entre los distintos componentes dentro de dichos sistemas .

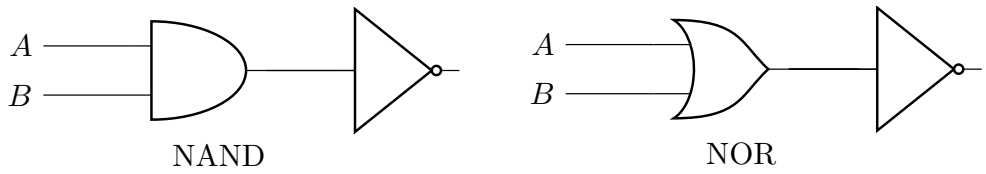



**Figura 2.3:** Representación gráfica de las puertas NAND y NOR
como combinación de puertas AND, OR y NOT.

Análogamente a la computación clásica, las operaciones sobre los qubits se realiza mediante las puertas cuánticas, las cuales pueden ser interconectadas y representadas gráficamente mediante diagramas de circuito (Fig. 2.4). Pero mientras que una puerta lógica realiza una acción determinista sobre los bits, las cuánticas deben atender a las propiedades probabilísticas y la coherencia de los estados cuánticos. En mecánica cuántica,

la ecuación de Schrödinger establece que la evolución de un sistema está dada por la acción de un operador unitario $\hat{U}$ sobre un estado $|\psi\rangle$ que describe el sistema en un momento dado, dando como resultado un nuevo estado $|\psi'\rangle = \hat{U}\,|\psi\rangle$. Los operadores unitarios tienen la propiedad de conservar la norma de los estados en los que se aplican, lo que mantiene la coherencia cuántica y asegura la reversibilidad de los cálculos [25] [27].

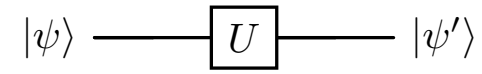



**Figura 2.4:** Diagrama de la operación $|\psi'\rangle = \hat{U}\,|\psi\rangle$.

Dado que el estado de un qubit se describe como un vector en el espacio $\mathbb{C}^2$ (Ec. 2.1), las transformaciones que actúan sobre estos deben ser lineales, es decir, deben cumplir:

$$U(|\alpha\psi_1\rangle + |\beta\psi_2\rangle) = U(\alpha\,|\psi_1\rangle) + U(\beta\,|\psi_2\rangle),. \tag{2.4}$$

Esta propiedad garantiza que cualquier operación pueda expresarse como una combinación lineal de los estados base $|0\rangle$ y $|1\rangle$. Esto lleva a que las puertas cuánticas, que involucran un qubit, también puedan ser representadas por matrices unitarias de dimensión 2 x 2. Como ejemplo, la puerta NOT ($X$), que invierte el estado de un qubit, se representa con la matriz:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \qquad \begin{aligned} X\,|0\rangle = |1\rangle\,, \quad X\,|1\rangle = |0\rangle \\ X\,(\alpha\,|0\rangle + \beta\,|1\rangle) = \beta\,|0\rangle + \alpha\,|1\rangle \end{aligned}$$

La matriz representativa de la puerta NOT ($X$) pertenece al conjunto de matrices de Pauli, las cuales tienen un papel fundamental en la mecánica cuántica [25]. Junto a la matriz de identidad, las matrices de Pauli son:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2.5}$$

Estas matrices, que a partir de ahora denotaremos como $I$, $X$, $Y$, $Z$, forman una base del espacio de matrices 2 x 2. Esto implica que cualquier puerta cuántica $U$ que actúa sobre un qubit puede expresarse como combinación de estas matrices:

$$U = c_0 I + c_1 X + c_2 Y + c_3 Z, \tag{2.6}$$

donde $c_0$, $c_1$, $c_2$ y $c_3$ son números complejos y, por tanto, clasifica el conjunto como conjunto universal de puertas cuánticas para un qubit.

Como se mencionó en el apartado anterior, los estados de un qubit se representan gráficamente en la esfera de Bloch. En este contexto, las matrices de Pauli desempeñan un papel fundamental, ya que son los generadores del grupo $SU(2)$ en el álgebra de Lie [28]. Estos generadores permiten realizar rotaciones infinitesimales en los ejes de la esfera, cuyos operadores son:

$$R_x(\theta) = e^{-i\frac{\theta}{2}\sigma_x}, \quad R_y(\theta) = e^{-i\frac{\theta}{2}\sigma_y} \quad \text{y} \quad R_z(\theta) = e^{-i\frac{\theta}{2}\sigma_z}. \tag{2.7}$$

Y en su formas matriciales:

$$\begin{aligned} R_x(\theta) &= \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}, \\ R_y(\theta) &= \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}, \\ R_z(\theta) &= \begin{pmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{pmatrix}. \end{aligned} \tag{2.8}$$

Escogiendo dos de estas transformaciones, podemos realizar cualquier giro en la esfera, por lo que cualquier combinación de ellas forma un conjunto universal de puertas para un qubit. Sin embargo, aunque podemos expresar cualquier operación como combinación de los elementos de los conjuntos universales mencionados, existen otras puertas muy relevantes en la mayoría de los algoritmos, entre las que destaca la puerta de Hadamard ($H$), que es especialmente importante porque crea estados en superposición [25]:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \qquad \begin{aligned} H|0\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ H|1\rangle &= \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \end{aligned}$$

Al combinar dos puertas que actúan secuencialmente en un qubit, se multiplican las matrices que las representan, siguiendo el orden inverso al que aparece en el diagrama. Por ejemplo, si aplicamos la puerta NOT ($X$) seguida de la puerta de Hadamard ($H$) al

estado $|0\rangle$ (Fig. 2.5):

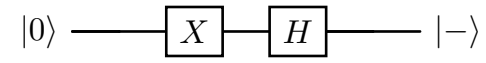



**Figura 2.5:** Diagrama de la operación $HX|0\rangle = H|1\rangle = |-\rangle$.

La expresión en notación braket sería:

$$HX|0\rangle = H|1\rangle = |-\rangle\,. \tag{2.9}$$

Expresada en forma matricial :

$$HX|0\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}\begin{pmatrix}1\\ 0\end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ -1 & 1\end{pmatrix}\begin{pmatrix}1\\ 0\end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\ -1\end{pmatrix} = |-\rangle\,. \tag{2.10}$$

### 2.1.3. Sistemas de múltiples qubits

Al trabajar con múltiples qubits, no podemos describir el estado de cada uno por separado; debemos tener en cuenta la interacción entre ellos. El espacio de Hilbert de un sistema compuesto de dos qubits se representa mediante el producto tensorial de los espacios de Hilbert individuales [29]:

$$H = H_1 \otimes H_2. \tag{2.11}$$

El producto tensorial da como resultado un espacio de Hilbert de dimensión $\mathrm{Dim}(H_1) \times \mathrm{Dim}(H_2)$. En el caso de un sistema con 2 qubits, genera un espacio de 4 dimensiones. Cualquier estado cuántico en este espacio se puede expresar en términos de la base como:

$$|\psi\rangle = \sum_{i,j} c_{ij}\,|i\rangle \otimes |j\rangle = \sum_{i,j} c_{ij}\,|ij\rangle\,, \tag{2.12}$$

donde $i, j \in \{0, 1\}$ y $c_{i,j}$ son números complejos que deben cumplir la condición de normalización $\sum_{i,j} |c_{i,j}|^2 = 1$, para que el estado cuántico $|\psi\rangle$ esté normalizado.

Cuando un estado cuántico $|\psi\rangle$ en el espacio compuesto puede expresarse como el producto tensorial de dos estados individuales $|\psi_1\rangle$ y $|\psi_2\rangle$:

$$\begin{aligned} |\psi\rangle &= (a_0 |0\rangle + a_1 |1\rangle) \otimes (b_0 |0\rangle + b_1 |1\rangle) \\ &= a_{00} |00\rangle + a_{10} |10\rangle + a_{01} |01\rangle + a_{11} |11\rangle , \end{aligned} \tag{2.13}$$

decimos que un estado separable, lo que implica que no existe interacción entre ambos sistemas. Si el estado no puede descomponerse de esta manera, se dice que el sistema está entrelazado [30]. El entrelazamiento es un fenómeno puramente cuántico y dado que:

$$\mathrm{Dim}(H_1) \times \mathrm{Dim}(H_2) \gg \mathrm{Dim}(H_1) + \mathrm{Dim}(H_2),$$

la mayoría de los estados de un sistema compuesto de qubits estarán entrelazados, con un ratio de $\frac{2^n}{n}$, que para $n = 100$ es del orden de $6{,}3 \times 10^{27}$ [25].

De manera análoga al tratamiento de lo estados cuánticos, las puertas cuánticas que operan sobre sistemas compuestos también pueden describirse como el producto tensorial de las puertas que actúan sobre un solo qubit. Esto es posible cuando las operaciones se producen en paralelo, es decir, cuando son transformaciones locales y que no generan correlaciones entre los qubits.

Por ejemplo, si aplicamos la puerta de Hadamard $H$ a un qubit y la puerta NOT ($X$) a otro (Fig. 2.6):

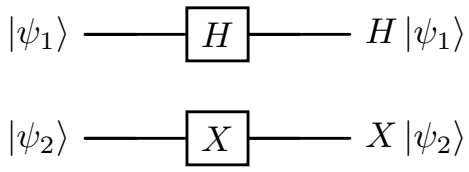


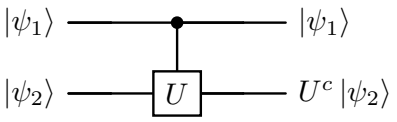

**Figura 2.6:** Diagrama de la operación $(H \otimes X)(|\psi_1\rangle \otimes |\psi_2\rangle)$.

la operación puede describirse como $H \otimes X$, actuando cada una únicamente sobre su propio qubit:

$$(H \otimes X)(|\psi_1\rangle \otimes |\psi_2\rangle) = H |\psi_1\rangle \otimes X |\psi_2\rangle . \tag{2.14}$$

Sin embargo, al igual que ocurre con los estados entrelazados, existen puertas cuánticas que no pueden expresarse como producto tensorial, ya que requieren la interacción entre los distintos sistemas. Por ejemplo, la puerta Controled-U (CU) es una familia de puertas

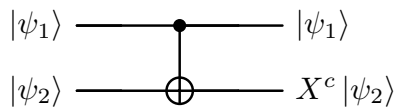
de dos qubits que presenta esta característica. Utiliza un qubit de control y otro de destino: la operación $U$ se aplica sobre el qubit de destino solo si el qubit de control está en el estado $|1\rangle$. Su representación en el diagrama y su forma matricial se muestran en la Fig. 2.7, donde $I_2$ es la matriz unidad 2 x 2 y $U^c$ es la transformación unitaria condicionada.




$$\text{CU} = \begin{pmatrix} I_2 & 0 \\ 0 & U^c \end{pmatrix}$$

**Figura 2.7:** Diagrama y expresión matricial de la puerta Controlled-U.

Dentro de esta familia, la más relevante es la puerta cuya transformación unitaria es la operación NOT($X$), conocida como puerta CNOT ($CNOT$)(Fig. 2.8). Esta puerta tiene un símbolo específico en los diagramas y, junto con los conjuntos de puertas universales para un único qubit mencionados anteriormente, conforma un conjunto universal capaz de implementar cualquier operación unitaria en computación cuántica [25].




$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

**Figura 2.8:** Diagrama y expresión matricial de la puerta CNOT.

### 2.1.4. Circuitos cuánticos y mediciones

De manera general, un circuito cuántico de $n$ qubits describe un espacio de Hilbert $(\mathbb{C}^2)^{\otimes n}$ y tiene los siguientes componentes [25]:

- Un registro cuántico de $n$ qubits, que tipicamente se inicia en el estado base $|0\rangle^{\otimes n}$.
- Una transformación unitaria $U_{\text{alg}} \in \mathbb{C}^{2^n \times 2^n}$, que representa el algoritmo que se desea implementar y que, como vimos anteriormente, se construye mediante una combinación de puertas cuánticas aplicadas de manera secuencial y/o paralela.
- Una serie de mediciones, necesarias para extraer la información deseada, colapsando los qubits a estados clásicos.

Formalmente, la medición corresponde a una colección de operadores proyectores:

$$P_x = |x\rangle \langle x|, \tag{2.15}$$

donde $x \in \{0,1\}^n$ y cada $|x\rangle$ representa uno de los $2^n$ posibles estados base del sistema. Hay que recordar que, tras la medición, el sistema colapsa al estado $|x\rangle$, y el proceso es probabilístico e irreversible. Por tanto, si queremos extraer la máxima información posible, es necesario ejecutar el circuito múltiples veces hasta obtener un muestreo representativo de los resultados.

Para ilustrar el funcionamiento de un circuito cuántico, resolveremos analíticamente el que se muestra en la figura 2.9:

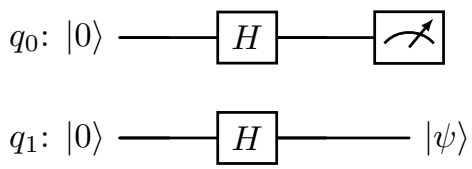



**Figura 2.9:** Esquema general de un circuito cuántico.

1. Se inicializa el circuito en el estado base:

$$|00\rangle = |0\rangle \otimes |0\rangle$$

2. Aplicamos la puerta de Hadamard a ambos qubit:

$$\begin{aligned} H|0\rangle \otimes H|0\rangle &= \frac{1}{2}(|00\rangle + |10\rangle + |01\rangle + |11\rangle) = \\ &= |0\rangle \otimes \left(\frac{1}{2}|0\rangle + \frac{1}{2}|1\rangle\right) + |1\rangle \otimes \left(\frac{1}{2}|0\rangle + \frac{1}{2}|1\rangle\right) = \\ &= \frac{1}{\sqrt{2}}|0\rangle \otimes \left(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle\right) + \frac{1}{\sqrt{2}}|1\rangle \otimes \left(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle\right) \end{aligned}$$

3. Por último medimos el primer qubit, que nos puede dar tanto $|0\rangle$ como $|1\rangle$ con probabilidad $\frac{1}{2}$. En ambos casos, puesto que el estado es totalmente simétrico, el segundo qubit colapsa al mismo estado de superposición:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle .$$

Lo cual es coherente, ya que el algoritmo consiste en dos transformaciones locales sin ninguna interacción entre los qubits.

Cuando se elige descartar todas las ejecuciones que no se correspondan con un valor de la medición concreto, se dice que se ha post-seleccionado el resultado. La post-selección es una técnica que permite simular operaciones no lineales, a pesar de que la evolución cuántica es estrictamente lineal y unitaria. Esta capacidad es fundamental para el desarrollo de modelos de redes neuronales cuánticas, como veremos en el siguiente apartado.

## 2.2. Aprendizaje automático cuántico

### 2.2.1. Circuitos variacionales

Los circuitos variacionales son el equivalente cuántico de las redes neuronales clásicas. Los datos se procesan a través de operaciones con parámetros ajustables que se optimizan para realizar tareas de aprendizaje. La arquitectura básica del circuito (Fig. 2.10) sigue una secuencia donde cada uno de los elementos juega un papel fundamental en la preparación, procesamiento y extracción de la información cuántica:

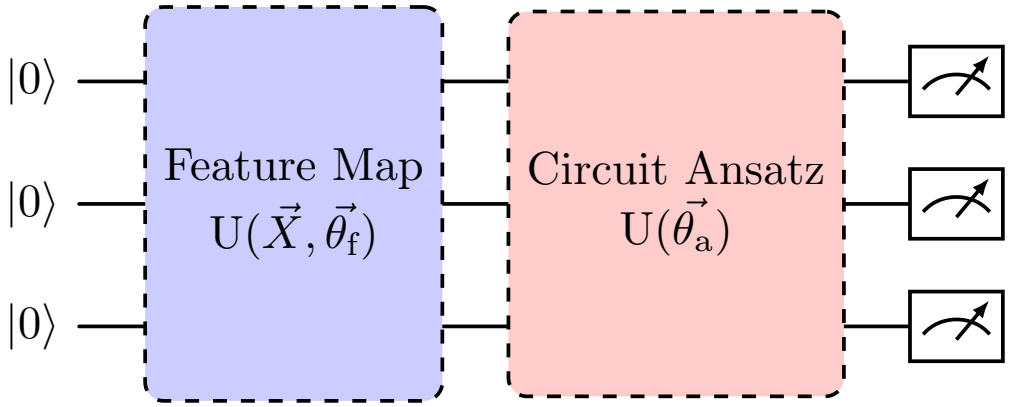



**Figura 2.10:** Arquitectura general de un circuito variacional.

#### Feature Map

Los datos clásicos deben ser transformados en estados cuánticos para poder procesarlos. Esto se realiza mediante un circuito parametrizado, comúnmente denominado Feature Map, que codifica la información utilizando diversas técnicas. Las más comunes son [31]:

- Codificación en base: En esta ténica, los bits clásicos se mapean directamente en los estados de la base computacional de los qubits, es decir, $|0\rangle$ y $|1\rangle$. Por ejemplo, el número 5 se representa 101 en binario, y se mapea en el estado cuántico:

$$5 \implies |\psi\rangle = |101\rangle\,.$$

- Codificación angular: En esta técnica, los datos clásicos se mapean en ángulos de rotación que se implementan mediante puertas cuánticas de diversas maneras. Por ejemplo, el número 5 se puede implementar como un conjunto de rotaciones en la esfera de Bloch:

$$5 \implies |\psi\rangle = R_{e_3}(\pi)R_{e_2}(0)R_{e_1}(\pi)\,|0\rangle \quad \text{o} \quad |\psi\rangle = R_{e_1}(\frac{5\pi}{180})\,|0\rangle\,,$$

  donde $e_1, e_2, e_3 \in \{x, y, z\}$, representan los ejes de rotación.

- Codificación en amplitud: La información se codifica en las amplitudes de un estado cuántico. Por ejemplo, el vector de datos [1.5,2,1.2,0.1], se mapea en las amplitudes de un estado cuántico de 2 qubits:

$$[1.5, 2, 1.2, 0.1] \implies |\psi\rangle = \frac{1.5}{\sqrt{7.7}}\,|00\rangle + \frac{2}{\sqrt{7.7}}\,|01\rangle + \frac{1.2}{\sqrt{7.7}}\,|10\rangle + \frac{0.1}{\sqrt{7.7}}\,|11\rangle\,,$$

  donde el denominador $\sqrt{7.7}$ es debido a la normalización para que el vector sea unitario.

**Circuit Ansatz**

Una vez que los datos clásicos se codifican en estados cuánticos, se aplica una transformación unitaria parametrizada encargada de modificar dichos estados para minimizar el error entre las predicciones y los datos etiquetados durante la fase de entrenamiento. A esta estructura básica se le denomina *ansatz* (pl. *ansätze*). Al igual que en las redes neuronales, existen muchas arquitecturas posibles, y la elección depende de cada caso específico. En cualquier caso, la simulación clásica de estos circuitos resulta costosa.

Uno de los *ansätze* más comúnmente utilizados es el circuito *Instantaneous Quantum Polynomial* (IQP), introducido por D. Shepherd y M.J. Bremner en su trabajo *Temporally*

*unstructured quantum computation* [32], que posee las siguientes características [33]:

- Las puertas cuánticas son conmutativas, lo que significa que las operaciones aplicadas a los qubits pueden realizarse en cualquier orden, incluso simultáneamente, sin que el resultado final cambie. Esto permite que el circuito opere de forma paralela e independiente en cada qubit.
- Las puertas son diagonales en la base computacional, lo que implica que las operaciones sobre los qubits corresponden a rotaciones de fase. Las transformaciones que cumplen esta propiedad no modifican el entrelazamiento entre los qubits, lo que simplifica la complejidad del circuito.

Las ventajas de este modelo de circuito son que está diseñado para resolver problemas en tiempo polinómico respecto al número de qubits.

**Mediciones**

Las mediciones en los circuitos variacionales no solo sirven para obtener los datos de salida del circuito, sino que también cumplen varias funciones: influyen en el proceso de aprendizaje, intervienen en los algoritmos de actualización de parámetros y actúan como puente entre la información cuántica procesada y la información clásica utilizada para ajustar y obtener resultados. La salida del *ansatz* es un estado en superposición que, al ser medido, colapsa en uno de los posibles resultados con una probabilidad determinada. Esta información puede ser relacionada de varias maneras de forma que podamos extraer conclusiones válidas para la tarea específica que se desea abordar:

- Medición del valor esperado de un observable: En mecánica cuántica, un observable $\hat{O}$ representa cualquier propiedad de un sistema físico que puede ser medida experimentalmente. Se representan como operadores hermíticos, y el valor esperado de un observable $\langle \hat{O} \rangle$ proporciona un promedio de las mediciones obtenidas tras múltiples repeticiones. Este valor está directamente relacionado con las amplitudes de probabilidad del estado cuántico y se calcula mediante la expresión [25]:

$$\langle \hat{O} \rangle = \langle \psi | \, \hat{O} \, | \psi \rangle \, , \tag{2.16}$$

donde $|\psi\rangle$ es el estado cuántico del sistema tras aplicar el circuito. En computación cuántica, es común utilizar el operador de Pauli $\sigma_z$ ($Z$), ya que se relaciona con la base computacional de la siguiente manera:

$$\sigma_z |0\rangle = |0\rangle \quad \text{y} \quad \sigma_z |1\rangle = -|1\rangle . \tag{2.17}$$

- Medición de las amplitudes de probabilidad del estado: Al medir un observable sobre un estado, el sistema colapsa a uno de los posibles estados de la base computacional. Medir las amplitudes de probabilidad es equivalente a realizar un muestreo normalizado de las mediciones obtenidas. En mecánica cuántica, el operador que calcula las probabilidades de medir un estado específico $|i\rangle$ es el proyector asociado al estado $P_i$, dado por [25]:

$$P_i = |i\rangle \langle i| \tag{2.18}$$

Aunque las mediciones representan el paso final del circuito, son en realidad el primer paso para que el modelo cuántico pueda aprender. Como en el caso clásico, los resultados obtenidos guiarán la optimización de los parámetros del circuito.

### 2.2.2. Optimización de parámetros

El Aprendizaje automático cuántico con circuitos variacionales es un enfoque híbrido, no completamente cuántico, ya que combina componentes clásicos en el proceso de aprendizaje. Generalmente, la optimización se lleva a cabo con técnicas de aprendizaje convencional, como descenso por el gradiente, aunque la evaluación de ciertos parámetros de un circuito pueda implicar operaciones cuánticas.

En este enfoque, un circuito parametrizado (Fig. **??**) representa el modelo de aprendizaje, mientras que un optimizador clásico ajusta los parámetros del circuito calculando una función de coste dependiente de las mediciones realizadas, $\mathscr{L}(\vec{\theta})$. En cada iteración del optimizador, el circuito se ejecuta y se mide; proceso que se repite hasta alcanzar los parámetros óptimos.

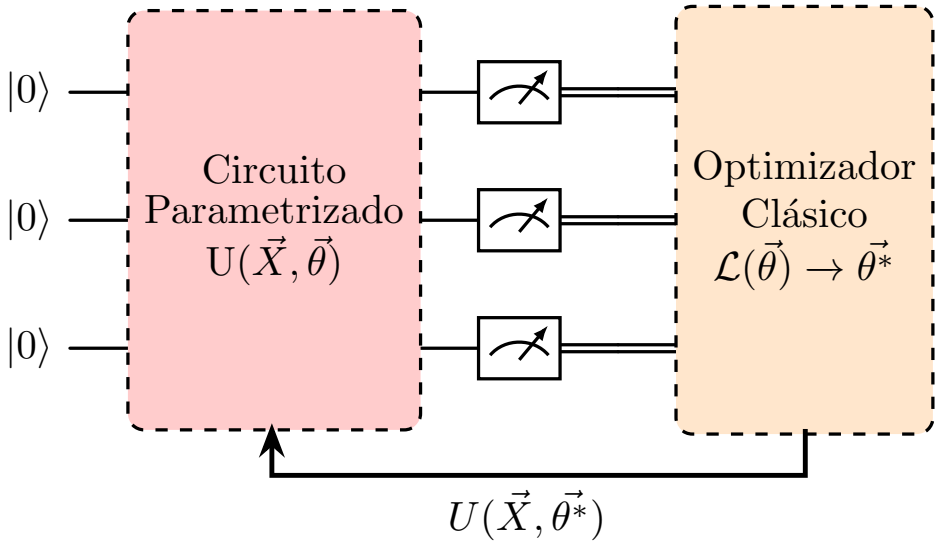



**Figura 2.11:** Diagrama de optimización de un algoritmo cuántico con optimización clásica.

Existen diversos algoritmos de optimización que han sido diseñados específicamente para la naturaleza de los circuitos cuánticos, y otros que, aunque no fueron desarrollados con este propósito, han demostrado ser efectivos. Los algoritmos más representativos de estas dos vertientes son el *Parameter Shift Rule* [34] y el *Simultaneous Perturbation Stochastic Approximation* [35].

### Parameter Shift Rule

El algoritmo *Parameter Shift Rule* es una técnica para calcular la derivada exacta del valor esperado de un operador $\hat{O}$, con respecto a los parámetros de un circuito cuántico. Si la función $f(\theta)$, representa el valor esperado del operador medido en función de un parámetro $\theta$, podemos escribir la ecuación **??** como [34]:

$$f(\theta) = \langle\psi| \hat{U_G^\dagger}(\theta)\hat{O}\hat{U}_G(\theta) |\psi\rangle , \tag{2.19}$$

donde $\hat{U}_G(\theta)$ es la transformación unitaria del sistema bajo el parámetro $\theta$. Considerando que las transformaciones unitarias de las puertas cuánticas pueden expresarse como:

$$\hat{U}_G(\theta) = e^{-ia\hat{G}}, \tag{2.20}$$

donde $\hat{G}$ es el operador hermítico de la puerta y $a$ es una constante real, la regla establece que si $\hat{G}$ tiene dos únicos autovalores, $e_0$ y $e_1$, la derivada de $f(\theta)$ respecto a $\theta$

es proporcional a la diferencia de los valores esperados del $\hat{O}$ con un desplazamiento del parámetro:

$$\frac{d}{d\theta}f(\theta) = r\left[f(\theta + \frac{\pi}{4r}) - f(\theta - \frac{\pi}{4r})\right], \tag{2.21}$$

donde $r = \frac{a}{2}(e_1 - e_0)$. El valor $r$ depende de la transformación. Por ejemplo, para las puertas de Pauli:

$$\begin{aligned} R_x(\theta) &= e^{-i\frac{\theta}{2}\sigma_x} \quad con \quad r = \frac{1}{2}, \\ R_y(\theta) &= e^{-i\frac{\theta}{2}\sigma_y} \quad con \quad r = \frac{1}{2}, \\ R_z(\theta) &= e^{-i\frac{\theta}{2}\sigma_z} \quad con \quad r = \frac{1}{2}, \end{aligned} \qquad \frac{d}{d\theta}f(\theta) = \frac{1}{2}\left[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})\right]. \tag{2.22}$$

Esta regla nos permite calcular gradientes directamente en el circuito cuántico sin tener que recurrir a aproximaciones numéricas. El número de ejecuciones aumenta con los parámetros del circuito, ya que cada parámetro requiere dos ejecuciones del circuito para su cálculo. Además, la presencia de ruido en las operaciones y medidas, comunes en la era NISQ, puede reducir la precisión de las derivadas.

**Simultaneous Perturbation Stochastic Approximation**

El uso de hardware para computación cuántica y la simulación de circuitos en hardware clásico presenta limitaciones en términos de costo y capacidad de procesamiento. En circuitos con un gran número de qubits, una elevada cantidad de parámetros, una alta profundidad o con presencia de ruido, los algoritmos de optimización basados en el descenso del gradiente, como el *Parameter Shift Rule*, resultan ineficientes. Por ello, es necesario explorar otras estrategias que puedan abordar las particularidades de los circuitos cuánticos. Sería ventajoso encontrar un optimizador que:

- Minimice el número de ejecuciones del circuito.
- Sea resistente al ruido en las operaciones cuánticas.
- Sea escalable a sistemas de alta dimensionalidad.
- Tenga independencia del número de parámetros por iteración.

El algoritmo *Simultaneous Perturbation Stochastic Approximation* (SPSA) es un método de optimización estocástico que cumple con estas características. Está diseñado para optimizar funciones cuando no se conoce su forma explícita o cuando calcular el gradiente exacto resulta costoso. Introducido por J.C. Spall [35], ofrece una alternativa más eficiente que los algoritmos basados en diferencias finitas, especialmente en problemas de alta dimensionalidad.

El algoritmo SPSA procede de manera similar a los demás algoritmos de descenso del gradiente en los que se calcula de forma iterativa la actualización de parámetros [36]:

$$\hat{\theta}_{k+1} = \hat{\theta}_k - a_k \hat{g}_k(\hat{\theta}_k). \tag{2.23}$$

Sin embargo, $\hat{g}_k$ es la estimación estocástica del gradiente $g(\theta) = \frac{d\mathscr{L}}{d\theta}$ con los parámetros del paso anterior y $a_k$ es la tasa de aprendizaje, definida positiva. En cada iteración $k$, la estimación viene dada por la expresión:

$$\hat{g}_{ki}(\hat{\theta}_k) = \frac{f(\hat{\theta}_k + c_k\Delta_k) - f(\hat{\theta}_k - c_k\Delta_k)}{2c_k\Delta_{ki}}, \tag{2.24}$$

donde:

- $\hat{g}_{ki}(\hat{\theta}_k)$: Es la estimación del gradiente del i-ésimo parámetro.
- $f(\hat{\theta})$: Es la función que queremos minimizar, evaluada en los parámetros $\hat{\theta}_k$.
- $c_k$: Es un una constante que controla la amplitud de las perturbaciones y que, como ocurre con la tasa de aprendizaje, suele decaer conforme aumentan las iteraciones.
- $\Delta_k$: Es el vector de perturbaciones aleatorias cuya dimensión es el número de parámetros. Cada uno de sus elementos $\Delta_{ki}$ son independientes y toma valores de $\pm 1$, o son generados aleatoriamente mediante una distribución gaussiana estándar.

La ventaja de esta aproximación es que limita a dos las ejecuciones del circuito, independientemente del número de parámetros a optimizar. Además, la aleatoriedad en la elección de $\Delta_k$ hace que el proceso sea resistente al ruido, lo que lo convierte en una opción robusta en sistemas cuánticos con errores de operación. Por último, también cumple con

el criterio de escalabilidad, ya que no es necesario calcular explícitamente cada derivada parcial. Por estos motivos, utilizaremos este algoritmo en la parte práctica del trabajo.

## 2.3. Teoría de categorías

Como hemos visto, la computación cuántica representa la información mediante qubits, vectores en espacios de Hilbert, y la procesa mediante puertas unitarias que se organizan en circuitos cuánticos. Este enfoque es muy ventajoso para diseñar algoritmos, pero puede resultar algebraicamente complejo y dificulta el razonamiento sobre la composición de los procesos o la equivalencia entre circuitos, especialmente a medida que los cálculos aumentan en complejidad. Para abordar este problema, la mecánica cuántica categórica recurre a la teoría de categorías, proporcionando una representación más general y estructurada de los procesos cuánticos.

La teoría de categorías es un marco formal centrado en el estudio de las estructuras matemáticas a través de sus relaciones. En lugar de analizar los objetos matemáticos por su contenido, estudia las relaciones estructurales entre ellos por medio de morfismos. Su objetivo es describir propiedades comunes de las distintas áreas matemáticas, facilitando la conexión y transferencia de conceptos entre ellas.

### 2.3.1. Categorías

Una categoría consiste en una colección de objetos $Ob(\mathscr{C})$, también llamados sistemas, denotados como $A$, $B$, $C$, etc; y una coleción de morfismos $Ar(\mathscr{C})$, también llamados procesos o flechas, denotados como $f$, $g$, $h$, etc; que cumplen las siguientes condiciones [37]:

- Cada morfismo $f : A \to B$, tiene un sistema de entrada (dominio) $A$ y un sistema de salida (codominio) $B$, denotados como $\mathrm{Dom}(f)$ y $\mathrm{Cod}(f)$, respectivamente.
- Para cada objeto, existe un morfismo identidad $1_A : A \to A$, que representa no modificar el sistema.

- Si existe un morfismo $f : A \to B$, y otro morfismo $g : B \to C$; también existe un morfismo, que es su composición secuencial, tal que $f \circ g : A \to C$, que también pertenece a la categoría.
- La composición secuencial es asociativa, es decir: $f \circ (g \circ h) = (f \circ g) \circ h$.
- Los morfismos de identidad son elementos neutros respecto a la composición, es decir, $1_A \circ f = f = f \circ 1_B$.

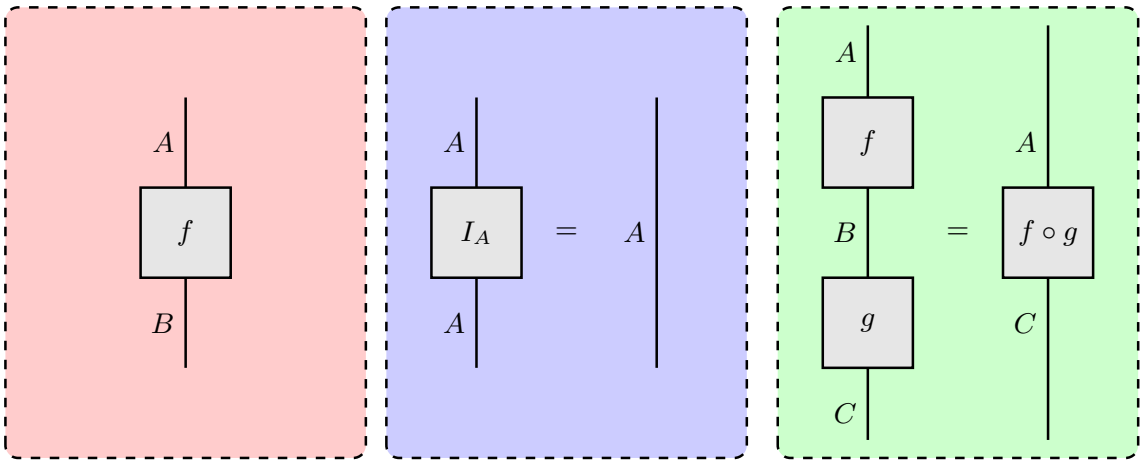



**Figura 2.12:** Representación gráfica de los morfismos característicos en una categoría: a la zquierda, un morfismo $f : A \to B$; en el centro, el morfismo idenntidad $I_A : A \to A$; a la derecha, una composición de morfismos $f \circ g : A \to C$.

Una de las herramientas más poderosas de la teoría de categorías es la representación gráfica de procesos mediante diagramas de cuerdas. En ellos, los objetos se representan con lineas y los morfismos con cajas. Por ejemplo, en la Fig. 2.11 se muestran el diagrama correspondiente a un proceso $f : A \to B$, representado como una caja con la linea de entrada ($\mathrm{Dom}(f)$) en la parte superior y la linea de salida ($\mathrm{Cod}(f) = B$) en la parte inferior.

Esta representación también permite observar la equivalencia de los diagramas: por ejemplo, el proceso identidad $I_A : A \to A$ puede representarse como una linea sin caja, ya que no modifica el objeto. Otra simplificación es la aplicación secuencial de procesos: $f : A \to B$ y $g : B \to C$ es equivalente a un único proceso ($f \circ g : A \to C$), donde $\mathrm{Dom}(f \circ g) = \mathrm{Dom}(f)$, y $\mathrm{Cod}(f \circ g) = \mathrm{Cod}(g)$.

Aquí empezamos a ver la conexión con los circuitos cuánticos; pero la definición de categoría todavía necesita más elementos estructurales para poder representar las propie-

dades de la mecánica cuántica. Algunos conjuntos que sí cumplen con estas características son [37]:

- Categoría **Vect**:
  - Objetos: Espacios vectoriales.
  - Morfismos: Aplicaciones lineales entre espacios vectoriales, $f : V \to W$.
  - Identidad: La aplicación que no modifica los elementos de $V$, $I_V : V \to V$
  - Composición: La composición de funciones, $f(gh) = (fg)h$.
- Categoría **Mat**:
  - Objetos: Números naturales $\mathbb{N}$, que indican la dimensión de espacios vectoriales.
  - Morfismos: Matrices $m \times n$, que modifica la dimensión del espacio.
  - Identidad: La matriz identidad $n \times n$, $I_n$.
  - Composición: La multiplicación matricial.

### 2.3.2. Functores

Las categorías también pueden relacionarse entre sí como si fueran objetos. Los morfismos entre categorías se denominan functores. Dadas dos categorías **C** y **D**, un functor $F : \mathbf{C} \to \mathbf{D}$, asigna [37]:

- A cada objeto $A \in \mathrm{Ob}(\mathbf{C})$, un objeto $F(A) \in \mathrm{Ob}(\mathbf{D})$;
- A cada morfismo $f : A \to B$ en **C**, un morfismo $\mathrm{F}(f) : \mathrm{F}(A) \to \mathrm{F}(B)$ en D.

Además, debe cumplir las siguientes propiedades:

- $\mathrm{F}(g \circ f) = \mathrm{F}(g) \circ \mathrm{F}(f)$, para todos los morfismos $f : A \to B$ y $g : B \to C$ en **C**;
- $\mathrm{F}(\mathrm{Id}_A) = \mathrm{Id}_{\mathrm{F}(A)}$ para cada objeto $A \in \mathbf{C}$.

El functor nos permite definir la equivalencia entre categorías. Podemos decir que la categoría **C** es una subcategoría de **D** cuando todos los objetos en **C** son objetos en **D**, cada morfimo en **C** es un morfismo en **D**, y las composiciones e identitades en **C** son las mismas que en **D**. Si, además, cada morfismo de **D** proviene de un morfismo de **C**, las

categorías son equivalentes. La equivalencia implica que no importa con cual trabajemos, ya que podemos trasladar los resultados de una categoría con la que nos resulte más fácil trabajar hacia una categoría equivalente.

### 2.3.3. Categorías monoidales

Para poder representar plenamente la estructura de los circuitos cuánticos, necesitamos extender las propiedades de las categorías. Las categorías monoidales incorporan los componentes necesarios para modelar interacciones entre sistemas cuánticos.

Que una categoría sea monoidal significa que incorpora la operación binaria llamada producto tensorial, $\otimes : \mathbf{C} \times \mathbf{C} \to \mathbf{C}$, la cual actua tanto sobre objetos como morfismos. Esto quiere decir que para los morfimos $f : A \to B$ y $g : C \to D$ el producto tensorial produce la composición paralela [37]:

$$(f \otimes g) : A \otimes C \to B \otimes D.$$

Además, la categoría monoidal incluye:

- Los unitores por izquierda y derecha, que relacionan la identidad con el producto tensorial: $\lambda_A : I \otimes A \to A$ y $\rho_A : A \otimes I \to A$.
- Un asociador, que le atribuye la propiedad asociativa: $\alpha_{A,B,C} : (A \otimes B) \otimes C \to A \otimes (B \otimes C)$.

En la Fig. 2.12 se muestran las representaciones gráficas de estas propiedades. A la izquierda, se muestra como el producto tensorial representa procesos simultáneos sobre distintos sistemas. En el centro y a la derecha, las acciones de los unitores y el asociador. Aunque estos dos últimos no tienen representación explícita, puesto que en las categorías monoidales la identidad tampoco la tiene, su existencia es necesaria para que el formalismo sea coherente.

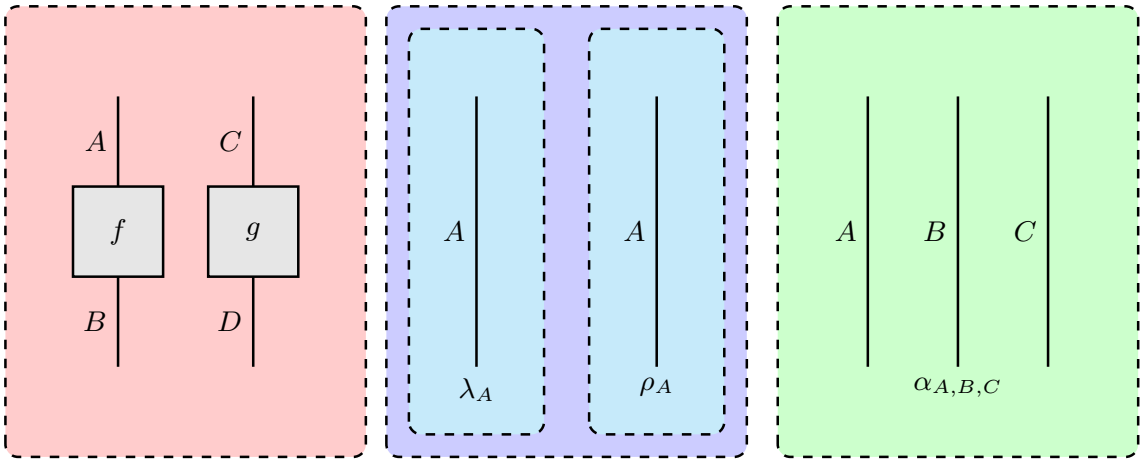



**Figura 2.13:** Representación gráfica de las propiedades del producto tensorial en la categoría monoidal: a la zquierda, el proceso $f \otimes g : A \otimes C \to B \otimes D$; en el centro, los unitores izquierdo $\lambda_A : I \otimes A \to A$, y derecho $\rho_A : A \otimes I \to A$; a la derecha, la propiedad asociativa $\alpha_{A,B,C} : (A \otimes B) \otimes C \to A \otimes (B \otimes C)$.

Dado que el objeto unidad monoidal $I$ representa el sistema trivial, un morfismo $a : I \to A$ puede interpretarse como la preparación de un sistema a un estado $a$, por ello, a estos morfismos se les denomina estados [37]. De forma análoga, un morfismo $x : A \to I$ representa la destrucción del sistema y se interpreta como una medición cuyo resultado ha sido $x$, y son conocidos como efectos. Esto no significa que $x$ sea el único resultado posible, sino que sólo estamos interesados en ese efecto, lo que puede interpretarse como la técnica de post-selección.

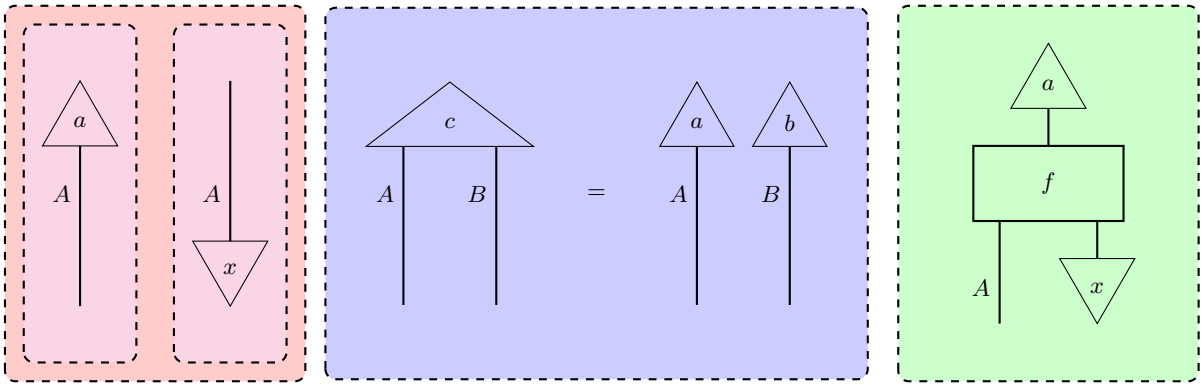



**Figura 2.14:** Representación gráfica de estados y efectos: a la izquierda la creación del estado $a$ y medición del efecto $x$; en el centro, la representación de un estado que puede descomponerse en producto tensorial; y a la derecha, el diagrama de una secuencia con preparación y post-selección.

En la Fig. 2.13 se muestran varios ejemplos de estados y efectos. A la izquierda, como la identidad no tiene representación y no tiene un dominio o codominio visible, los estados y efectos aparecen como triángulos y no cajas. En el centro, se muestra la descomposición

en producto tensorial de un estado separable. Por último, se representa una cadena de eventos donde se prepara un estado $a$, pasa por un proceso $f$ que lo separa en dos sistemas, y uno de ellos es medido con resultado $x$.

### 2.3.4. Categorías monoidales con objetos duales

Como hemos visto en la notación matemática de los circuitos cuánticos —y en la mecánica cuántica en general—, los vectores de estado se representan mediante kets $|\psi\rangle$, que tienen asociados unos duales, los bras $\langle\phi|$, cuya aplicación conjunta, el braket $\langle\phi|\psi\rangle$, da lugar a un número complejo cuyo módulo al cuadrado da la probabilidad de obtener el estado $|\phi\rangle$ al medir $|\psi\rangle$, y viceversa. Las categorías con duales se denominan categorías monoidales rígidas.

Una categoría monoidal se dice rígida por la derecha si cada objeto $A$ tiene un dual derecho $A^r$, y rígida por la izquierda si cada objeto tiene un dual izquierdo $A^l$. Cuando tiene ambas, se denomina categoría monoidal rígida [38]. Además, cada dual debe tener asociado los morfismos de evaluación $\epsilon$ y coevaluación $\eta$:

$$\begin{aligned} &\epsilon^l_A : A^l \otimes A \to \mathrm{id}_A, \quad \eta^l_A : \mathrm{id}_A \to A \otimes A^l, \\ &\epsilon^r_A : A \otimes A^r \to \mathrm{id}_A, \quad \eta^r_A : \mathrm{id}_A \to A^r \otimes A. \end{aligned} \tag{2.25}$$

Y deben satisfacer las identidades conocidas como ecuaciones *snake*:

$$\begin{aligned} &(\mathrm{id}_A \otimes \epsilon^l_A) \circ (\eta^l_A \otimes \mathrm{id}_A) = \mathrm{id}_A, \quad (\epsilon^l_A \otimes \mathrm{id}_{A^l}) \circ (\mathrm{id}_{A^l} \otimes \eta^l_A) = \mathrm{id}_{A^l}, \\ &(\epsilon^r_A \otimes \mathrm{id}_A) \circ (\mathrm{id}_A \otimes \eta^r_A) = \mathrm{id}_A, \quad (\mathrm{id}_{A^r} \otimes \epsilon^r_A) \circ (\eta^r_A \otimes \mathrm{id}_{A^r}) = \mathrm{id}_{A^r}. \end{aligned} \tag{2.26}$$

Al igual que ocurría con los estados y efectos, el hecho de que la identidad no tenga representación gráfica conduce a una visualización característica de estos morfismos. En la Fig. 2.14, a la izquierda, se muestra cómo las cuerdas se 'doblan', dando lugar a las formas especiales cup ($\cup$) y cap ($\cap$), que corresponden a la evaluación y coevaluación, respectivamente. A la derecha, se ilustran las relaciones que deben cumplir estos morfismos en relación con $\mathrm{id}_A$, cuya forma da origen al término.

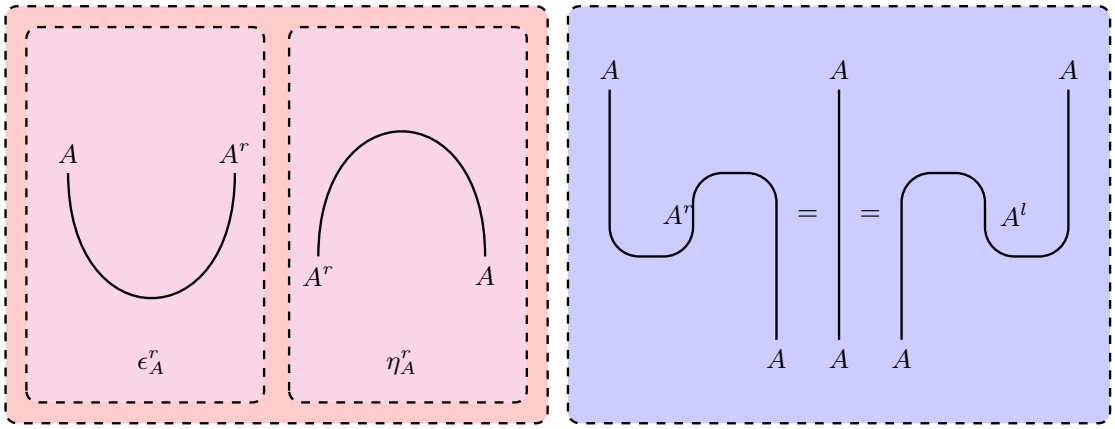



**Figura 2.15:** Representación gráfica de las propiedades de la dualidad: a la zquierda, evaluación $\epsilon^r_A : A \otimes A^r \to \mathrm{id}_A$ (cup $\cup$) y coevaluación $\eta^r_A : \mathrm{id}_A \to A^r \otimes A$ (cap $\cap$); a la derecha, las ecuaciones de *snake*: $(\epsilon^r_A \otimes \mathrm{id}_A) \circ (\mathrm{id}_A \otimes \eta^r_A) = \mathrm{id}_A = (\mathrm{id}_A \otimes \epsilon^l_A) \circ (\eta^l_A \otimes \mathrm{id}_A)$.

Desde el punto de vista cuántico, los caps representan los estados entrelazados, aquellos que no pueden separarse como un producto tensorial de estados independientes (Fig. 2.13); mientras que los cups se relacionan con las mediciones generales, concretamente en las medidas *Positive Operator-Valued Measure* (POVM), que son especialmente útiles para medir estados en interacción [39].

Cuando una categoría monoidal rígida es simétrica respecto al producto tensorial, se denomina categoría compacta cerrada. Un ejemplo de esto es la categoría **FinVect**, que consiste en la categoría **Vect** limitada a espacios vectoriales de dimensión finita. Esta categoría es monoidal compacta cerrada con respecto al producto tensorial de espacios vectoriales:

$$V \otimes W \simeq W \otimes V. \tag{2.27}$$

Además, el espacio dual $V^*$ es dual de $V$ tanto por la izquierda como por la derecha, es decir, $V^* = V^l = V^r$.

Las categorías monoidales con objetos duales juegan un papel fundamental en modelos categóricos como DisCoCat, que trata de conectar la teoría de la información cuántica con la teoría de categorías. Aunque no es una estructura capaz de describir todos los aspectos de la teoría cuántica, donde el producto tensorial no es necesariamente simétrico, es suficiente para formalizar la composición del significado lingüístico en el modelo.

## 2.4. Modelo del significado: DisCoCat

Como introdujimos en la sección 1.1, la filosofía tras de la semántica distribucional —basada en el aprendizaje de representaciones del significado de las palabras mediante vectores semánticos (*embeddings*), a partir de su distribución en los textos— tiene una gran capacidad para capturar el significado de las palabras individuales en función del contexto.

Sin embargo, estas representaciones presentan limitaciones cuando se trata de modelar la composicionalidad del lenguaje. Es decir, aunque los modelos distribucionales pueden capturar el significado de las palabras en un contexto, tienen dificultades para representar cómo se combinan esos significados para formar el significado de unidades estructurales, como frases o párrafos. El orden de las palabras puede alterar completamente el significado de una oración debido a la naturaleza composicional del lenguaje. En este apartado, abordaremos uno de los modelos que intenta combinar la semántica distribucional con la semántica composicional: el *categorical compositional distributional model of meaning* (DisCoCat) [19].

### La gramática de pregrupo

Según la gramática categórica desarrollada por J. Lambek, una gramática $\mathscr{X}$ de un lenguaje con vocabulario $\Sigma$ es un álgebra $\mathscr{X}$ generada libremente por el conjunto parcialmente ordenado de los tipos básicos $\mathscr{B}$ del lenguaje y una asignación de tipos $\mathscr{D}$ que asigna términos de $\mathscr{X}$ a los elementos de $\mathscr{B}$ [40]. En la publicación *Logical Aspects of Computational Linguistics*, Lambek estableció que $\mathscr{X}$ era un pregrupo [41].

De manera más formal, un álgebra de pregrupo $P$ es un monoide parcialmente ordenado $(P,\cdot,1,\leq)$ [19], donde cada elemento $p \in P$ tiene un adjunto izquierdo $p^l$ y derecho $p^r$. Un monoide es un conjunto $(P,\cdot,1)$ con una operación binaria "$\cdot$" entre dos elementos $p, q \in P$, cuyo resultado es un elemento de $P$ y 1 es el elemento unidad: $1 \cdot p = p = p \cdot 1$. El monoide estará parcialmente ordenado si para cada $p, q, r \in P$, se cumple que

$$p \leq q \implies p \cdot r \leq q \cdot r, \quad \text{y} \quad r \cdot p \leq r \cdot q; \tag{2.28}$$

y que para cada $p, p^r, p^l \in P$ se cumple que

$$p \cdot p^r \leq 1 \leq p^r \cdot p, \quad \text{y} \quad p^l \cdot p \leq 1 \leq p \cdot p^l. \tag{2.29}$$

Además, los elementos del pregrupo cumplen las siguientes propiedades:

- Los adjuntos son únicos.
- Los adjuntos revierten el orden de los elementos:

$$p \leq q \implies q^r \leq p^r \quad \text{y} \quad q^l \leq p^l.$$

- La unidad 1, es autoadjunta:

$$1^l = 1 = 1^r.$$

- La multiplicación es autoadjunta:

$$(p \cdot q)^r = q^r \cdot p^r \quad \text{y} \quad (p \cdot q)^l = p^l \cdot q^l.$$

- Adjuntos opuestos se contrarrestan:

$$(p^l)^r = p = (p^r)^l.$$

- Los mismos adjuntos se repiten:

$$p^{ll} \cdot p^l \leq 1 \leq p^r \cdot p^{rr}, \quad p^{lll} \cdot p^{ll} \leq 1 \leq p^{rr} \cdot p^{rrr}, \quad \text{etc.}$$

La gramática de pregrupo adquiere la siguiente estructura: es una tupla $G = (B, \Sigma, \Delta, s)$ donde $B$ y $\Sigma$ son los conjuntos finitos de tipos básicos y vocabulario, respectivamente; $\Delta \subseteq \Sigma \times P_B$ es el diccionario que relaciona las palabras del vocabulario con sus posibles tipos y $s$ es el tipo oración. Siendo el diccionario finito, el lenguaje generado por $G$ viene dado por [42]:

$$L(G) = \Big\{ w_1 w_2 \dots w_n \in \Sigma^n \;\Big|\; n \in \mathbb{N}\,, \quad \exists t \in \prod_{i \leq n} \Delta(w_i) \quad t \leq s \Big\}. \tag{2.30}$$

Es decir, una secuencia de palabras $w_1 w_2 \dots w_n \in \Sigma^n$ se considera gramaticalmente válida cuando existe una entrada del diccionario $(w_i, t_i) \in \Delta$ por cada palabra $w_i$, tal que la secuencia de sus tipos $t_1 t_2 \dots t_n$ se reduce al tipo oración $s$. Esta definición puede simplificarse describiéndola según la teoría de categorías:

$$L(G) = \Big\{ w_1 w_2 \dots w_n \in \Sigma^n \;\Big|\; C_G(w_1 w_2 \dots w_n, s) \neq \emptyset \Big\}, \tag{2.31}$$

donde $C_G$ es una categoría monoidal rígida, cuyos objetos son los elementos de $B + \Sigma$ y cuyos morfismos son las entradas del diccionario $(w, t) \in \Delta$ con $\mathrm{Dom}(w, t) = w$ y $\mathrm{Cod}(w, t) = t$. Esto permite representar gráficamente la reducción sintáctica $t_1 t_2 \dots t_n \leq s$: una secuencia de palabras es gramaticalmente válida si existe un diagrama de cuerdas que contraiga los tipos desde la secuencia de palabras $t_1 t_2 \dots t_n$, hasta el tipo oración $s$. Por ejemplo, la secuencia

$$\{(\mathrm{Bob}, n), (\mathrm{hates}, n^r s n^l), (\mathrm{John}, n)\} \in \Delta \implies n(n^r s n^l)n = \cancel{(nn^r}\overset{1}{\cancel{)}}s\cancel{(n^l n)}\overset{1}{\cancel{\leq}}\, s,$$

se representa con el diagrama de la Fig. 2.15, donde los estados (triángulos) representan el significado de las palabras y la contracción de tipos forman cups. Esto significa que el significado del verbo modifica el significado del sujeto y el objeto de la oración.

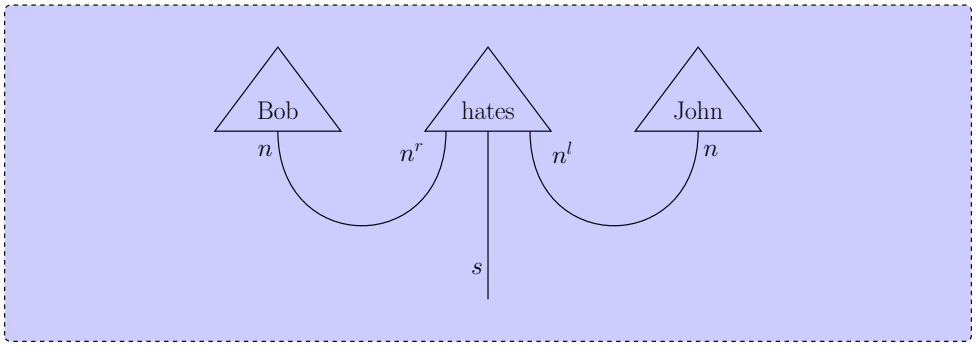



**Figura 2.16:** Diagrama de la oración *"Bob hates John"*. Los cups conectan el verbo con el sujeto y objeto dando lugar a una contracción de tipos, lo que implica una interacción que cambia el significado.

### Conexión entre gramática y significado

El modelo DisCoCat reformula el modelo distribucional utilizando la categoría monoidal compacta **FinVect** en lugar de simplemente trabajar con espacios vectoriales, es

decir, incorpora sus morfismos y el producto tensorial. La categoría se combina con el modelo composicional de la gramática de pregrupo, que también se corresponde con una categoría monoidal compacta [19].

La unión de la categoría **FinVect** con la categoría **P** (el pregrupo), es decir, **FinVect** × **P**, forma otra categoría monoidal compacta. Sus objetos son pares $(V, t)$, donde $V$ es un espacio vectorial y $t$ es un tipo gramatical. Estos pares se denominan espacios de significado, y sus morfismos son pares $(f, \leq) : (V, p) \to (W, q)$.

Además, la categoría está equipada con los morfismos de evaluación y coevaluación:

$$\begin{aligned} &(\epsilon^l_V, \leq) : (V^* \otimes V, p^l \cdot p) \to (\mathbb{R}, 1), \quad (\eta^l_V, \leq) : (\mathbb{R}, 1) \to (V \otimes V^*, p \cdot p^l), \\ &(\epsilon^r_V, \leq) : (V \otimes V^*, p \cdot p^r) \to (\mathbb{R}, 1), \quad (\eta^r_V, \leq) : (\mathbb{R}, 1) \to (V^* \otimes V, p^r \cdot p). \end{aligned} \tag{2.32}$$

Para obtener el significado de una oración completa, es decir, una secuencia de palabras $\overrightarrow{w_1 \dots w_n}$, el modelo emplea un morfismo que sigue la lógica de la definición de lenguaje dada en la Ec. 2.21:

$$(f, \leq) : (W_i \otimes \cdots \otimes W_n, p_i \dots p_n) \to (X, x). \tag{2.33}$$

Aquí, $W_i \otimes \cdots \otimes W_n$ es el espacio tensorial que proviene de la transformación lineal $h$:

$$\overrightarrow{w_1 \dots w_n} := h(\vec{w_i} \otimes \cdots \otimes \vec{w_n}),$$

donde cada palabra $\vec{w_1} \in W_i$ se sustituye por su correspondiente espacio vectorial $W_i$. La secuencia $p_i \dots p_n$ es la concatenación de los tipos gramaticales que satisface $p_i \dots p_n \leq x$, y la transformación $f$ se denomina aplicación *"from-meaning-of-words-to-meaning-of-a-sentence"* [19, p. 16], ya que representa la reducción del espacio de significados de la secuencia de palabras individuales al espacio asignado para el conjunto de la oración:

$$f : W_i \otimes \cdots \otimes W_n \to X.$$

Por ejemplo, volviendo al caso anterior con la oración *"Bob hates John"*, podemos

representar los espacios de significado de las palabras *Bob* y *John* como $(V, n)$ y $(W, n)$, respectivamente; y el verbo *hates* como $(V^* \otimes S \otimes W^*, n^r s n^l)$. El morfismo que reduce el espacio de significado de la secuencia de palabras al espacio atómico correspondiente a la oración, $(S, s)$, se expresa como:

$$(f, \leq) : (V \otimes (V^* \otimes S \otimes W^*) \otimes W, n(n^r s n^l) n) \to (S, s), \tag{2.34}$$

donde $f$, teniendo en cuenta la simetría de **FinVect**, $V \otimes V^* \simeq V^* \otimes V$, es:

$$f = \epsilon_V \otimes \mathrm{Id}_S \otimes \epsilon_W : V \otimes V^* \otimes S \otimes W \otimes W^* \to S. \tag{2.35}$$

La representación gráfica de aplicar $f$ al vector de significado de la secuencia $\overrightarrow{Bob} \otimes \overrightarrow{hates} \otimes \overrightarrow{John}$ (Fig. 2.16) es la misma que el diagrama de reducción sintáctica de la Fig. 2.15:

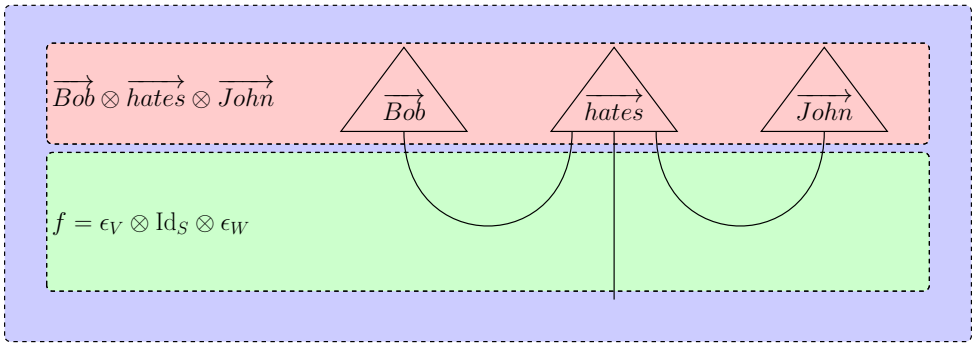



**Figura 2.17:** Diagrama de la contracción $f = \epsilon_V \otimes \mathrm{Id}_S \otimes \epsilon_W : \overrightarrow{Bob} \otimes \overrightarrow{hates} \otimes \overrightarrow{John} \to \vec{s}$.

Si expresamos el verbo transitivo en notación braket, tenemos:

$$\sum_{ijk} c_{ijk} \langle \vec{v}_i | \otimes | \vec{s}_j \rangle \otimes \langle \vec{w}_k | \in V^* \otimes S \otimes W^*, \tag{2.36}$$

entonces la transformación lineal $f$ puede entenderse en términos de productos internos:

$$\begin{aligned} f(\vec{v} \otimes \vec{T} \otimes \vec{w}) = \epsilon_V \otimes \mathrm{Id}_S \otimes \epsilon_W(\vec{v} \otimes \vec{T} \otimes \vec{w}) = \sum_{ijk} c_{ijk} \langle \vec{v}_i | \vec{v} \rangle | \vec{s}_j \rangle \langle \vec{w}_k | \vec{w} \rangle = \\ = \sum_j \Big( \sum_{jk} c_{ijk} \langle \vec{v}_i | \vec{v} \rangle \langle \vec{w}_k | \vec{w} \rangle \Big) | \vec{s}_j \rangle = | \vec{s} \rangle \,. \end{aligned} \tag{2.37}$$

Este resultado es invariante bajo cualquier otro diagrama equivalente resultado de la aplicación de los morfismos de la categoría.

El enfoque categórico del modelo DisCoCat, en el que el significado de una oración puede obtenerse a partir de distintas representaciones diagramáticas equivalentes, permite implementar diversas estrategias de cálculo. En particular, dado que los circuitos cuánticos también pueden describirse mediante categorías monoidales compactas, este enfoque abre la puerta hacia implementaciones del procesamiento del lenguaje natural en sistemas cuánticos. Esta conexión ha dado lugar a librerías específicas como *Lambeq* [21], que aporta las herramientas necesarias para desarrollar aplicaciones de QNLP.

# Capítulo 3

# Clasificación de intenciones con Lambeq

## 3.1. Descripción del dataset

Para la terea de clasificación de intenciones, se utilizará el conjunto de datos SNIPS [20], comúnmente utilizado en esta y otras tareas relacionadas con el procesamiento del lenguaje natural, como el *slot filling* y la detección de entidades. El conjunto está compuesto por aproximadamente 14,000 oraciones, consultas y peticiones a asistentes de voz, distribuidas entre 7 categorías que representan diferentes intenciones de los usuarios.

Dado el coste computacional que implica el procesamiento de los datos para su utilización en sistemas cuánticos, se ha seleccionado un subconjunto compuesto por 4,000 oraciones correspondientes a 4 categorías: *add_to_playlist*, *book_restaurant*, *get_weather* y *search_screening_event*. Este subconjunto se divide de forma balanceada en tres conjuntos:

| Conjunto | Oraciones |
|---|---|
| Entrenamiento | 3,200 |
| Evaluación | 400 |
| Test | 400 |

**Tabla 3.1:** Distribución de oraciones del subconjunto de datos de SNIPS.

En el análisis exploratorio inicial, nos enfocamos en las carasterísticas léxicas del conjunto de entrenamiento después de aplicar un proceso de lematización y eliminación de

signos de puntuación. La Fig. 3.1 muestra la distribución de las diez palabras más frecuentes, de un total de 31,632 tokens. Como se puede observar, las palabras con mayor frecuencia corresponden, en su mayoría, a palabras funcionales —como pronombres, artículos o preposiciones— que sirven para estructurar la frase, pero no aportan mucho contenido semántico por sí solas. A estos términos se les denomina *stop words*. También se pueden incluir en esta definición aquellas palabras que se repiten frecuentemente en todas las categorías.

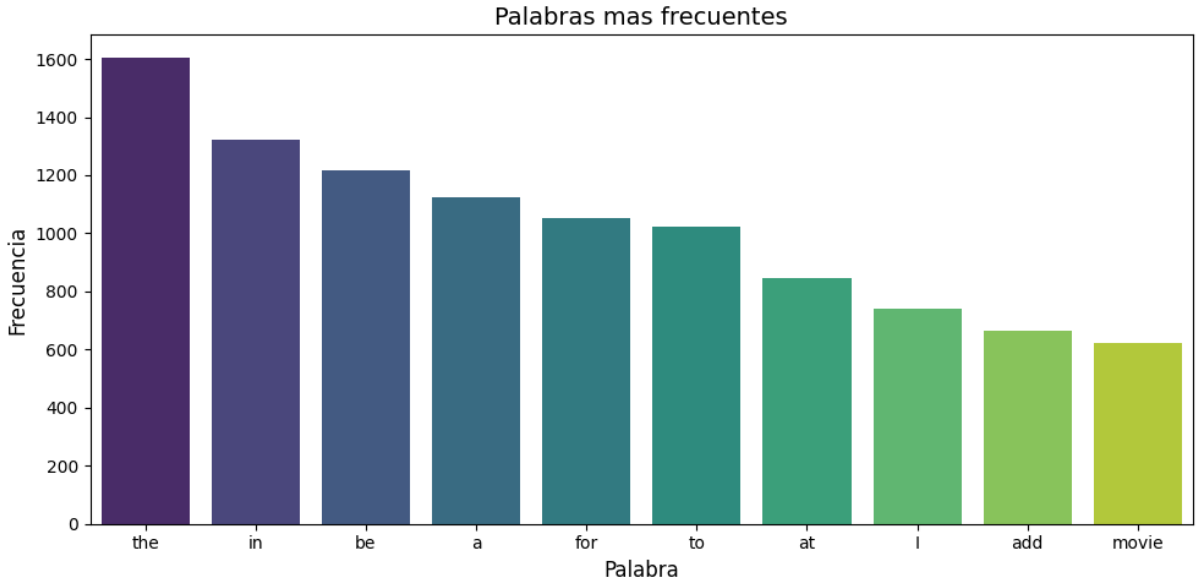



**Figura 3.1:** Palabras mas frecuentes en el conjunto de entrenamiento.

En contraposición a las *stop words*, existen las palabras distintivas, determinantes en el enfoque distribucional del lenguaje. Al analizar el conjunto por categorías, es posible identificar palabras que aparecen exclusivamente en una clase concreta; estas palabras aportan contenido semántico relevante para la clasificación. En la tabla 3.2 se muestran: el número de palabras exclusivas por categoría, cuántas de ellas se repiten al menos una vez y la medía de tokens por oración, un valor relacionado directamente con la complejidad sintáctica de las frases y que influye directamente en el coste de construir los circuitos cuánticos. Cuanto mayor es la media, mas compleja tiende a ser la estructura del circuito asociado.

| Categoría | Palabras exclusivas | Exclusivas repetidas | Media de Tokens |
|---|---|---|---|
| *add_to_playlist* | 1,564 | 386 | 9.20 |
| *book_restaurant* | 919 | 204 | 11.94 |
| *get_weather* | 698 | 134 | 9.57 |
| *search_screening_event* | 611 | 119 | 8.83 |

**Tabla 3.2:** Distribución de palabras exclusivas por cada categoría y la media de número de tokens en el conjunto de entrenamiento.

El siguiente paso es la implementación del modelo DisCoCat sobre este conjunto de datos. Para ello, se utilizará la librería de Python *Lambeq*.

## 3.2. Flujo de trabajo de Lambeq

Lambeq es un *framework* de código abierto, desarrollado inicialmente por *Cambridge Quantum* y actualmente mantenido por el equipo de QNLP de Quantinuum. Esta librería de Python aporta las herramientas necesarias para implementar técnicas de procesamiento de lenguaje natural cuántico basadas en modelos composicionales con objetivo de explorar las ventajas del enfoque categórico monoidal [22]:

- Compatibilidad con la computación cuántica: Como se vió en el capítulo 2, la equivalencia estructural de los circuitos cuánticos y las composiciones sintácticas permite codificar directamente estructuras lingüísticas en circuitos y operar con ellos.
- Interpretabilidad: Las operaciones matemáticas que rigen la composición del significado se corresponden directamente con principios lingüísticos, lo que favorece la comprensión del modelo, trazabilidad y modificación.
- Generalización y flexibilidad: Es un marco abstracto que permite trabajar con distintas representaciones del lenguaje de manera unificada.
- Aplicaciones interdisciplinares: La composicionalidad resulta útil para otras disciplinas como la teoría de sistemas o la bioinformática.

La arquitectura del *framework* está diseñada de forma modular, diferenciando las distintas fases del proceso necesarias para realizar tareas de procesamiento de lenguaje natural cuántico. Cada módulo —desde el análisis de la oración, hasta la transformación de los circuitos cuánticos y el entrenamiento del modelo— está encapsulado jerárquicamente

de manera que facilita la modificación, extensión o sustitución de sus elementos. Además, es compatible con las plataformas más conocidas de computación cuántica como *Qiskit*, *Pennylane* o *tket*; y entornos clásicos para el aprendizaje como *Pytorch* o *TensorFlow*, lo que permite realizar experimentos tanto cuánticos como híbridos [21].

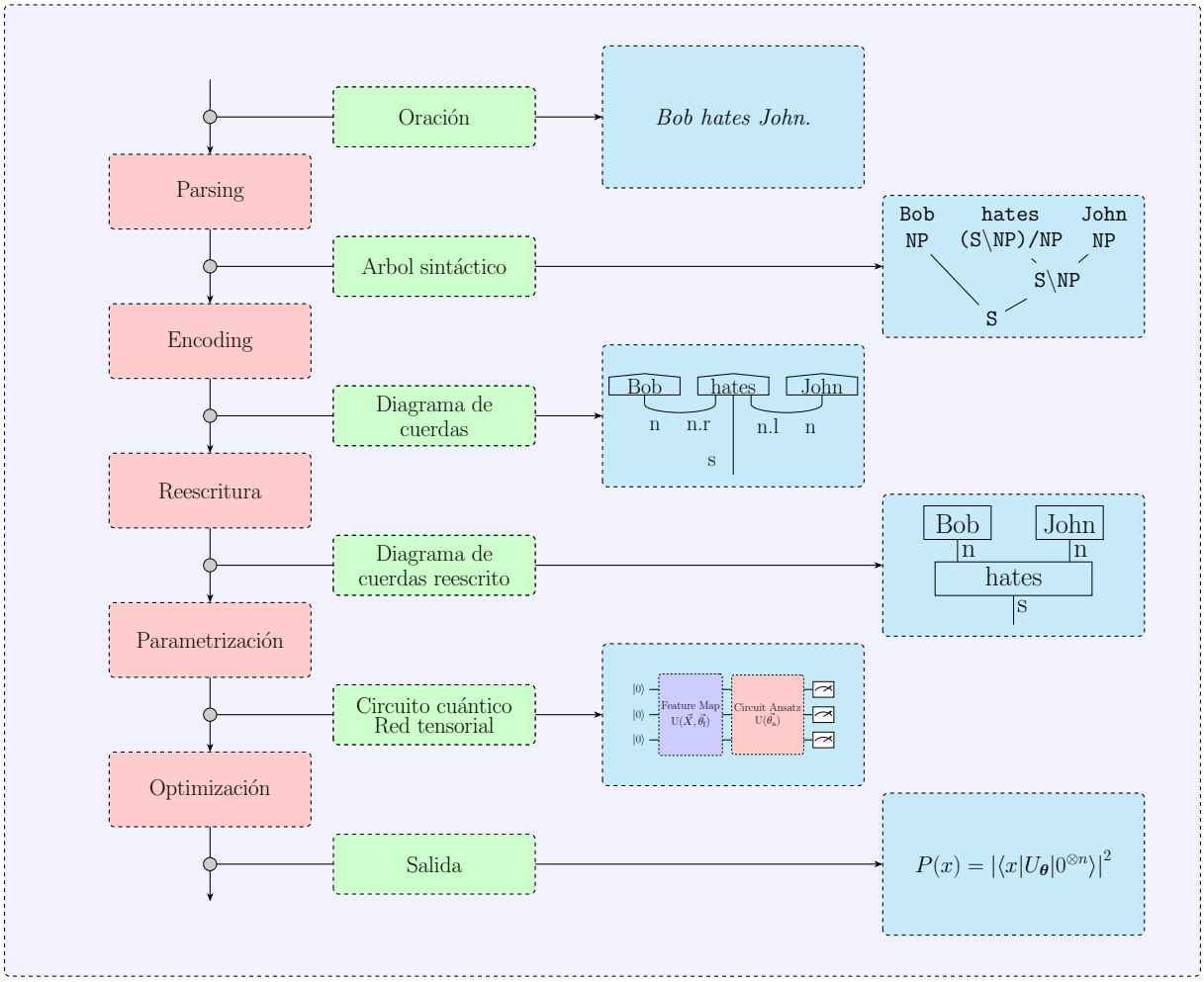



**Figura 3.2:** Flujo de trabajo de Lambeq para modelos DisCoCat con circuitos variacionales.

En la figura 3.2 se muestra el flujo de trabajo de Lambeq para trabajar con modelos DisCoCat y circuitos variacionales, organizado en los distintos módulos correspondientes a cada fase del proceso. De manera general, la oración es procesada en una fase de *Parsing* que genera un árbol sintáctico. La información es transformada en un diagrama de cuerdas, conforme a la estructura composicional de la oración, en una fase de *Encoding*. Dicho diagrama puede ser reescrito mediante un conjunto de reglas para eliminar la interacción entre palabras poco relevantes o reducir la complejidad para mejorar la eficiencia. El diagrama simplificado se convierte en un circuito cuántico parametrizado, o una red

tensorial, siguiendo el esquema establecido por un determinado Ansatz. Finalmente, se genera un modelo a partir de los circuitos cuánticos parametrizados y se entrena utilizando técnicas de optimización clásicas o cuánticas.

En la siguiente sección, seguiremos detalladamente este flujo de trabajo para entrenar un modelo DisCoCat. Explicaremos el funcionamiento de cada fase y la aplicaremos a nuestro conjunto de datos.

## 3.3. Modelo DisCoCat

### 3.3.1. Parsing

El objetivo del *Parsing* es analizar la sintaxis de la oración para construir una representación jerárquica que refleje las relaciones entre las palabras. Este paso es esencial cuando se trata de modelos categóricos como DisCoCat, pero no es estrictamente necesario si se van a emplear modelos que no requieran una estructura sintáctica detallada. En estos casos, las técnicas habituales del procesamiento del lenguaje natural, como la tokenización o el filtrado de *stop words*, pueden ser suficientes para preparar la información necesaria utilizada para construir los diagramas.

El objeto encargado de realizar el análisis sintáctico es el Parser. Este componente asigna una categoría gramatical específica a cada palabra y las relaciona, devolviendo el árbol sintáctico de la oración. Por defecto, Lambeq realiza este análisis con Bobcat, un *Combinatory Categorial Grammar* (CCG) Parser, como alternativa al formalismo de J. Lambek, ofreciendo mayor flexibilidad para construir estructuras tipo pregrupo.

La arquitectura de Bobcat está basada en el C&C (Clark & Curran) Parser [43], que realiza la tarea en dos partes:

1. Un modelo BERT entrenado con CCGBANK, un corpus de derivaciones CCG, realiza un *supertagging* sobre cada palabra en función de su categoría gramatical y su contexto en la oración.

2. Se utiliza un Chart Parser que almacena las categorías de las palabras y explora sus

posibles combinaciones, donde un algoritmo A* optimiza la búsqueda de las más prometedoras y genera el árbol sintáctico.

Aunque Lambeq permite la posibilidad de integrar cualquier otro Parser externo mediante subclases que hereden de la clase `CCGParser`, en este modelo se ha empleado Bobcat.

### 3.3.2. Encoding

Para obtener un diagrama a partir de la información generada en la fase anterior, Lambeq utiliza DisCoPy [44], una librería de Python diseñada para implementaciones basadas en categorías monoidales, como gramáticas formales y circuitos cuánticos. DisCoPy permite construir diagramas monoidales a partir del árbol sintáctico o de otras reglas composicionales.

La estructura resultante de del análisis CCG sigue el formalismo de Lambek (Fig. 3.3), interpretando dicha notación en términos de gramática de pregrupos, lo que da lugar al tipo de diagramas que se requieren para modelos composicionales como DisCoCat:

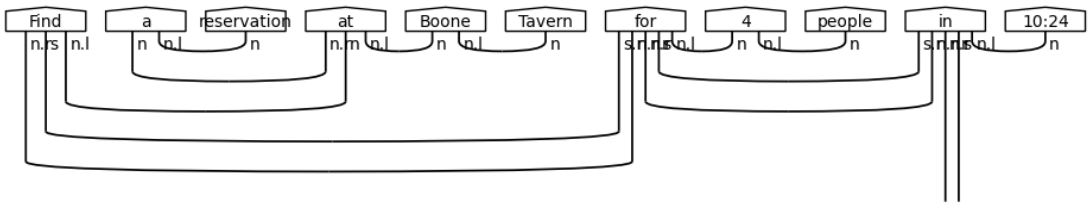



**Figura 3.3:** Ejemplo de diagrama generado a partir del análisis CCG de una oración del conjunto de entrenamiento.

Sin embargo, también existe la posibilidad de utilizar otros esquemas de generación de diagramas que no requieren un análisis composicional detallado. Por ejemplo, `cups_reader` es una subclase de `LinearReader` que asigna a cada palabra, independientemente de su función sintáctica, los tipos necesarios para establecer una composición secuencial:

De esta forma, como veremos en la sección 3.4, es posible explorar distintas alternativas basadas en representaciones más simples. Las nuevas reglas de composición pueden inte-

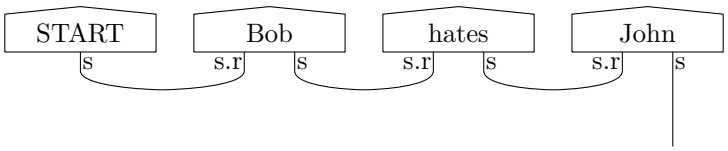



**Figura 3.4:** Diagrama generado por el Reader *cups_reader* de Lambeq.

grarse mediante la extensión de las clases `Reader` o `LinearReader`.

Dado que el conjunto contiene ejemplos recogidos de interacciones reales, la naturaleza espontánea del lenguaje induce una alta variabilidad en las construcciones sintácticas que se refleja directamente en los diagramas generados.

Para cuantificar esta variabilidad, se define un indice basado en la entropía de Shannon $H_s$ [24]:

$$H_s = -\sum_i p_i log_2(p_i), \tag{3.1}$$

donde $p_i$ es la frecuencia relativa de la estructura. Este indice, que denominaremos índice de dominancia (ID), se calcula como:

$$ID = 1 - \frac{H_s}{log_2(N)}, \tag{3.2}$$

y refleja cómo se distribuyen las $N$ estructuras sintácticas que aparecen en el conjunto. Valores cercanos a 1 indican que una sola estructura es predominante, mientras que valores cercanos a 0 señalan que las estructuras se reparten de forma más uniforme entre todas las posibilidades.

En la Tabla 3.3 se presenta el número total de estructuras únicas identificadas en el conjunto de entrenamiento, tanto globalmente como por categorías, junto al índice de dominancia. El estudio revela un total de 2,197 diagramas distintos en todo el conjunto, con un bajo valor del indice de dominacia a nivel global (ID = 0.0521), lo que indica que están distribuidas de manera uniforme. Por otro lado, el análisis por categorías muestra menor diversidad, siendo *add_to_playlist* la categoría con mayor concentración de estructuras (ID = 0.3134), lo que indica que una parte de los ejemplos se concentran en algunas categorías. Este indicador proporciona una base para evaluar el impacto que los esquemas

de construcción de diagramas, así como la fase de reescritura, afectan a la diversidad de los patrones composicionales y al rendimiento del modelo.

| Categoría | Estructuras | ID |
|---|---|---|
| Global | 2,197 | 0.0521 |
| *add_to_playlist* | 422 | 0.3134 |
| *book_restaurant* | 691 | 0.1655 |
| *get_weather* | 607 | 0.1968 |
| *search_screening_event* | 562 | 0.2141 |

**Tabla 3.3:** Número de estructuras únicas e índice de dominancia del conjunto de entrenamiento a nivel global y por categorías en el modelo DisCoCat.

### 3.3.3. Reescritura

En un análisis sintáctico es crucial conservar todas las palabras que aparecen en una oración, ya que es necesario discernir correctamente la función gramatical de cada una para obtener la estructura que forman. Por esta razón, no se recomienda aplicar técnicas como eliminación de *stop words* o lematización antes de la fase de *encoding*. Tanto las estructuras gramaticales como los diagramas generados a partir de ellas pueden ser muy complejos, lo que dificulta la ejecución de los circuitos cuánticos posteriores, cuya eficiencia depende del número de qubits y operaciones.

Para abordar esta problemática, existen herramientas para modificar los diagramas —*rewriters*— con el objetivo de simplificar o encontrar representaciones equivalentes que reduzcan los recursos necesarios para las etapas posteriores. Los procesos de transformación en la fase de reescritura se producen a dos niveles:

- Nivel de Caja: Son transformaciones que actuan sobre cada caja de forma individual, modificando su contenido o eliminándola por completo. Son útiles para aplicar técnicas como la lematización, *stemming*, remplazo de entidades o eliminación de *stop words*. Las transformaciones que aplicaremos a nuestro caso práctico, y ya están implementadas en Lambeq, son:

  - *Determiner, Auxiliary, Connector*: Elimina las cajas cuyo contenido sea un determinante, un verbo auxiliar o un conector, y las reemplaza por caps.

- *Prepositional_phrase*: Simplifica las relaciones entre palabras de las frases preposicionales, reduciendo la dimensión del codominio de las preposiciones mediante el uso de caps.

- Nivel de Diagrama: Son las transformaciones que modifican la estructura global del diagrama, realizando cambios que necesitan una visión más general. Las transformaciones ya implementadas y que también aplicaremos a nuestro caso son:

  - `RemoveCupsRewriter`: Utiliza las reglas de dualidad categóricas para modificar el dominio y codominio de las cajas, reduciendo así el número de cups. Esto disminuye la anchura del diagrama, lo que implica número menor de qubits para codificar la oración.
  - `UnifyCodomainRewriter`: Modifica el codominio del diagrama unificando la dimensionalidad de la salida de todo el conjunto.

Se pueden crear nuevas reglas de reescritura, tanto a nivel de caja como de diagrama, extendiendo las clases `RewriteRule` y `DiagramRewriter`, respectivamente. Gracias a esta característica, es posible aplicar las técnicas de preprocesamiento tradicionales sin alterar la composicionalidad de las oraciones. En este estudio, se han implementado las siguientes reglas personalizadas a nivel de caja, utilizando la librería spaCy [45]:

- `LemmaRewriteRule`: Sustituye el contenido de las cajas por su lema correspondiente.
- `NumbersRewriteRule`: Reemplaza el contenido de las cajas que contienen valores numéricos por la etiqueta `<NUM>`.
- `TimeRewriteRule`: Reemplaza el contenido de las cajas por `<HOUR>`, o `<DATE>`, dependiendo de si se trata de a una hora o una fecha.

Para finalizar, se ha aplicado la regla nativa `UnknownWordsRewriteRule`. Su función es identificar las palabra que no aportan contenido relevante debido a su baja frecuencia en el conjunto de entrenamiento —en este caso, se ha elegido un umbral mínimo de aparición de 2— y sustituir el contenido de sus cajas por la etiqueta `<UNK>`. De este modo se simplifica el modelo, evitando representar palabras para las que no se ha podido aprender una codificación, o que no contribuyen significativamente al rendimiento general.

La figura 3.5 muestra el resultado de aplicar todas las transformaciones citadas al diagrama de la Fig. 3.3:

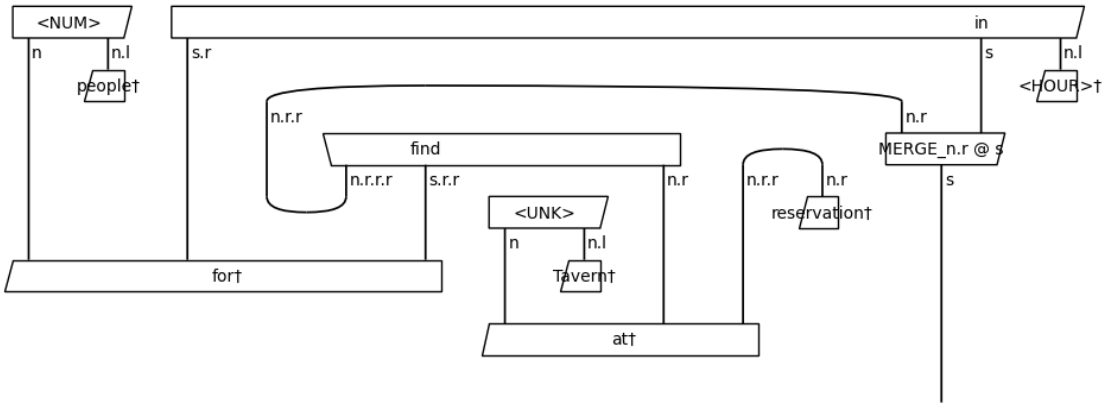



**Figura 3.5:** Diagrama simplificado de la figura 3.3 después aplicar las transformaciones de la fase de reescritura.

El diagrama resultante adquiere una forma simplificada que conserva el contenido semántico de la oración y da lugar a un circuito cuántico con una ejecución más eficiente.

Sin embargo, la mejora en la eficiencia tiene un coste. La Tabla 3.4 muestra cómo el proceso ha incrementado el número de estructuras únicas en el conjunto de entrenamiento a 3,034, lo que representa casi la totalidad del tamaño del conjunto e indica un aplanamiento de la distribución (ID = 0.0058). El análisis por categorías revela resultados similares, con un número de estructuras únicas comparable al tamaño de cada subconjunto, por lo que no predomina ningún patrón composicional.

| Categoría | Estructuras | ID |
|---|---|---|
| Global | 3,034 | 0.0058 |
| *add_to_playlist* | 764 | 0.1756 |
| *book_restaurant* | 793 | 0.1678 |
| *get_weather* | 750 | 0.1818 |
| *search_screening_event* | 728 | 0.1893 |

**Tabla 3.4:** Número de estructuras únicas e índice de dominancia del conjunto de entrenamiento a nivel global y por categorías después de la fase de reescritura.

### 3.3.4. Parametrización

La fase de parametrización es la última transformación del proceso, y consiste en convertir el diagrama en un objeto computable. En este paso se toman las decisiones de bajo nivel mediante un *ansatz*. En nuestro caso, la construcción en circuitos cuánticos depende de los siguientes parámetros:

- *ob_map*: Define el mapeo que asigna un número de qubits a cada tipo gramatical atómico presente en el diagrama.
- *single_qubit_rotations*: Número de rotaciones empleadas en el *embedding* de las palabras cuya representación depende de un único qubit.
- *n_layers*: El número de capas en las que se aplica el *ansatz*.
- *discard*: Indica si la información del circuito es extraída mediante descarte de qubits o mediante post-selección.

La arquitectura del circuito depende del *ansatz* elegido. Pueden definirse nuevos *Ansätze* extendiendo la clase `CircuitAnsatz`, pero también incluye algunas opciones predeterminadas como `IQPAnsatz` o `StronglyEntanglingAnsatz`. Debido a mayor eficiencia computacional, en la parte práctica de este trabajo se utiliza `IQPAnsatz`.

La elección del número de qubits asignados a los tipos gramaticales atómicos que codifican las palabras, como los sustantivos (`N`), afecta directamente a la anchura del circuito. Además, el asignado a la salida del circuito, correspondiente al tipo gramatical atómico (`S`), debe ser elegido en función del número de categorías presentes en la clasificación. La salida del circuito se representa con un vector de $2^n$ elementos, donde $n$ el número de qubits asignados a `S`. En nuestro caso, si queremos que nuestro modelo clasifique entre las cuatro etiquetas disponibles, debe asignarse 2 qubits a la salida.

En la Fig. 3.6 se muestra el circuito cuántico generado mediante el `IQPAnsatz` a partir del diagrama de la Fig. 3.5. Se ha optado por representar el tipo gramatical atómico sustantivo (`N`) con un único qubit y el tipo oración (`S`) con dos. El *embedding* de las palabras atómicas se realiza con tres rotaciones ($r = 3$), y el bloque IQP —que establece la correlación cuántica de las palabras no atómicas, así como las interacciones entre distintas

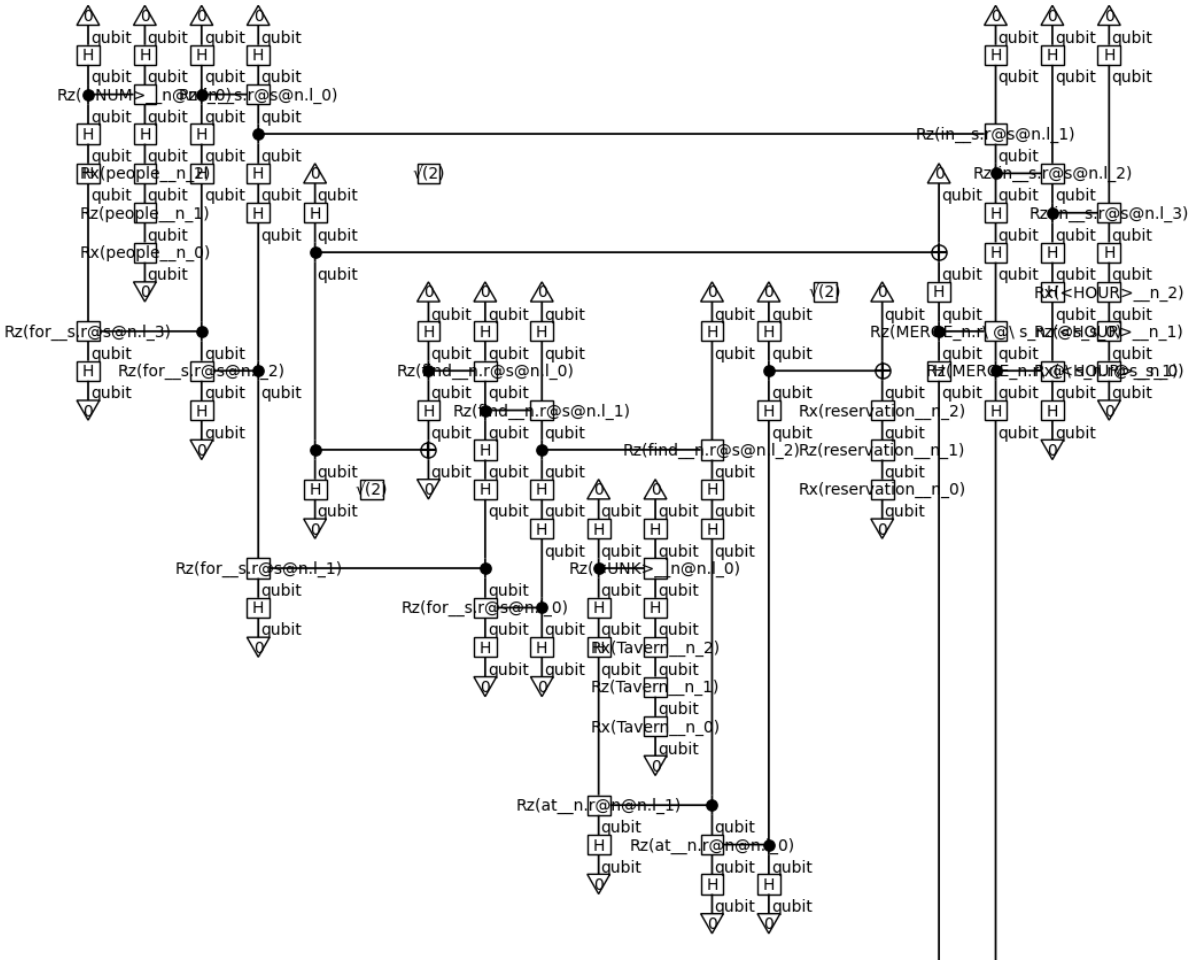



**Figura 3.6:** Circuito de la Fig. 3.5 con el *ansatz* `IQPCircuit`. Donde `N` y `S` están codificados en uno y dos qubits, respectivamente. Se emplean tres rotaciones en las palabras atómicas ($n = 3$) y el bloque IQP sólo se aplica una vez ($l = 1$).

palabras— se aplica una sola vez ($l = 1$). Finalmente, se lleva a cabo una post-selección dejando dos qubits libres, en correspondencia con la dimensión asignada al tipo `S`.

En este punto, las oraciones están representadas en una estructura adecuada para construir un modelo entrenable. La clase `Model` gestiona los parámetros extraídos del conjunto de circuitos, sus pesos —los ángulos de rotación en las puertas parametrizadas—, así como los métodos necesarios para operar con el modelo, como *forward pass*, que devuelve la predicción.

Lambeq permite seleccionar entre distintos tipos de modelos dependiendo del enfoque del experimento: Simulaciones cuánticas en ordenadores clásicos, ya sea mediante cálculos exactos de las transformaciones unitarias aplicadas por los circuitos o mediante simula-

ciones no deterministas basadas en multiples ejecuciones (*shots*), con o sin la presencia de ruido; ejecuciones en *hardware* cuántico real; y modelos clásicos —o híbridos— entrenables, como redes neuronales implementadas en *PyTorch*.

Debido al coste computacional de la simulación de circuitos con ruido, en este trabajo se opta por emplear simulaciones exactas con un *backend* clásico basado en *NumPy*, el cual permite entrenar modelos de forma determinista y en un tiempo menor. En este escenario, la salida del circuito es un vector de probabilidades, donde cada componente indica la probabilidad de que la oración pertenezca a una determinada categoría. Esto implica que las etiquetas originales deben transformarse mediante codificación *one-hot*, lo que permite compararlas directamente con la salida del modelo durante el entrenamiento.

### 3.3.5. Optimización

El proceso de entrenamiento del modelo sigue el esquema estándar: se minimiza una función de pérdida mediante un ciclo de *epochs* y *batches*, con validación sobre un conjunto de evaluación. En Lambeq, este proceso se estructura en torno a dos clases principales: `Dataset` y, para nuestro caso particular, `QuantumTrainer`, una extensión de la clase `Trainer` preparada para trabajar con el modelo *NumPy*.

La clase `Dataset` controla los datos de entrada del entrenamiento e implementa las funciones de *batching* y *shuffling*, si procede, en cada *epoch*. La clase `Trainer` —y sus subclases— controla todas las facetas del entrenamiento. Recibe el modelo que se va a entrenar, los datos en forma de instancias de `Dataset`, el número de *epochs*, la función de evaluación, el optimizador y la función de pérdida.

Cada entrenamiento realizado en este trabajo se ha configurado con una duración de 1,000 *epochs* y un tamaño de *batch* de 80, utilizando el *accuracy* como criterio de evaluación. En Lambeq, tanto el optimizador como la función de pérdida están implementados como clases. Se emplea el algoritmo SPSA —descrito en la sección 2.2.4— mediante la clase `SPSAOptimizer`. Como función de pérdida se ha utilizado la entropía cruzada, implementada en la clase `CrossEntropyLoss`, aunque ha sido necesario redefinirla debido

a un error en el código original de la librería: la salida del circuito devuelve un vector de probabilidades con la forma `[2,n]`, donde `n` es el número de qubits asignados al tipo gramatical `S` en la construcción del circuito, mientras que la función de pérdida espera una salida aplanada. Este problema no se presenta en tareas de clasificación binaria, ya que en ese caso las dimensiones sí coinciden.

Las curvas de evolución de la función de pérdida y el *accuracy* de los dos modelos DisCoCat entrenados se muestran en la Fig. 3.7: ambos modelos tienen asignados un único qubit para los tipos gramaticales atómicos, dos para la oración y tres rotaciones para palabras atómicas. La diferencia entre ellos radica en el número de capas del bloque IQP: el modelo $\text{DisCoCat}_1$ tiene una única capa ($l = 1$) y el modelo $\text{DisCoCat}_2$ tiene dos ($l = 2$). Las fluctuaciones observadas son consecuencia de la aleatoriedad presente en el algoritmo SPSA, que induce una evolución estocástica de los resultados. En cuanto al rendimiento, el modelo $\text{DisCoCat}_2$ muestra mejores resultados, aunque con un mayor sobreajuste en comparación con el modelo $\text{DisCoCat}_1$. Este último presenta una evolución similar entre el conjunto de entrenamiento y evaluación, lo que sugiere una mejor generalización.

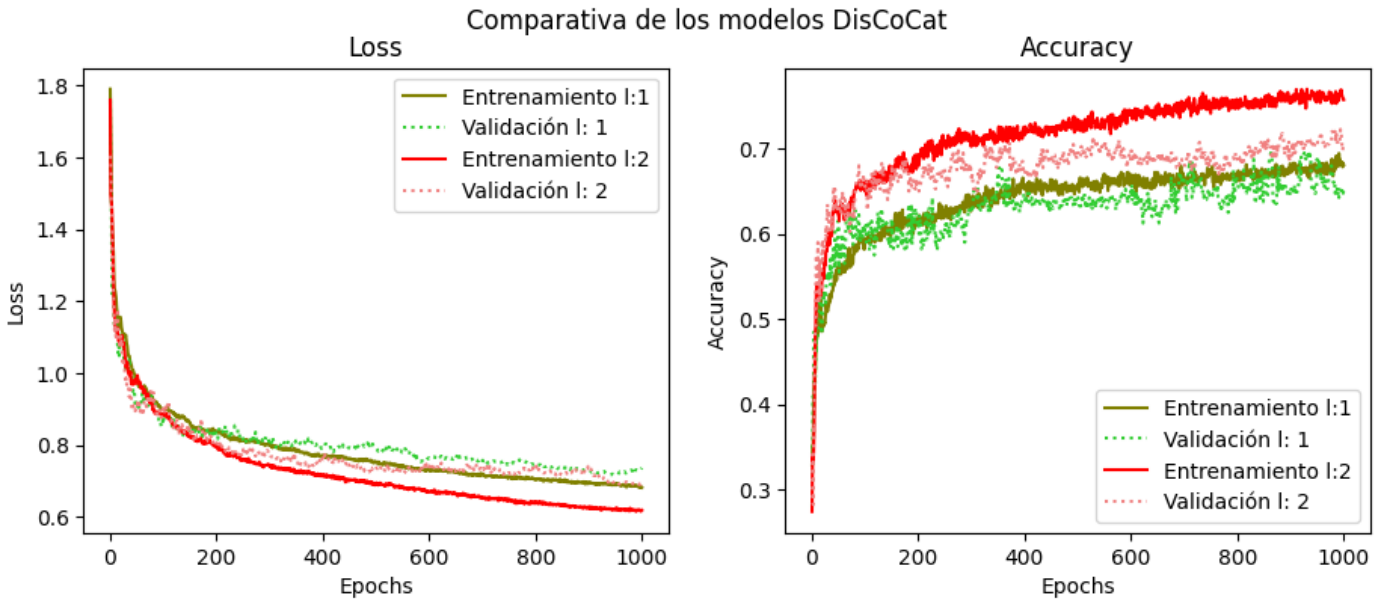



**Figura 3.7:** Comparación de la función de pérdida y el *accuracy* durante el entrenamiento y la validación a lo largo de 1,000 *epochs* para los modelos $\text{DisCoCat}_1$ ($l = 1$) y $\text{DisCoCat}_2$ ($l = 2$). El modelo $\text{DisCoCat}_2$ obtiene mejores valores en ambas métricas, pero presenta un mayor sobreajuste en comparación con su análogo $\text{DisCoCat}_1$, cuya evolución en el *accuracy* es similar tanto en entrenamiento como en evaluación.

En la tabla 3.5 se presentan los datos más relevantes entre ambos modelos. El modelo

$\text{DisCoCat}_2$ requiere un número mayor de parámetros, debido a la duplicidad del bloque IQP, lo que incrementa la profundidad del circuito y, por ello, el tiempo de entrenamiento. Para la evaluación final en el conjunto de prueba, se seleccionará el modelo con el mínimo valor de pérdida, incluso si su *accuracy* es ligeramente inferior, como muestra la columna Acc. (Min Loss). Minimizar la pérdida maximiza la amplitud de probabilidad asociada a la clasificación correcta, lo cual es especialmente relevante en la implementación del modelo en sistemas cuánticos reales, en presencia de ruido.

| Modelo | Parámetros | Tiempo de Entrenam. | Max Acc. | Min Loss | Acc. (Min Loss) |
|---|---|---|---|---|---|
| $\text{DisCoCat}_1$ | **4716** | **3d 10h 51m 41s** | 0.70 | 0.7158 | 0.69 |
| $\text{DisCoCat}_2$ | 7461 | 5d 5h 35m 27s | **0.72** | **0.6884** | **0.71** |

**Tabla 3.5:** Comparativa de los datos de entrenamiento de los modelos $\text{DisCoCat}_1$ y $\text{DisCoCat}_2$: Número de parámetros, tiempo de entrenamiento total, *accuracy* máxima alcanzada, valor de pérdida mínima y *accuracy* en mínima pérdida.

### 3.3.6. Resultados

La evaluación final de los modelos con menor pérdida se realiza según sus puntuaciones de *accuracy*, *precicion*, *recall* y la puntuación F1. Los resultados, presentados en la tabla 3.6, revelan que, aunque el modelo $\text{DisCoCat}_2$ obtuvo mejores métricas durante la fase de validación, su rendimiento en el conjunto de prueba es inferior. En cambio, la mejor capacidad de generalización del modelo $\text{DisCoCat}_1$ le permite alcanzar mejores valores, además de coherentes con sus resultados durante la evaluación. Puesto que ambos modelos requieren el mismo número de qubits (108) para su ejecución, las mejores prestaciones del modelo $\text{DisCoCat}_1$, junto al menor número de parámetros y tiempo de entrenamiento, señalan que es la opción mas adecuada.

| Modelo | Qubits | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|
| $\text{DisCoCat}_1$ | 108 | **0.69** | **0.69** | **0.69** | **0.68** |
| $\text{DisCoCat}_2$ | 108 | 0.67 | 0.65 | 0.67 | 0.66 |

**Tabla 3.6:** Resultados de los modelos $\text{DisCoCat}_1$ y $\text{DisCoCat}_2$ aplicados al conjunto de prueba, junto al número de qubits necesarios para la ejecución completa del modelo, desde el entrenamiento hasta la prueba.

## 3.4. Modelo Box of Words

Como alternativa al modelo anterior, basado en el análisis categórico-gramatical (CCG) para la construcción de diagramas, se presenta el modelo *Box of Words* —denominado así por su similitud con el clásico *Bag of Words*—. El modelo prescinde del análisis gramatical y opta por una composionalidad limitada, construida de forma directa. Este enfoque permite estudiar hasta qué punto una estructura composicional menos detallada, dentro del marco DisCoCat, puede capturar el significado de la oración.

Dado que este modelo no depende de un análisis sintáctico, se puede prescindir de la fase de *Parsing*. Esto abre la posibilidad de construir los diagramas después de aplicar tanto la lematización como la eliminación de *stop words*.

### 3.4.1. Encoding

Durante la fase de *encoding*, aprovechamos el enfoque categórico para generar los diagramas, entendiendo los tipos gramaticales únicamente como elementos de construcción, sin atender a su función sintáctica en la oración. Para ello, definimos la clase `BoxReader`, una subclase de `Reader` creada como versión personalizada de `CombReader`, presentada en una de las guías disponibles en la documentación [22], orientada a la extensión de la librería. Nuestro constructor asigna a todos los *tokens*, tras lematización y eliminación de *stop words* previo, el tipo gramatical `N` y los conecta de forma uniforme a una única caja cuyo codominio representa el significado de la oración `S` (Fig. 3.8):

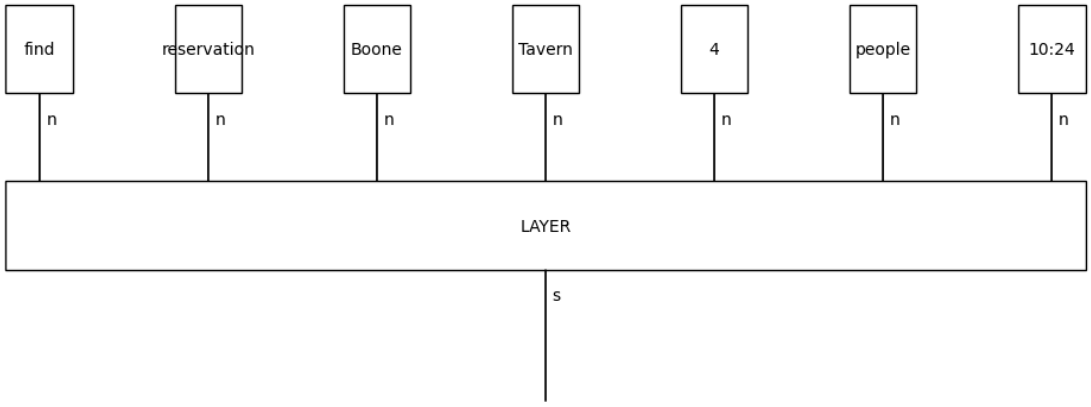



**Figura 3.8:** Ejemplo de diagrama generado por `BoxReader` de una oración del conjunto de entrenamiento.

Este diseño minimiza la contribución composicional favoreciendo a la componente distribucional; es decir, mantiene una estructura composionalidad mínima y se centra en el aporte individual de cada palabra al significado global.

La nueva representación implica que el número de estructuras posibles se limita al número de *tokens* que componen la oración, circunstancia que se ve reflejada en los índices de dominancia (ID) mostrados en la tabla 3.7. Frente a las 2,197 estructuras encontradas en la fase de *encoding* del modelo DisCoCat para el conjunto de entrenamiento, ahora se obtiene un valor significativamente inferior: 14, y en una concentración mucho mayor (ID = 0.3075). El análisis por categoría también muestra una mejora en todas ellas, especialmente si se tiene en cuenta que ahora se distribuyen entre un número total de estructuras mucho más reducido.

| Categoría | Estructuras | ID |
|---|---|---|
| Global | 14 | 0.2339 |
| *add_to_playlist* | 11 | 0.3075 |
| *book_restaurant* | 13 | 0.2255 |
| *get_weather* | 12 | 0.2590 |
| *search_screening_event* | 10 | 0.2973 |

**Tabla 3.7:** Número de estructuras únicas e índice de dominancia del conjunto de entrenamiento a nivel global y por categorías en el modelo *Box of Words*.

Con un conjunto de estructuras significativamente menor, es posible visualizar la distribución en una gráfica (Fig. 3.9). En ella, se muestra que algunas estructuras son comunes a todas las categorías y presentan una alta frecuencia de aparición, mientras que otras dos muestran una predominancia de las categorías *get_weather* y *search_screening_event*, especialmente la primera. Por otro lado, algunas estructuras son residuales, con una frecuencia de aparición total muy reducida.

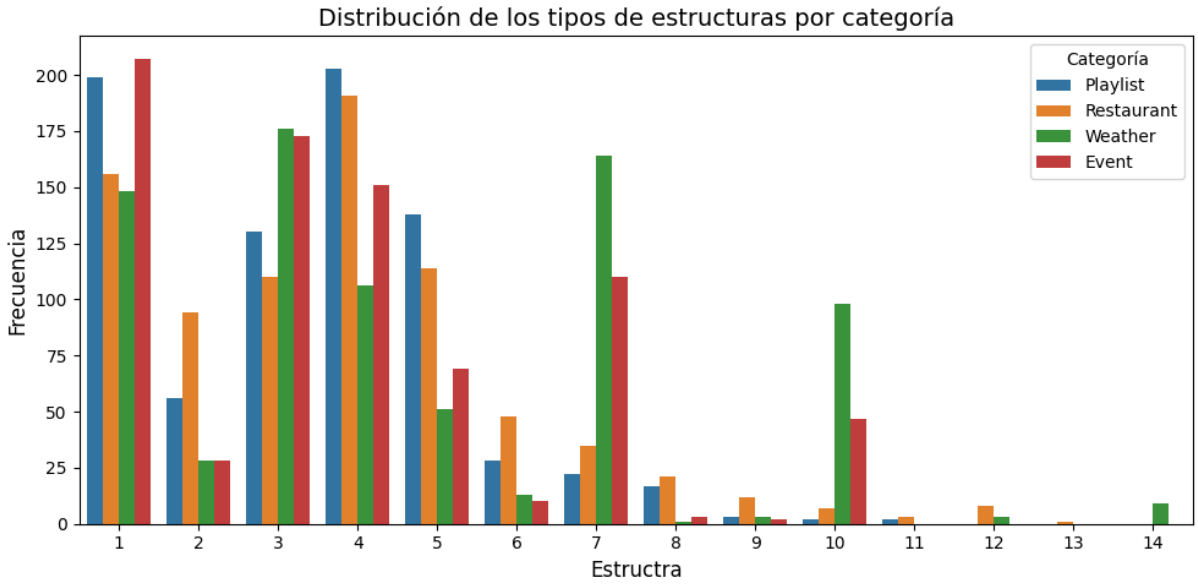



**Figura 3.9:** Distribución de las distintas estructuras generadas por el modelo *Box of Words* separadas por categoría.

### 3.4.2. Reescritura

Al haber eliminado las *stop words*, la fase de reescritura se simplifica considerablemente. No es necesario aplicar reglas que eliminan cajas correspondientes a determinantes, verbos auxiliares, conectores o simplifican frases preposicionales, como sí ocurría en el modelo DisCoCat. En consecuencia, la regla `RemoveCupsRewriter` tampoco es necesaria. Además, el enfoque minimalista del modelo *Box of Words* garantiza que la salida de todos los diagramas pertenezca al tipo gramatical atómico `S`, por lo que también se puede omitir la transformación `UnifyCodomainRewriter`.

Sin embargo, dado que la única sustitución en el contenido de las cajas se produjo previamente aplicando lematización, aún es necesario aplicar las reglas `NumbersRewriteRule`, `TimeRewriteRule` y `UnknownWordsRewriteRule`. Como estas transformaciones no modifican la estructura general de los diagramas, el número de estructuras únicas del conjunto se mantiene, por lo que los valores mostrados en la tabla 3.7 no sufren cambios.

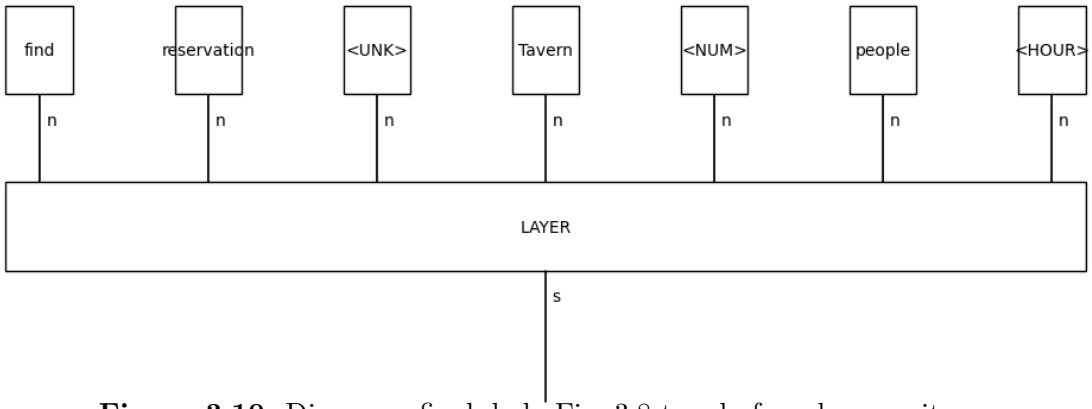



**Figura 3.10:** Diagrama final de la Fig. 3.8 tras la fase de reescritura.

### 3.4.3. Parametrización

La estructura propuesta por este modelo sigue las pautas de un circuito variacional básico (Fig. 2.10), y se puede describir de la siguiente manera:

- Un *ansatz* que forma el *feature map* representa las palabras en un espacio de Hilbert de dimensión $2^{\mathtt{N}}$, donde `N` es el número de qubits asignados a cada tipo gramatical. El hecho de tratar cada palabra de manera uniforme permite asociar a cada una un conjunto de rotaciones parametrizadas concreto. Este conjunto es análogo a la capa de entrada en una red tradicional, y se encarga de realizar el *embedding*.

- Un segundo *ansatz* agrupa el significado de las palabras a través de un circuito cuántico cuya arquitectura depende del *ansatz* elegido, siendo en nuestro caso `IQPAnsatz`. Este bloque actúa como una red de capas ocultas en una red tradicional. El parámetro de repetición del bloque ($n_layers$) controla la profundidad del circuito.

- Al igual que en el modelo DisCoCat, la salida del circuito depende del número de qubits asignados al tipo gramatical atómico `S`. La medida realizada sobre el estado cuántico final es análoga a la aplicación de una de una última capa *softmax* en una red neuronal, produciendo un vector de probabilidades de longitud $2^{\mathtt{S}}$.

En el circuito de la Fig. 3.11, obtenido del diagrama tras la reescritura (Fig. 3.10), se diferencian la fase de *embedding* y la fase de agrupación. Representar todas las palabras con un único qubit ($\mathtt{N} = 1$) permite que todas convivan en el mismo espacio vectorial, mientras que asignar tres rotaciones ($r = 3$) garantiza que todos los estados del espacio estén disponibles. El bloque del *ansatz* IQP se duplica al haber seleccionado $l = 2$,

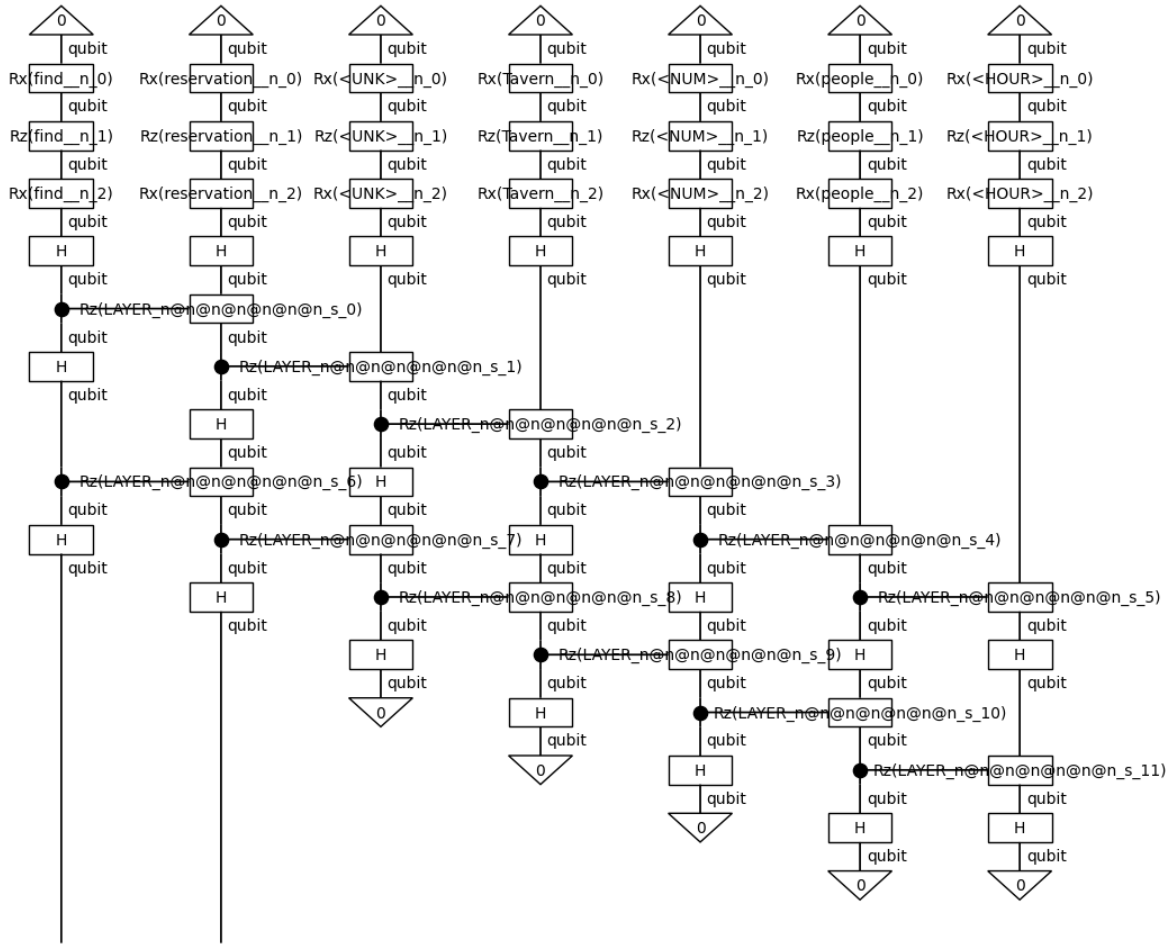



**Figura 3.11:** Circuito de la Fig. 3.10 con el *ansatz* `IQPCircuit`. Donde `N` y `S` están codificados en uno y dos qubits, respectivamente. Se emplean tres rotaciones en las palabras atómicas ($r = 3$) y el bloque IQP se aplica 2 veces ($l = 2$).

incrementando así la profundidad del circuito. En este caso, la elección del *ansatz* determina que la arquitectura de la composición tenga un orden secuencial, donde las puertas controladas establecen una relación entre cada palabra con la anterior.

### 3.4.4. Optimización

Con circuitos más simples y rápidos de ejecutar que los del modelo DisCoCat, esta fase de entrenamiento permite explorar una mayor diversidad de configuraciones. En la Fig. 3.12 se muestran las curvas de la fase de validación, donde se estudian los diferentes modelos $\text{Box}_{\texttt{N},r,l}$, cuyos *ansätze* varían en el número de qubits que codifican el sustantivo (`N`), la profundidad del circuito ($l$) y el número de rotaciones en el *embedding* de las palabras atómicas ($r$). En el Apéndice A se incluyen estas curvas, junto con la evolución del

conjunto de entrenamiento, para todos los modelos por separado.

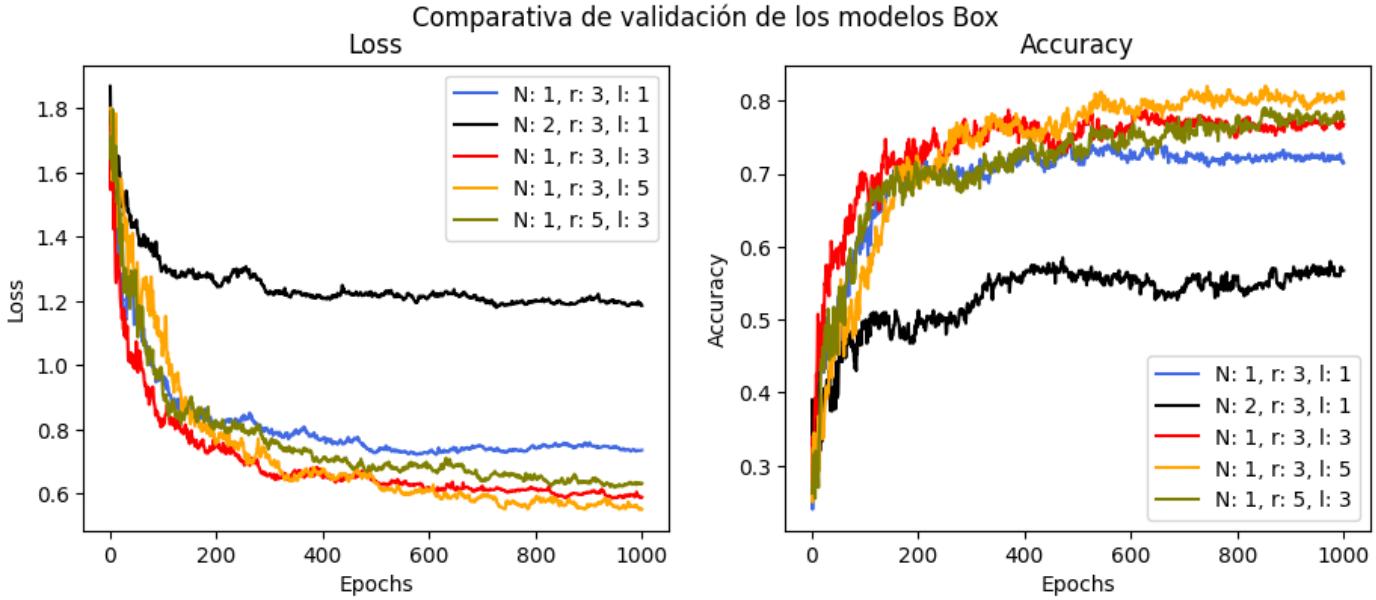



**Figura 3.12:** Comparación de la función de pérdida y el *accuracy* durante el entrenamiento y la validación a lo largo de 1,000 *epochs* para los modelos $\text{Box}_{\mathtt{N},r,l}$, con $\mathtt{N} \in \{1, 2\}$, $r \in \{3, 5\}$ y $l \in \{1, 3, 5\}$.

En las gráficas se observa que el rendimiento del modelo con $\mathtt{N} = 2$ es considerablemente inferior al resto. Esto se debe principalmente a que el *embedding* de una palabra representada por dos qubits se realiza mediante una única puerta controlada que las conecta. Aunque el espacio vectorial de representación es grande, sólo se explora una fracción. En cambio, en un sistema con un único qubit y tres rotaciones, se tiene acceso a todos los estados posibles, aunque el espacio vectorial sea menor.

En la tabla 3.8 se presentan los datos del entrenamiento y validación, los cuales confirman la de reducción del número de parámetros del modelo $\text{Box}_{2,3,1}$ frente a la versión con un único qubit $\text{Box}_{1,3,1}$. Además, el incremento de la anchura de los circuitos duplica el tiempo de entrenamiento. Por otro lado, se puede observar que las versiones multicapa ($l \in \{3, 5\}$) obtienen los mejores resultados, siendo el modelo $\text{Box}_{1,3,5}$ el que alcanza la menor pérdida y un mejor *accuracy* asociado (Acc. (Min Loss)) de 0.80.

| Modelo | Parámetros | Tiempo de Entrenam. | Max Acc. | Min Loss | Acc. (Min Loss) |
|---|---|---|---|---|---|
| $\text{Box}_{1,3,1}$ | 4223 | **1d 5h 29m 56s** | 0.74 | 0.7216 | 0.72 |
| $\text{Box}_{2,3,1}$ | **1578** | 2d 14h 34m 19s | 0.59 | 1.1793 | 0.55 |
| $\text{Box}_{1,3,3}$ | 4413 | 2d 3h 55m 53s | 0.79 | 0.5857 | 0.78 |
| $\text{Box}_{1,3,5}$ | 4603 | 3d 1h 26m 12s | **0.82** | **0.5510** | **0.80** |
| $\text{Box}_{1,5,3}$ | 7165 | 2d 7h 18m 13s | 0.79 | 0.6190 | 0.79 |

**Tabla 3.8:** Comparativa de los datos de entrenamiento de los modelos $\text{Box}_{\mathtt{N},r,l}$: Número de parámetros, tiempo de entrenamiento total, *accuracy* máximo alcanzado, valor de pérdida mínima y *accuracy* en mínima pérdida.

### 3.4.5. Resultados

Los resultados de la evaluación con el conjunto de prueba se muestran en la tabla 3.9, donde se observa que los modelos monocapa ($l = 1$) continúan obteniendo un rendimiento inferior frente a las versiones multicapa. Estas últimas concentran las mejores métricas, similares entre sí, siendo el modelo $\text{Box}_{1,3,3}$ el que alcanza la mejor puntuación, con un valor de 0.79 en todas la métricas evaluadas: *accuracy*, *precision*, *recall* y puntuación F1. Es importante señalar que dichos resultados se alcanzan con el menor número de qubits empleados hasta ahora, 17.

| Modelo | Qubits | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|
| $\text{Box}_{1,3,1}$ | **17** | 0.66 | 0.68 | 0.66 | 0.66 |
| $\text{Box}_{2,3,1}$ | 34 | 0.59 | 0.61 | 0.59 | 0.59 |
| $\text{Box}_{1,3,3}$ | **17** | **0.79** | **0.79** | **0.79** | **0.79** |
| $\text{Box}_{1,3,5}$ | **17** | 0.78 | 0.77 | 0.78 | 0.77 |
| $\text{Box}_{1,5,3}$ | **17** | 0.78 | 0.78 | 0.78 | 0.77 |

**Tabla 3.9:** Resultados de los modelos $\text{Box}_{\mathtt{N},r,l}$ aplicados al conjunto de prueba, junto al número de qubits necesarios para la ejecución completa del modelo, desde el entrenamiento hasta la prueba.

## 3.5. Comparativa

Tras haber analizado en detalle cada uno de los modelos, se procede a comparar entre los modelos DisCoCat y *Box of Words* considerando tanto su rendimiento como su eficiencia computacional.

En la tabla 3.10 se muestran los datos de entrenamiento de las tablas 3.5 y 3.8 unificados, donde se comparan el número de parámetros, tiempo de entrenamiento, máxima exactitud, menor pérdida y la exactitud asociada a dicha pértida (Acc. (Min Loss)). En

ella se observa que el modelo $\text{Box}_{2,3,1}$ presenta el menor número de parámetros, a costa de sacrificar su capacidad predictiva. Exceptuando este caso, los modelos *Box of Words* obtienen mejores resultados de rendimiento sobre el conjunto de validación, así como en el tiempo de entrenamiento.

| Modelo | Parámetros | Tiempo de Entrenam. | Max Acc. | Min Loss | Acc. (Min Loss) |
|---|---|---|---|---|---|
| $\text{DisCoCat}_1$ | 4716 | 3d 10h 51m 41s | 0.70 | 0.7158 | 0.69 |
| $\text{DisCoCat}_2$ | 7461 | 5d 5h 35m 27s | 0.72 | 0.6884 | 0.71 |
| $\text{Box}_{1,3,1}$ | 4223 | **1d 5h 29m 56s** | 0.74 | 0.7216 | 0.72 |
| $\text{Box}_{2,3,1}$ | **1578** | 2d 14h 34m 19s | 0.59 | 1.1793 | 0.55 |
| $\text{Box}_{1,3,3}$ | 4413 | 2d 3h 55m 53s | 0.79 | 0.5857 | 0.78 |
| $\text{Box}_{1,3,5}$ | 4603 | 3d 1h 26m 12s | **0.82** | **0.5510** | **0.80** |
| $\text{Box}_{1,5,3}$ | 7165 | 2d 7h 18m 13s | 0.79 | 0.6190 | 0.79 |

**Tabla 3.10:** Comparativa de los datos de entrenamiento de los modelos $\text{Box}_{\mathtt{N},r,l}$: Número de parámetros, tiempo de entrenamiento total, exactitud máxima alcanzada, valor de pérdida mínima y exactitud en mínima pérdida.

Si bien el tiempo de entrenamiento está relacionado directamente con la anchura y profundidad de los circuitos, no es la única variable temporal relevante al valorar la eficiencia de los modelos. En la Fig. 3.13 se presentan los tiempos de preprocesamiento de las oraciones, desglosados en tres fases: la creación de los diagramas, su reescritura y la conversión final en circuitos cuánticos. El análisis CCG que realiza el modelo DisCoCat implica un coste computacional considerable, tanto en la fase de creación como las subsiguientes tranformaciones, extendiendo todo el proceso a unos 30 minutos. En contraste, el enfoque simplificado de los modelos *Box of Words* permite reducir este tiempo a un máximo de 1.4 minutos para el modelo mas complejo $\text{Box}_{2,3,1}$, lo que supone una mejora en la velocidad de ejecución en aproximadamente un factor 20.

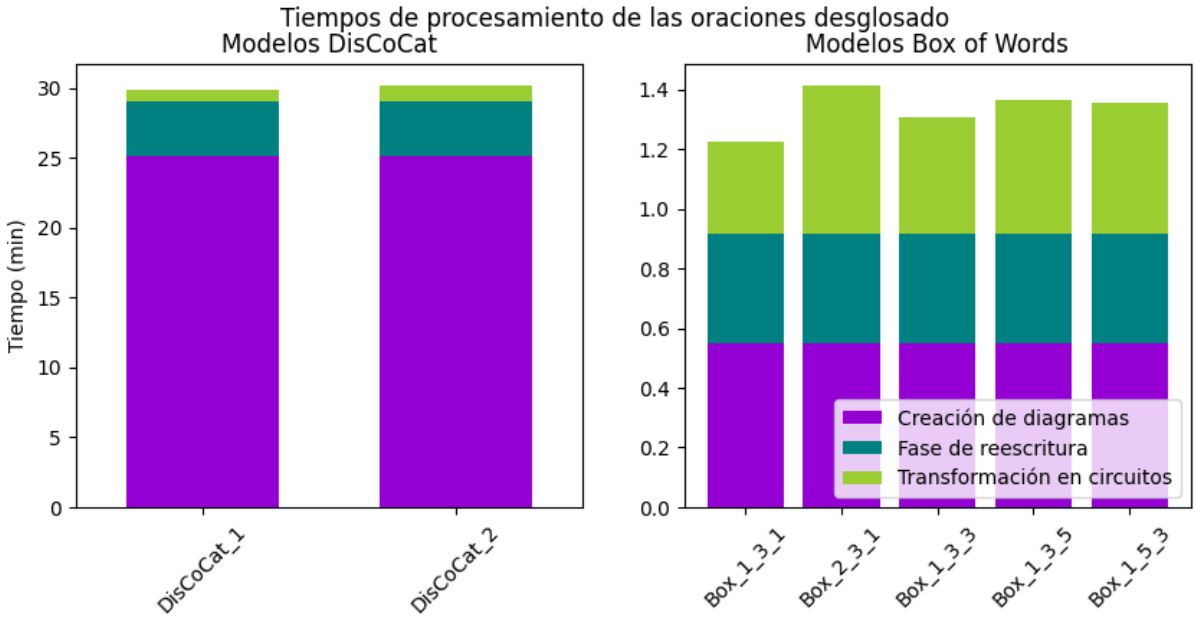



**Figura 3.13:** Desglose de tiempo de creación de los diagramas, su reescritura y transformación en circuitos para los diferentes modelos.

Los datos de la tabla de valores de rendimiento del conjunto de prueba unificada (Fig. 3.11) constituyen la evidencia final de la superioridad de los modelos *Box of Words*:

| Modelo | Qubits | Exactitud | Precisión | Exhaustividad | F1 |
|---|---|---|---|---|---|
| $\text{DisCoCat}_1$ | 108 | 0.69 | 0.69 | 0.69 | 0.68 |
| $\text{DisCoCat}_2$ | 108 | 0.67 | 0.65 | 0.67 | 0.66 |
| $\text{Box}_{1,3,1}$ | **17** | 0.66 | 0.68 | 0.66 | 0.66 |
| $\text{Box}_{2,3,1}$ | 34 | 0.59 | 0.61 | 0.59 | 0.59 |
| $\text{Box}_{1,3,3}$ | **17** | **0.79** | **0.79** | **0.79** | **0.79** |
| $\text{Box}_{1,3,5}$ | **17** | 0.78 | 0.77 | 0.78 | 0.77 |
| $\text{Box}_{1,5,3}$ | **17** | 0.78 | 0.78 | 0.78 | 0.77 |

**Tabla 3.11:** Resultados de los modelos $\text{Box}_{\mathtt{N},r,l}$ aplicados al conjunto de prueba, junto al número de qubits necesarios para la ejecución completa del modelo, desde el entrenamiento hasta la prueba.

Estos modelos presentan los mejores valores de exactitud, precisión, exhaustividad y F1, destacando el modelo $\text{Box}_{1,3,3}$ como el de mejor rendimiento. El reducido número de qubits requeridos para su implementación, junto a sus bajos valores de pérdida, posicionan a los modelos *Box of Words* como candidatos para simulaciones cuánticas en presencia de ruido. Además, su arquitectura permite la representación visual interpretable de las palabras. Todas estas características serán exploradas con mayor detalle en el capítulo siguiente.

# Capítulo 4

# Consideraciones en la implementación de los modelos

Tras un estudio comparativo de los modelos DisCoCat y *Box of Words*, nos enfocamos ahora en explorar una serie de características inherentes a estos enfoques, que pueden resultar especialmente relevantes para obtener una visión más amplia de su implementación práctica en el contexto del aprendizaje automático cuántico. Mas allá del rendimiento observado en los entrenamientos del capítulo 3, es posible profundizar en algunas de las ventajas, limitaciones y particularidades.

En este capítulo se abordan tres líneas de análisis que extienden los estudios anteriores. Primero se estudia el *embedding* de las palabras en el modelo *Box of Words*, observando su distribución en la esfera de Bloch, con el objetivo de evaluar el impacto de un *embedding* previo y condicionado en el rendimiento del modelo. A continuación, se analiza la resistencia del modelo con mejor pérdida a la presencia de ruido cuántico, mediante una simulación más cercana a un escenario real. Para finalizar, se examina la técnica *one-versus-rest*, con el fin de comparar su rendimiento frente a los modelos estudiados en el capítulo anterior.

## 4.1. Efecto del embedding en el modelo Box of Words

Como se pudo observar en la Fig. 3.11, los modelos en los que las palabras se codifican con un único qubit ($\mathtt{N} = 1$) emplean un conjunto de puertas cuánticas dispuestas en serie

que rotan el estado que las representan alrededor de la superficie de la esfera de Bloch. El conjunto de trasformaciones aplicadas antes de alcanzar el *ansatz* que agrupa los significados de las palabras constituye la fase de *embedding*.

Durante el entrenamiento, los parámetros optimizados corresponden a la magnitud de dichas rotaciones, lo que permite obtener una representación visual de la distribución de las palabras en la esfera. La Fig. 4.1 muestra los estados de las palabras mas representativas de cada categoría —aquellas exclusivas y repetidas, según la tabla 3.2— utilizando los parámetros entrenados en el modelo $\text{Box}_{1,3,5}$, el cual mostró el mejor rendimiento en la fase de validación.

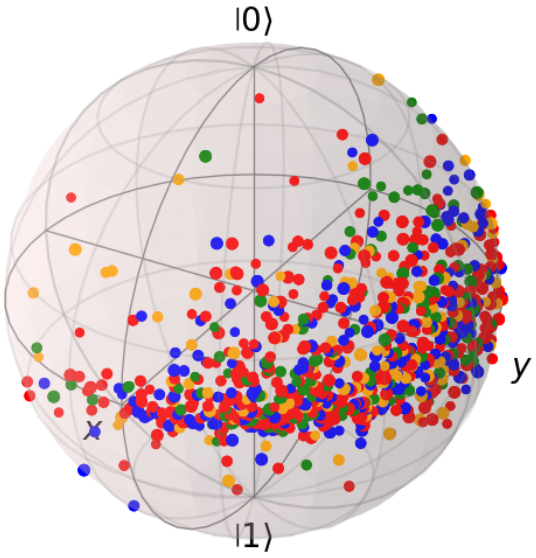



**Figura 4.1:** Representación de las palabras exclusivas y repetidas en la esfera de Bloch, diferenciadas por categoría.

En ella, se observa cómo los estados aparecen concentrados en un sector, pero no forman agrupaciones según su pertenencia a una categoría. Esto indica que el modelo no capta las diferencias entre clases a nivel del *embedding* cuántico. Este resultado es consecuencia del flujo de trabajo de Lambeq, en el cual las representaciones de las palabras son aprendidas en función del objetivo de la clasificación, sin incorporar ninguna información semántica explícita.

Para estudiar el impacto de incorporar la información semántica en el *embedding*, se entrena un modelo $\text{Box}_{1,3,5}$ en las mismas condiciones que su análogo del capítulo 3, pero

con una inicialización diferente: los parámetros iniciales distribuyen dichas palabras de forma aleatoria, agrupándolas en regiones diferenciadas de la esfera de Bloch. En la Fig. 4.2 se comparan las curvas de entrenamiento de este nuevo modelo con las de su análogo. En ella, se puede observar cómo la evolución de los dos modelos es similar, con el modelo que utiliza el nuevo *embedding* mostrando un rendimiento ligeramente inferior.

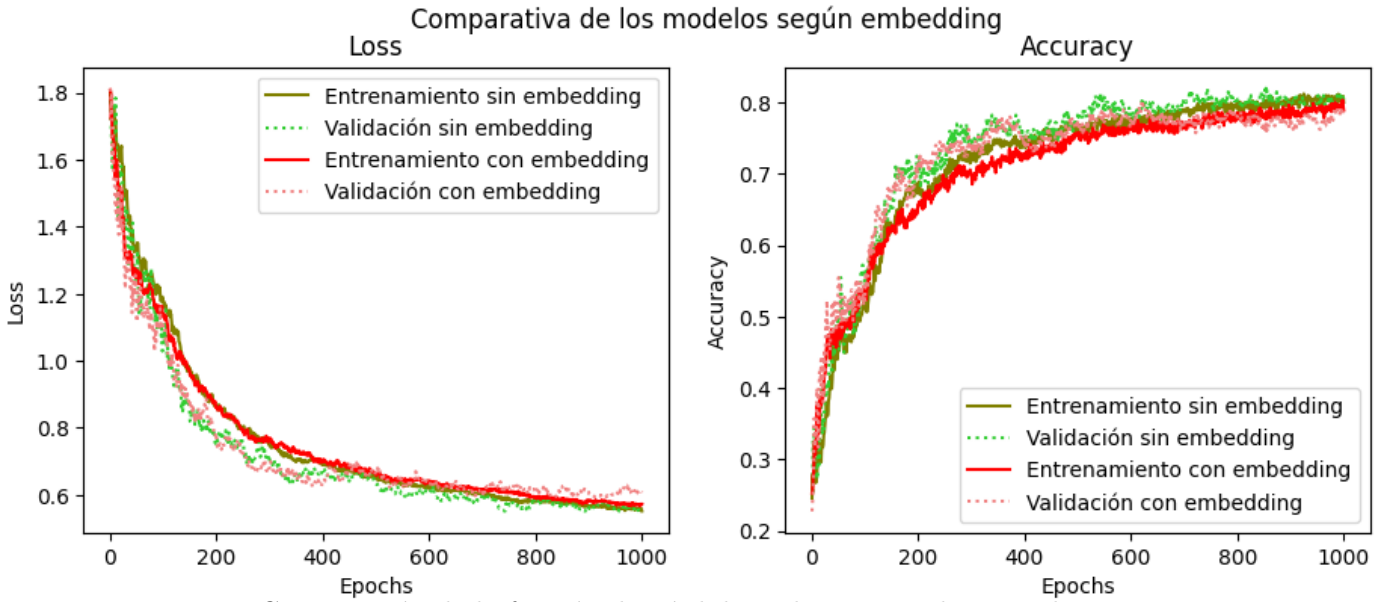



**Figura 4.2:** Comparación de la función de pérdida y el *accuracy* durante el entrenamiento y la validación a lo largo de 1,000 *epochs* para los modelos entrenados sin *embedding* condicionado (verde) y con él (rojo).

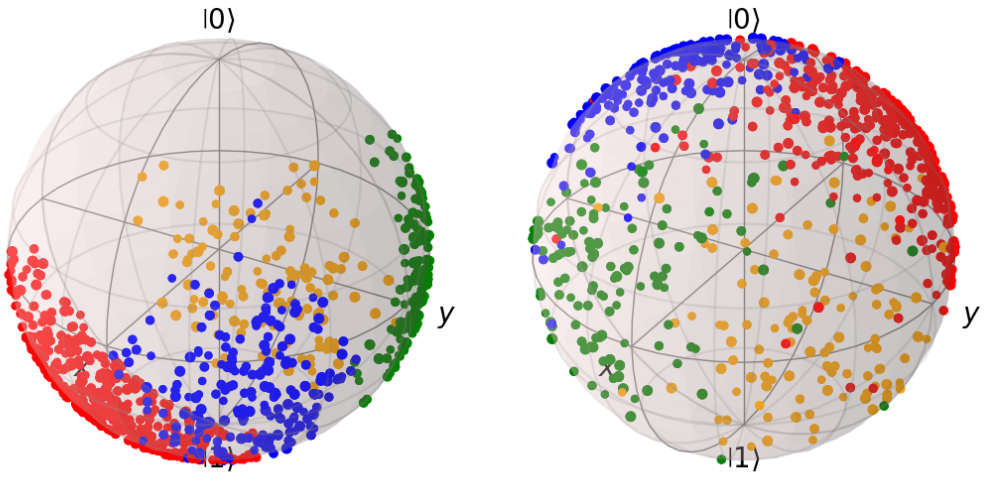
Los datos finales de rendimiento, tanto de evaluación como de prueba, se muestran en la Tabla 4.1, donde se comprueba numéricamente el empeoramiento de los resultados en comparación con el modelo original:

| Modelo | Evaluación | | Prueba | | | |
|---|---|---|---|---|---|---|
| | Min Loss | Acc. (Min Loss) | Accuracy | Precision | Recall | F1 |
| Sin *embedding* | **0.5510** | **0.80** | **0.78** | **0.77** | **0.78** | **0.77** |
| Con *embedding* | 0.5915 | 0.78 | 0.73 | 0.73 | 0.73 | 0.72 |

**Tabla 4.1:** Datos de validación y prueba de los modelos con y sin *embedding* condicionado.

Este empeoramiento puede deberse a que el *embedding* utilizado no tiene en cuenta la composición secuencial del modelo *Box of Words*, lo que afecta su capacidad en la tarea de establecer relaciones entre las palabras. Sin embargo, el modelo consigue preservar el significado semántico de las palabras. En la Fig. 4.3 se puede visualizar su representación en la esfera de Bloch, tanto antes como después del entrenamiento, donde los estados han

evolucionado, expandiéndose por toda la esfera en lugar de centrarse en un sector determinado, y manteniendo la coherencia entre las palabras de una misma categoría categoría.




Situación inicial Situación final

**Figura 4.3:** Representación de las palabras exclusivas y repetidas en la esfera de Bloch, diferenciadas por categorías antes y después del entrenamiento con *embedding* condicionado.

## 4.2. Resistencia al ruido del modelo Box of Words

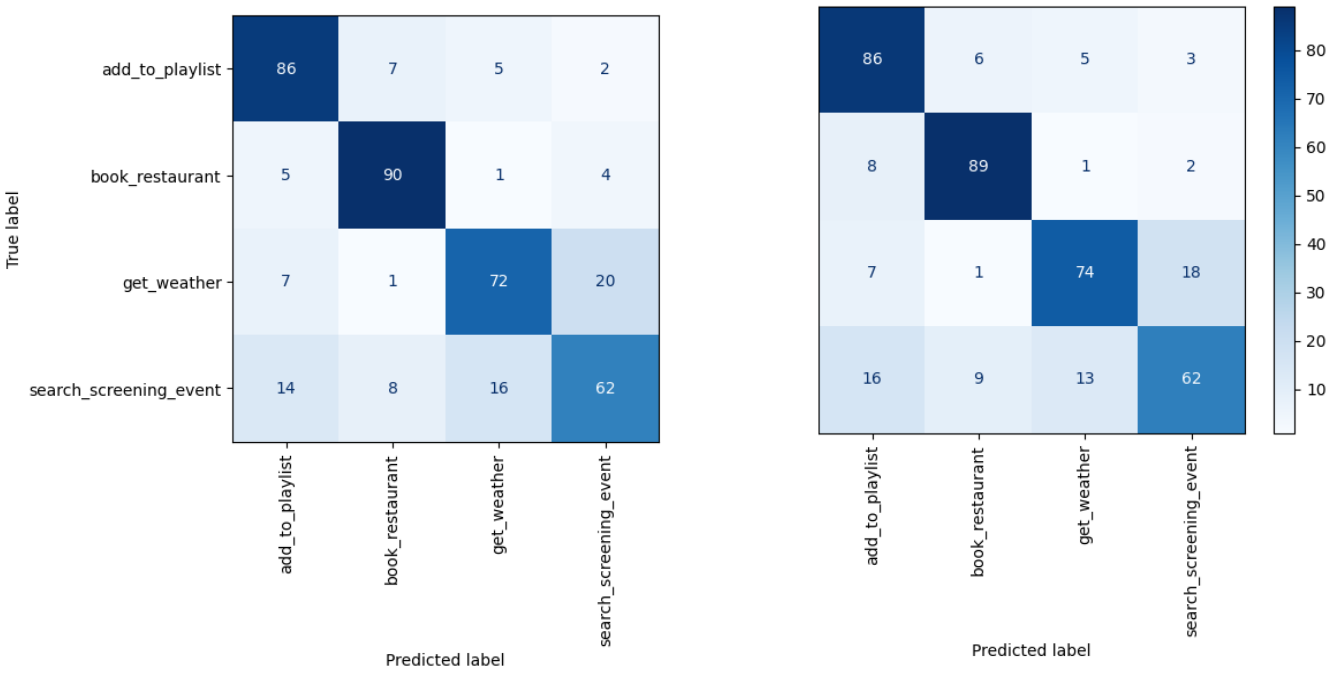
La elección del modelo con menor valor de pérdida durante el entrenamiento se fundamenta en el objetivo de maximizar la probabilidad de obtener el estado adecuado en el vector de probabilidades, con el fin de minimizar la influencia del ruido cuántico. En un escenario real, el ruido cuántico se manifiesta como pequeños errores en las operaciones, la decoherencia cuántica y la interacción indeseada entre los qubits, lo que afecta a la precisión de las computadoras cuánticas. Estos errores pueden alterar las probabilidades de predicción, sobretodo si la diferencia entre dos clases es pequeña. Al maximizar la probabilidad de la clase predicha, se reduce el riesgo de interferencia en la clasificación final.

En la tabla 4.2 se comparan los valores del rendimiento del modelo $\text{Box}_{1,3,5}$ en su ejecución clásica sin ruido y su ejecución cuántica en presencia de ruido, mostrando resultados equivalentes. La simulación se ha realizado con *AerBackend* de *Qiskit*, utilizando un modelo de ruido genérico de la misma librería, que simula un sistema de 17 qubits. La clase predicha se obtiene mediante el estado más frecuente en 8,192 ejecuciones (*shots)*:

| Modelo | Ruido cuántico | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|
| $\text{Box}_{1,3,5}$ | No | 0.78 | 0.77 | 0.78 | 0.77 |
| $\text{Box}_{1,3,5}$ | Sí | 0.78 | 0.77 | 0.78 | 0.77 |

**Tabla 4.2:** Comparativa del resultado del modelo $\text{Box}_{1,3,5}$ simulado clásicamente y ejecutado en una simulación cuántica en presencia de ruido.

Sin embargo, este resultado puede ser engañoso. Al analizar las matrices de confusión (Fig. 4.4), se observan pequeñas diferencias, lo que indica la interferencia del ruido en el proceso de clasificación:




Ejecución clásica sin ruido

Ejecución cuántica con ruido

**Figura 4.4:** Matrices de confusión del modelo $\text{Box}_{1,3,5}$ simulado clásicamente y ejecutado en una simulación cuántica en presencia de ruido.

Estas variaciones ponen de manifiesto la importancia de realizar análisis más detallados al evaluar modelos en sistemas cuánticos reales.

## 4.3. One-versus-rest en clasificación cuántica

Una de las técnicas habituales en problemas de clasificación multiclase es la denominada One-versus-rest (OvR), que descompone la tarea en múltiples clasificadores binarios independientes, donde cada uno aprende a distinguir una clase en particular frente al resto. En nuestro caso, dado que los modelos proporcionan directamente un vector de

probabilidades de pertenencia, esta estrategia puede integrarse de forma natural.

Para aplicar esta técnica, es necesario que la salida del circuito sea binaria, lo que se consigue asignando un único qubit a la oración ($\mathtt{S} = 1$) cuando se construye. Además, se necesita codificar las etiquetas de modo que sean $[1, 0]$ si la oración pertenece a la categoría del modelo que se entrena, y $[0, 1]$ en caso contrario. De esta forma, si la predicción del modelo binario independiente $i$ tiene la forma $[p_{0_i}, p_{1_i}]$, entonces $p_{0_i}$ es la probabilidad de pertenencia a su clase y $p_{1_i}$, la de no pertenencia.

Para generar una predicción multiclase, se construye el vector de probabilidades combinando las probabilidades de pertenencia estimadas por cada modelo independiente:

$$[p_{0_1}, p_{0_2}, p_{0_3}, p_{0_4}].$$

Y la clase predicha se asigna escogiendo la componente con mayor probabilidad.

En la tabla 4.3 se comparan los resultados de la evaluación con el conjunto de prueba de los modelos $\text{DisCoCat}_1$ y $\text{Box}_{1,3,3}$ —los mejores en sus respectivas modalidades— con sus análogos entrenados con la técnica One-versus-rest. Cada clasificador binario ha sido entrenado durante 250 *epochs*, manteniendo el total de 1000 *epochs* de entrenamiento total. Los resultados muestran que la versión OvR de DisCoCat no logra superar los resultados de su versión con clasificación directa; sin embargo, la variante OvR de $\text{Box}_{1,3,3}$ experimenta una mejora significativa, posicionándose como el mejor modelo de este trabajo, con una puntuación de 0.87 en *accuracy*, *precision*, *recall* y F1.

| Modelo | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| $\text{DisCoCat}_1$ | 0.69 | 0.69 | 0.69 | 0.68 |
| $\text{DisCoCat}_1$ (OvR) | 0.69 | 0.69 | 0.69 | 0.68 |
| $\text{Box}_{1,3,3}$ | 0.79 | 0.79 | 0.79 | 0.79 |
| $\text{Box}_{1,3,3}$ (OvR) | **0.87** | **0.87** | **0.87** | **0.87** |

**Tabla 4.3:** Comparativa de los mejores modelos DisCoCat y *Box of Words* respecto a sus análogos entrenados con One-versus-rest.

Para completar el análisis, en la Fig. 4.5 se muestra la matriz de confusión correspondiente a la versión OvR. Al compararla con la matriz de su equivalente (Fig. 4.4) se observa una mejora en las dos clases más problemáticas, *get_weather* y *search_screen_event*. En

la primera, esa mejora es significativa, situándose en valores similares a las mejores puntuadas; mientras que la otra, si bien sigue siendo la clase con peor rendimiento, siendo la clase con peor tasa de acierto, un 74 %.

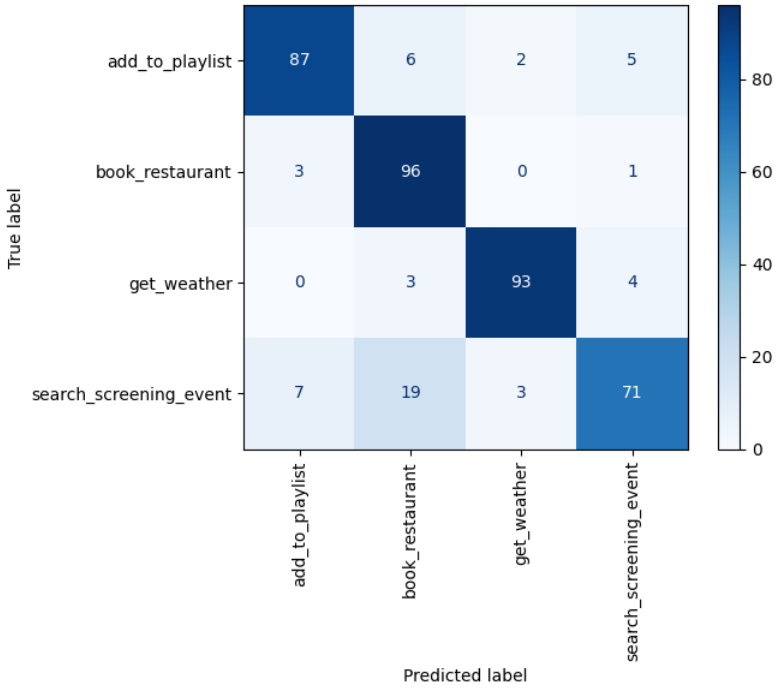



**Figura 4.5:** Matriz de confusión del modelo $\text{Box}_{1,3,3}$ entrenado con One-versus-rest

# Capítulo 5

# Conclusiones

A lo largo de este trabajo se han desarrollado los conocimientos teóricos necesarios para comprender uno de los enfoques más destacados en la aplicación de la computación cuántica al procesamiento del lenguaje natural: el uso de estructuras categóricas para modelar la composición del significado según el marco teórico de DisCoCat. Para ello, se han introducido los fundamentos de la computación cuántica y el aprendizaje automático cuántico, así como los conceptos básicos de la teoría de categorías, que permiten establecer una conexión entre estas disciplinas y comprender el funcionamiento del modelo formal.

Estos conocimientos se han aplicado en un caso práctico, abordando una de las tareas actuales en el ámbito del procesamiento de lenguaje natural: la clasificación de intenciones. Para ello, se ha utilizado Lambeq, una librería desarrollada específicamente para implementar modelos dentro de este marco teórico, detallando los procesos realizados en cada fase de su flujo de trabajo. El estudio se ha centrado en dos enfoques: por un lado, un modelo gramático-composicional, que realiza un análisis CCG para construir los correspondientes diagramas categóricos; y por otro, un modelo simplificado, que omite el análisis sintáctico y da lugar a estructuras más básicas. La comparación entre ambos enfoques, evaluando distintas configuraciones de cada modelo, ha mostrado que el modelo simplificado obtiene mejores resultados tanto en rendimiento como eficiencia computacional.

Estos resultados han servido como base para realizar una serie de estudios complementarios orientados a analizar otros aspectos del modelo y su comportamiento en distintos

contextos:

- Se ha mostrado la interpretabilidad del *embedding* cuántico, representándolo en la esfera de Bloch. Además, se ha comprobado que realizar un *embedding* previo no mejora necesariamente el rendimiento final del modelo.
- El modelo más prometedor ha sido evaluado en una simulación cuántica con presencia de ruido, lo que ha permitido comprobar su resistencia ante condiciones no ideales.
- Se han comparado dos estrategias de clasificación: la clasificación multiclase directa frente a otra basada en One-versus-rest. Esta última ha demostrado rendimiento significativamente superior, situándose como la alternativa con mejores resultados en el estudio.

Este enfoque presenta la ventaja de permitir su implementación tanto en hardware clásico como cuántico, lo que facilita la experimentación frente a una de las grandes limitaciones del estudio práctico en la computación cuántica: la escasa disponibilidad de ordenadores cuánticos, que además suelen contar con un número limitado de qubits a menudo insuficientes para abarcar la dimensión del problema. Aun así, el alto coste computacional de las simulaciones cuánticas puede obligar a restringir el tamaño de los conjuntos de datos, lo que limita la capacidad de generalización de los modelos y dificulta su competencia con modelos de mayor escala y eficiencia, como los *large language models*.

## 5.1. Futuras líneas de trabajo

El campo del *Quantum Natural Language Processing* es relativamente joven, y las innovaciones constantes en computación cuántica abren una amplia variedad de posibles lineas de investigación. Partiendo de este trabajo como punto de partida, orientado a la clasificación cuántica multiclase, se pueden plantear diferentes caminos para su ampliación según distintos enfoques:

Analizar el comportamiento de los modelos frente a las diferentes formas de construir el *embedding* de las palabras en función de los diferentes *ansätze*, y estudiar cómo la elección de este influye en la capacidad del modelo. En el proceso de construcción del *embedding*,

también se pueden explorar distintas estrategias de inicialización de parámetros, así como la posibilidad de utilizar representaciones clásicas preentrenadas que posteriormente sean codificadas en estados cuánticos.

Asímismo, sería interesante comprobar los resultados utilizando otras técnicas de optimización o, al igual que se ha probado con one-versus-rest, explorar otras estrategias de entrenamiento para clasificación multiclase.

Por otro lado, sería de especial interés estudiar la implementación completa en hardware cuántico real, evaluando cómo sus capacidades de paralelización y manipulación de espacios de alta dimensionalidad pueden aumentan la escalabilidad del modelo. Además, se puede analizar la integración del modelo como capa intermedia en arquitecturas híbridas clásico-cuánticas, donde las salidas de los circuitos cuánticos sean interceptadas y procesadas por clasificadores clásicos.

Para finalizar, sería interesante comparar los resultados con otras técnicas de *Quantum Machine learning*, como las *Quantum Recurrent Neural Networks* o las *Quantum Self-Atention Neural Networks*, las cuales no siguen el enfoque categórico.

# Apéndice A

# Gráficas de entrenamiento y evaluación de los modelos

## Modelos DisCoCat multiclase

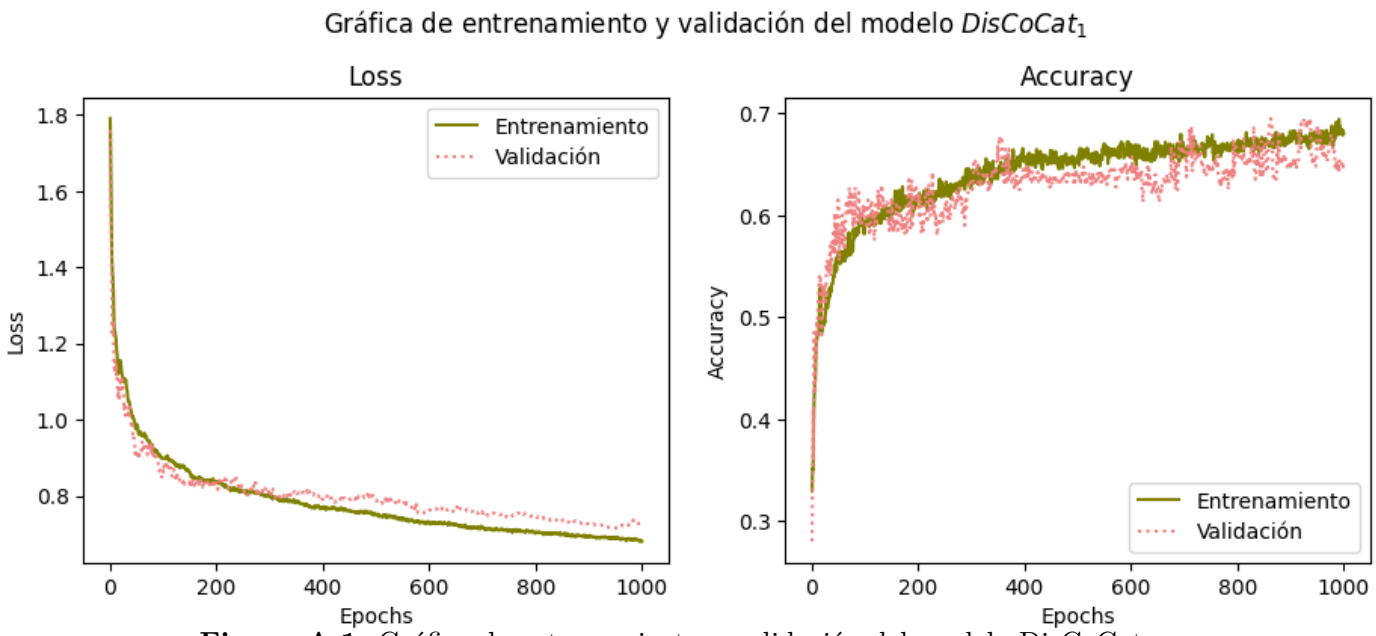



**Figura A.1:** Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$.

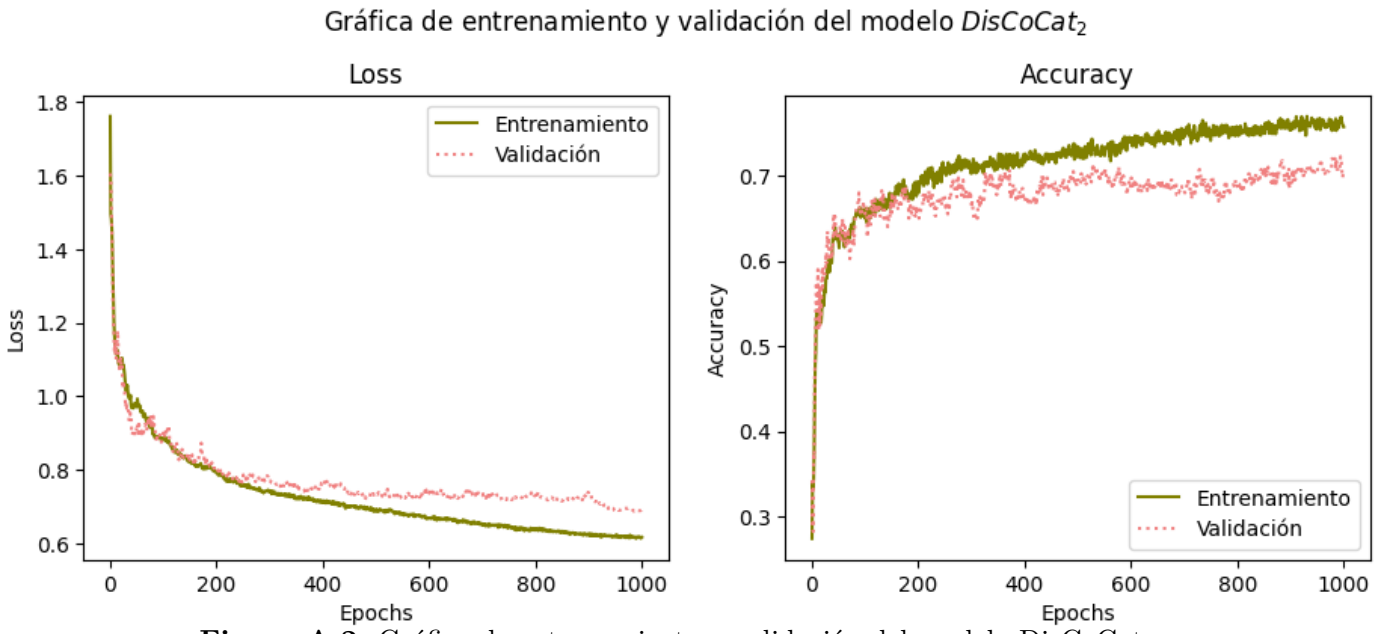



**Figura A.2:** Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_2$.

# Modelos Box of words multiclase

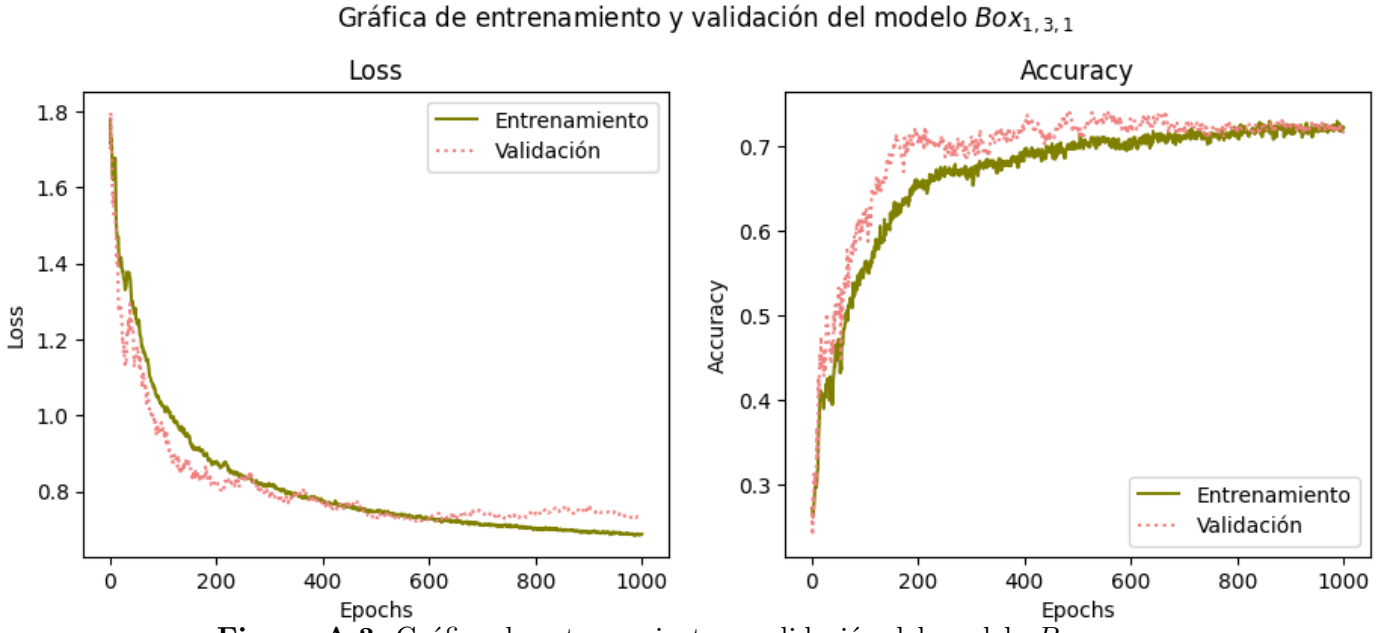



**Figura A.3:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,1}$.

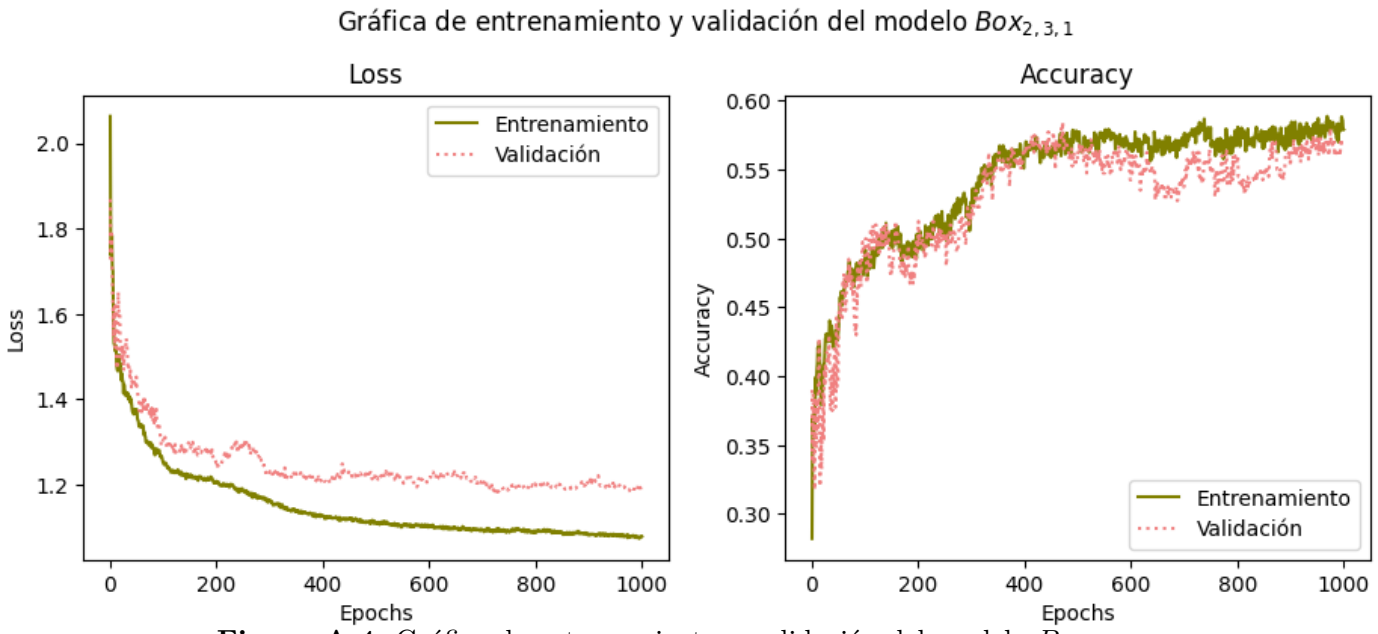



**Figura A.4:** Gráfica de entrenamiento y validación del modelo $Box_{2,3,1}$.

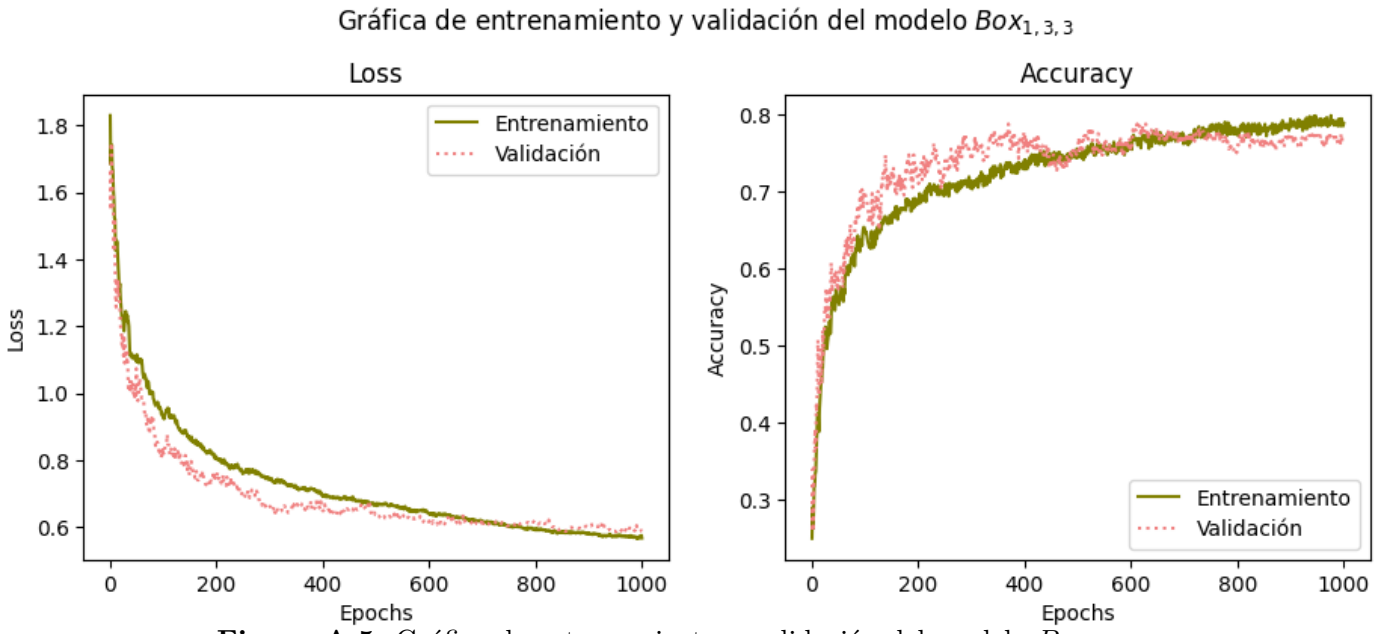



**Figura A.5:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$.

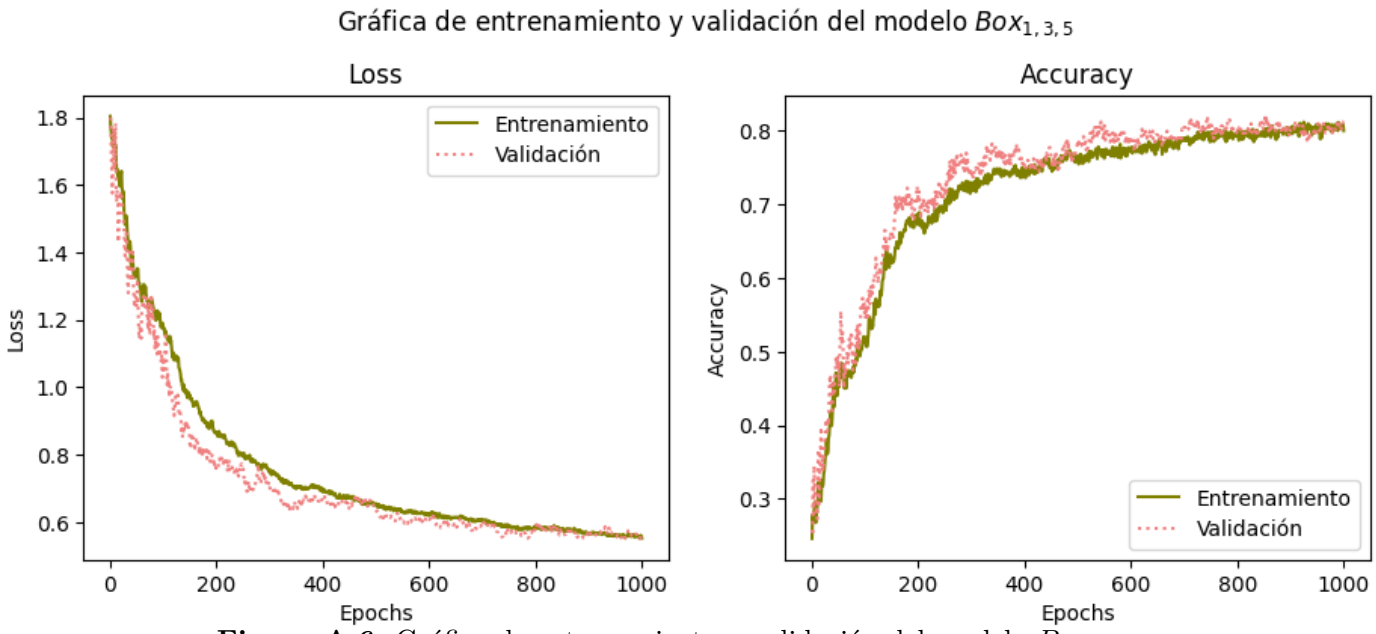



**Figura A.6:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,5}$.

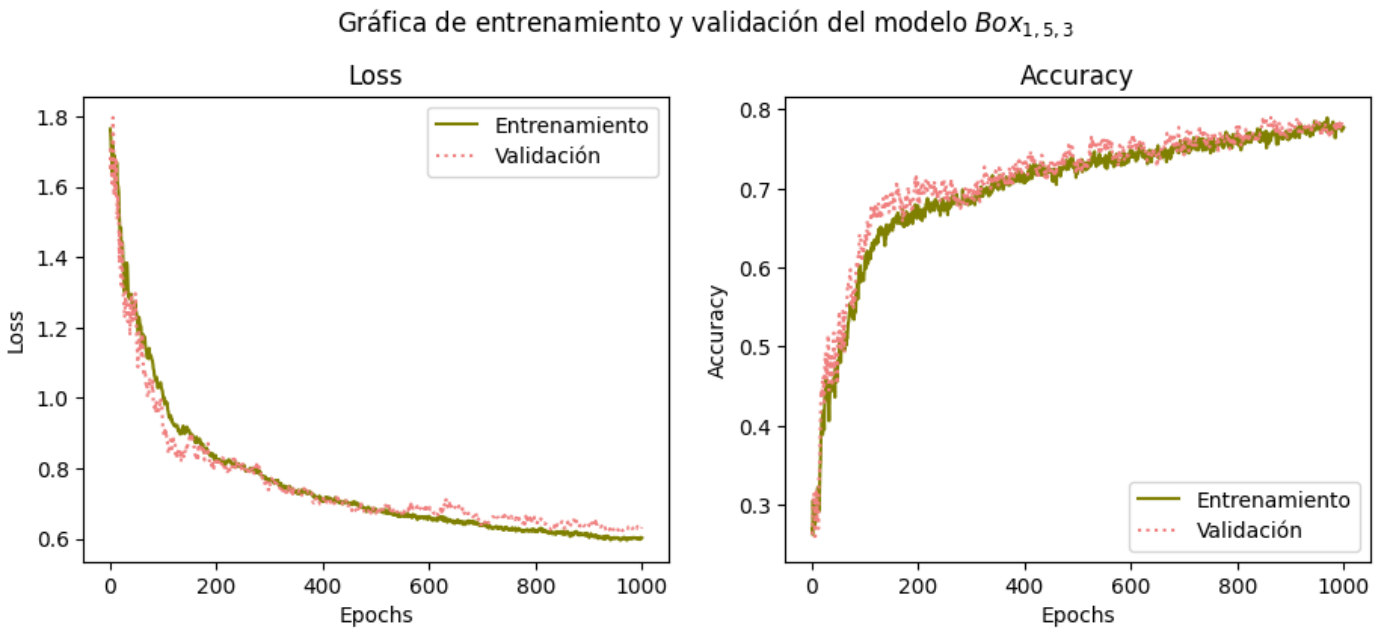



**Figura A.7:** Gráfica de entrenamiento y validación del modelo $Box_{1,5,3}$.

# Modelos Box of words multiclase con embedding condicional

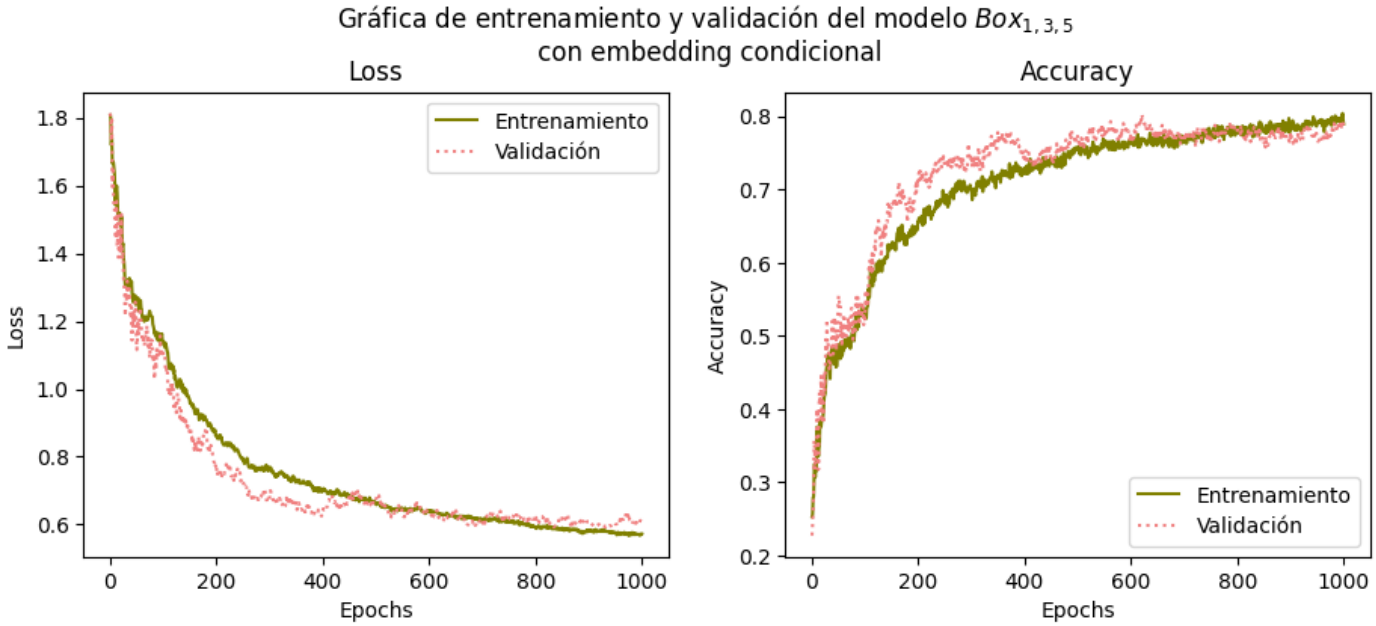



**Figura A.8:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,5}$ con *embedding* condicional.

# Modelos DisCoCat en One versus rest por clases

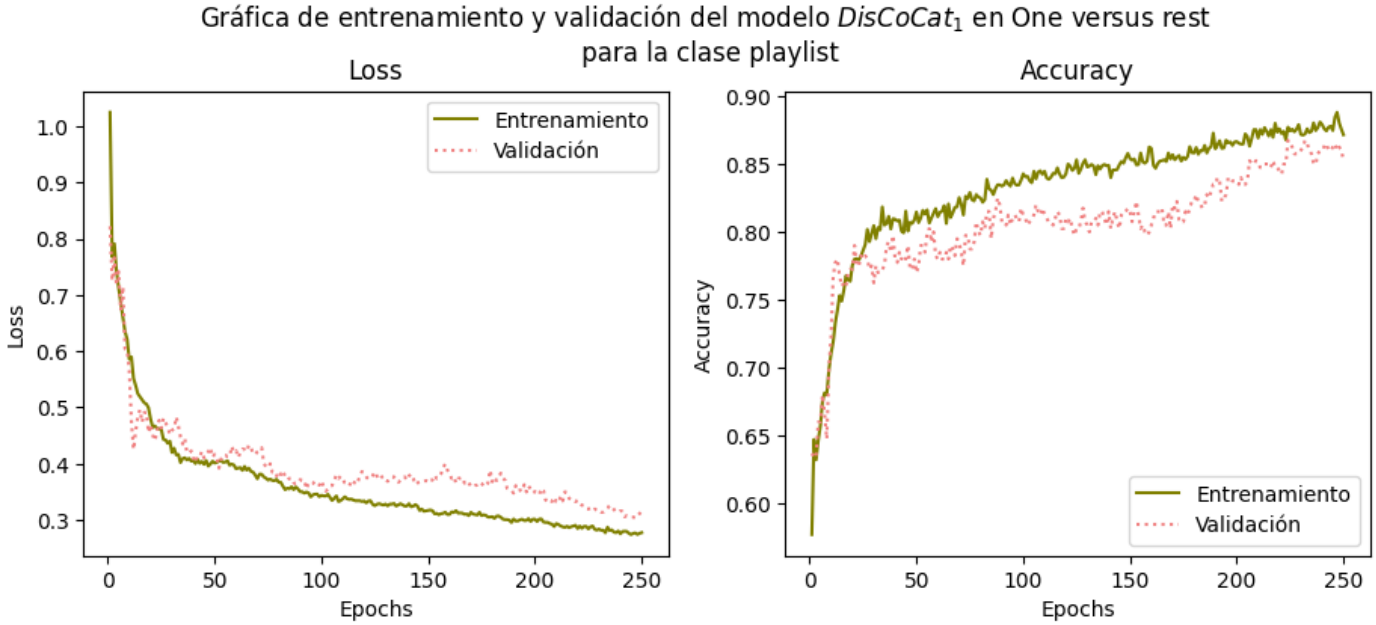



**Figura A.9:** Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *add_to_playlist*.

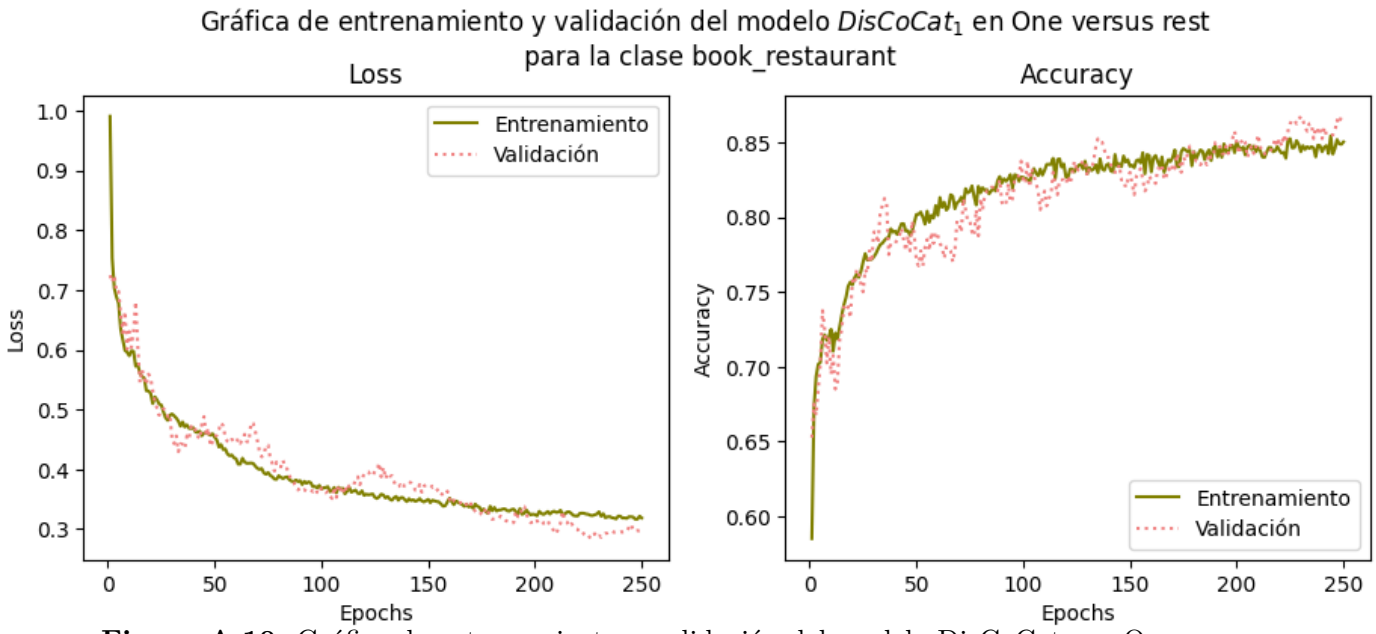



**Figura A.10:** Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *book_restaurant*.

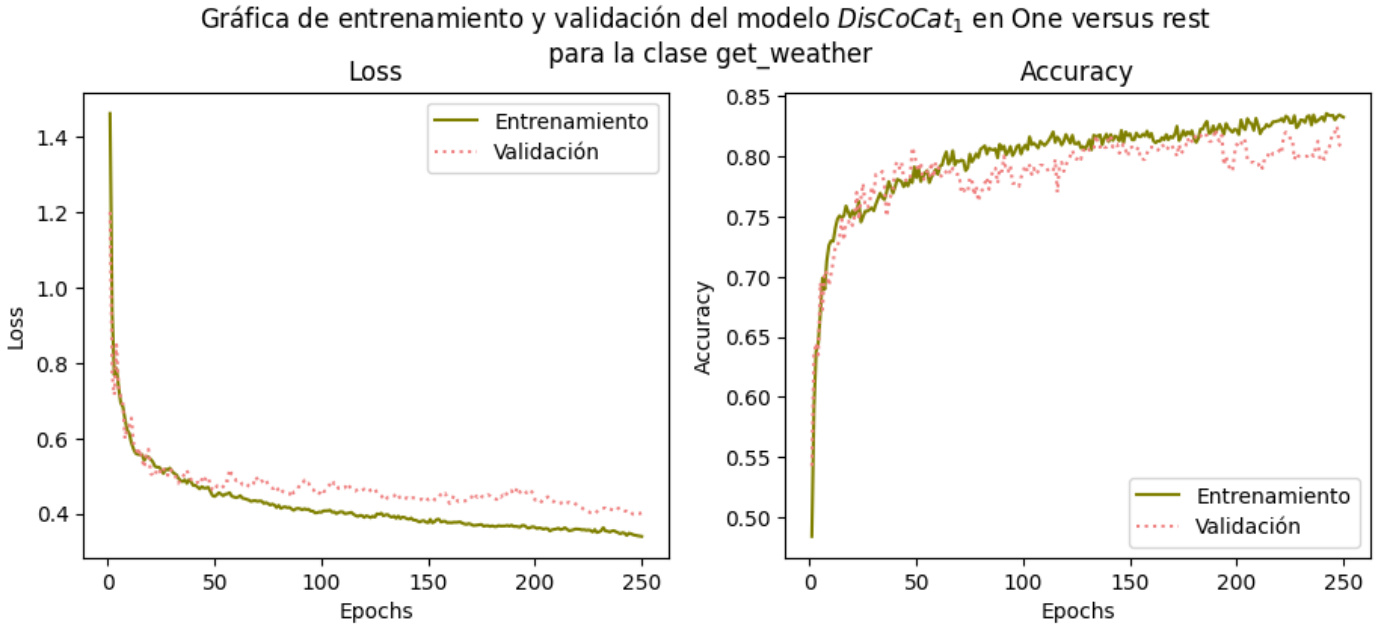



**Figura A.11:** Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *get_weather*.

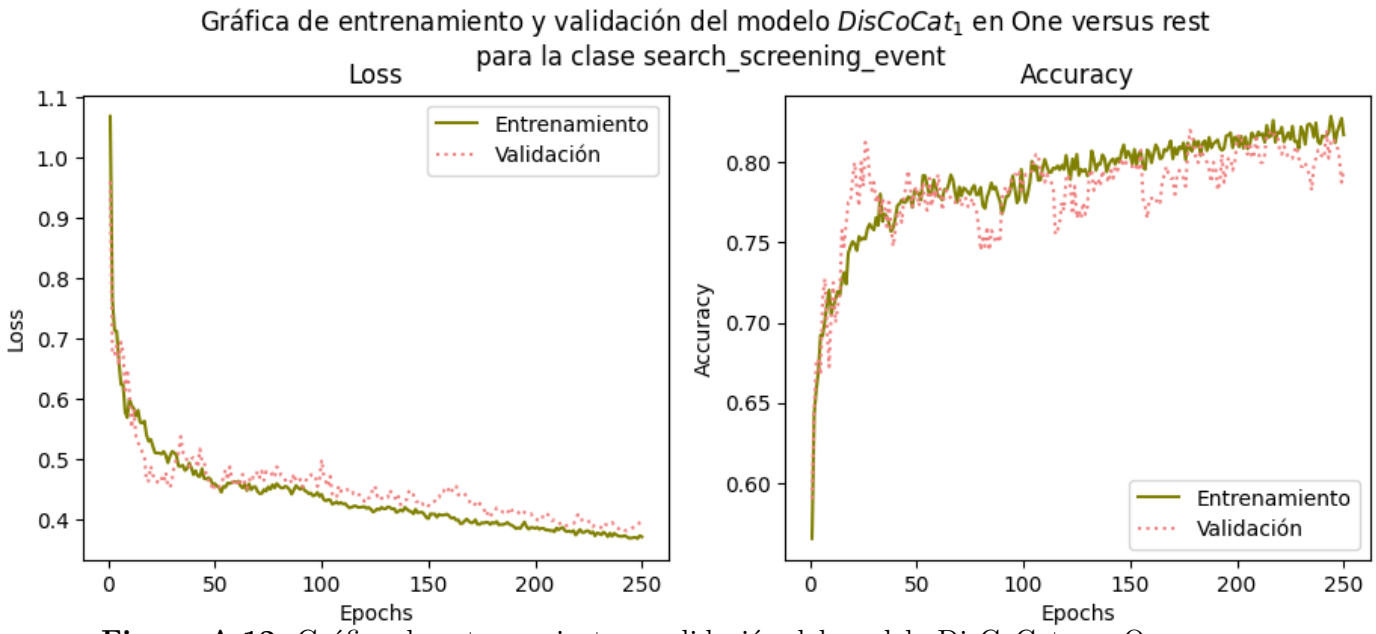



**Figura A.12:** Gráfica de entrenamiento y validación del modelo $\text{DisCoCat}_1$ en One versus rest para la clase *search_screening_event*.

# Modelos Box of words en One versus rest por clases

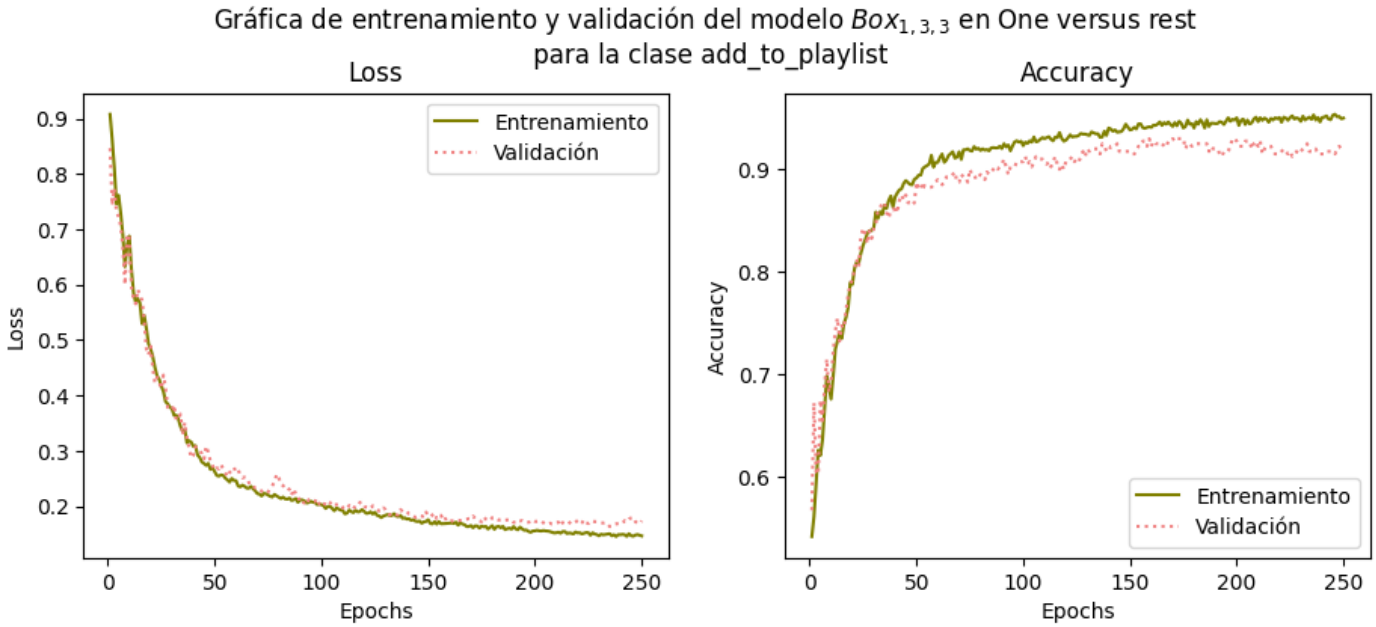



**Figura A.13:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *add_to_playlist*.

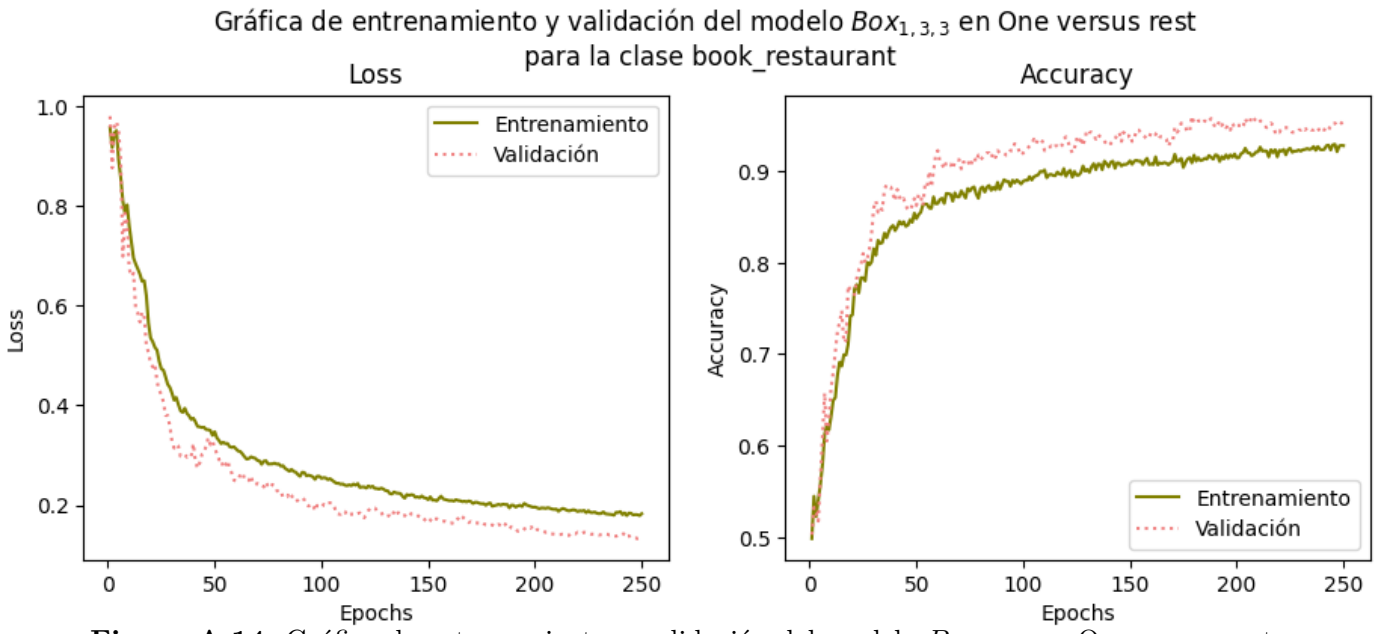



**Figura A.14:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *book_restaurant*.

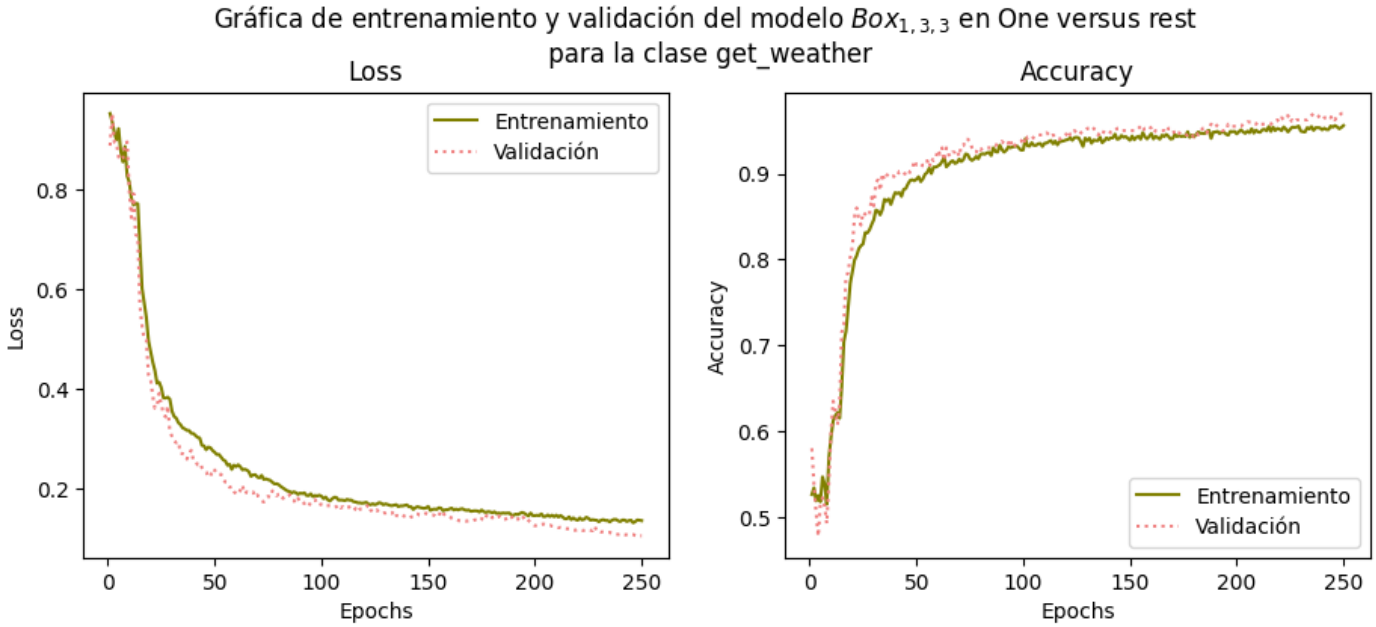



**Figura A.15:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *get_weather*.

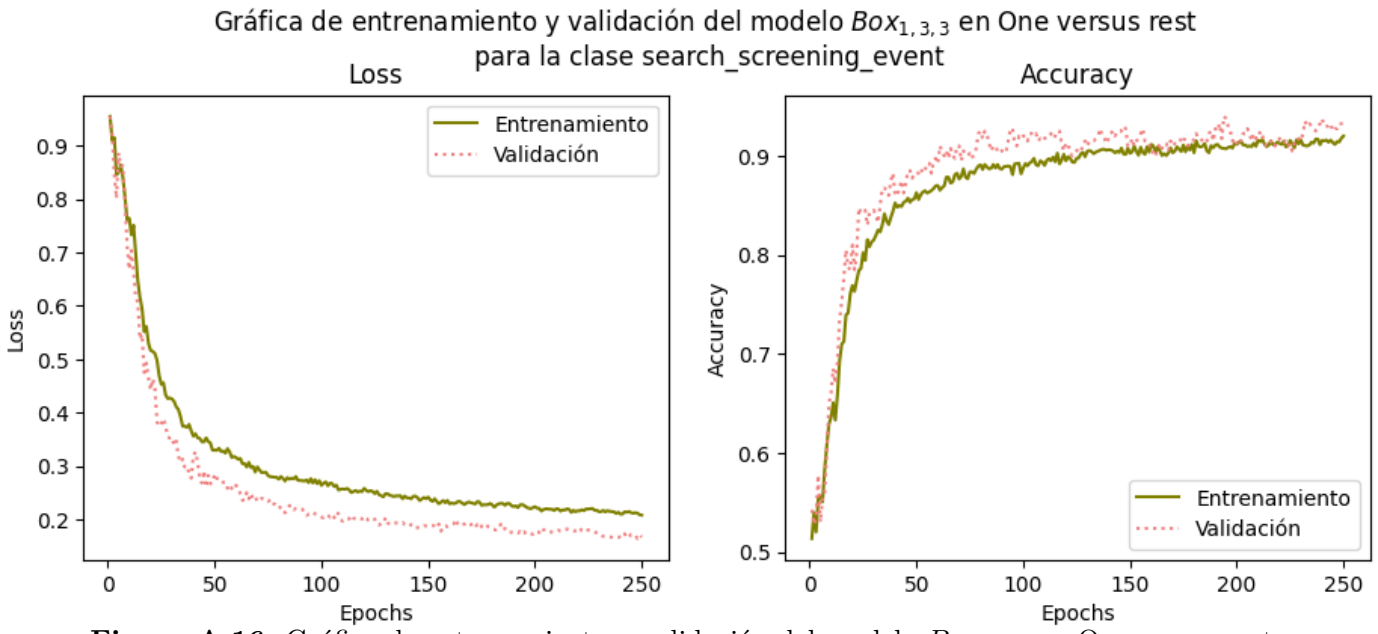



**Figura A.16:** Gráfica de entrenamiento y validación del modelo $Box_{1,3,3}$ en One versus rest para la clase *search_screening_event*.

# Bibliografía


[1] Ashish Vaswani et al. «Attention is all you need». In: *Advances in neural information processing systems*. Vol. 30. 2017. URL: https://arxiv.org/abs/1706.03762.

[2] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805.

[3] Alec Radford et al. *Improving Language Understanding by Generative Pre-Training*. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf. 2018.

[4] Kun Jing and Jungang Xu. *A Survey on Neural Network Language Models*. 2019. arXiv: 1906.03591 [cs.CL]. URL: https://arxiv.org/abs/1906.03591.

[5] Zellig S. Harris. «Distributional Structure». In: *Word* 10.2-3 (1954), pp. 146–162. DOI: 10.1080/00437956.1954.11659520.

[6] John Rupert Firth. «A Synopsis of Linguistic Theory 1930–1955». In: *Selected Papers of J. R. Firth 1952–1959*. Ed. by F. R. Palmer. London: Longman, 1968, pp. 168–205. URL: https://cs.brown.edu/courses/csci2952d/readings/lecture1-firth.pdf.

[7] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd. Online manuscript released January 12, 2025. 2025. URL: https://web.stanford.edu/~jurafsky/slp3/.

[8] Yoshua Bengio et al. «A Neural Probabilistic Language Model». In: *Journal of Machine Learning Research* 3 (2003), pp. 1137–1155. URL: https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf.

[9] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. 2013. arXiv: 1301.3781 [cs.CL]. URL: https://arxiv.org/abs/1301.3781.

[10] Konstantinos Meichanetzidis et al. «Grammar-aware sentence classification on quantum computers». In: *Quantum Machine Intelligence* 5.1 (Feb. 2023). ISSN: 2524-4914. DOI: 10.1007/s42484-023-00097-1. URL: http://dx.doi.org/10.1007/s42484-023-00097-1.

[11] Farha Nausheen et al. *Quantum Natural Language Processing: A Comprehensive Review of Models, Methods, and Applications*. 2025. arXiv: 2504.09909 [cs.CL]. URL: https://arxiv.org/abs/2504.09909.

[12] Lov K. Grover. *A fast quantum mechanical algorithm for database search*. 1996. arXiv: quant-ph/9605043 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9605043.

[13] Aliakbar Panahi, Seyran Saeedi, and Tom Arodz. *word2ket: Space-efficient Word Embeddings inspired by Quantum Entanglement*. 2020. arXiv: 1911.04975 [cs.LG]. URL: https://arxiv.org/abs/1911.04975.

[14] Diksha Sharma, Parvinder Singh, and Atul Kumar. «The role of entanglement for enhancing the efficiency of quantum kernels towards classification». In: *Physica A: Statistical Mechanics and its Applications* 625 (Sept. 2023), p. 128938. ISSN: 0378-4371. DOI: 10.1016/j.physa.2023.128938. URL: http://dx.doi.org/10.1016/j.physa.2023.128938.

[15] Yanan Li et al. *Quantum Recurrent Neural Networks for Sequential Learning*. 2023. arXiv: 2302.03244 [quant-ph]. URL: https://arxiv.org/abs/2302.03244.

[16] Guangxi Li, Xuanqiang Zhao, and Xin Wang. *Quantum Self-Attention Neural Networks for Text Classification*. 2023. arXiv: 2205.05625 [quant-ph]. URL: https://arxiv.org/abs/2205.05625.

[17] Gottlob Frege. «Über Sinn und Bedeutung». In: *Zeitschrift für Philosophie und philosophische Kritik* 100 (1892), pp. 25–50.

[18] Noam Chomsky. *Syntactic Structures*. The Hague: Mouton, 1957. ISBN: 978-3-11-017279-9.

[19] Bob Coecke, Mehrnoosh Sadrzadeh, and Stephen Clark. *Mathematical Foundations for a Compositional Distributional Model of Meaning*. 2010. arXiv: 1003.4394 [cs.CL]. URL: https://arxiv.org/abs/1003.4394.

[20] Alice Coucke et al. *Snips Voice Platform: an embedded Spoken Language Understanding system for private-by-design voice interfaces*. 2018. arXiv: 1805.10190 [cs.CL]. URL: https://arxiv.org/abs/1805.10190.

[21] Dimitri Kartsaklis et al. «lambeq: An Efficient High-Level Python Library for Quantum NLP». In: *arXiv preprint arXiv:2110.04236* (2021).

[22] Quantinuum. *Lambeq Documentation*. Accedido: 2025-06-27. 2024. URL: https://docs.quantinuum.com/lambeq.

[23] José Manuel Ramírez García. *Intents-Classification-Lambeq*. Creado: 2025-10-27. 2025. URL: https://github.com/josmanram/Intents-Classification-Lambeq.

[24] Claude E. Shannon. «A Mathematical Theory of Communication». In: *Bell System Technical Journal* 27.3 (1948), pp. 379–423. URL: https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf.

[25] Mikio Nakahara and Tetsuo Ohmi. *Quantum Computing: From Linear Algebra to Physical Realizations*. CRC Press, 2008.

[26] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2000. ISBN: 978-0-521-63503-5.

[27] Oswaldo Zapata. *A Short Introduction to Quantum Computing for Physicists*. 2024. arXiv: 2306.09388 [quant-ph]. URL: https://arxiv.org/abs/2306.09388.

[28] Walter Greiner and Bernd Müller. *Relativistic Quantum Mechanics: Symmetries*. Springer-Verlag, 1997. ISBN: 978-3-54-058080-5.

[29] J. Ossorio-Castillo and José M. Tornero. *Quantum computing from a mathematical perspective: a description of the quantum circuit model*. 2025. arXiv: 1810.08277 [quant-ph]. URL: https://arxiv.org/abs/1810.08277.

[30] Ryszard Horodecki et al. «Quantum entanglement». In: *Reviews of Modern Physics* 81.2 (June 2009), pp. 865–942. ISSN: 1539-0756. DOI: 10.1103/revmodphys.81.865. URL: http://dx.doi.org/10.1103/RevModPhys.81.865.

[31] Minati Rath and Hema Date. *Quantum Data Encoding: A Comparative Analysis of Classical-to-Quantum Mapping Techniques and Their Impact on Machine Learning Accuracy*. 2023. arXiv: 2311.10375 [quant-ph]. URL: https://arxiv.org/abs/2311.10375.

[32] Dan Shepherd and Michael J. Bremner. «Temporally unstructured quantum computation». In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 465.2105 (Feb. 2009), pp. 1413–1439. ISSN: 1471-2946. DOI: 10.1098/rspa.2008.0443. URL: http://dx.doi.org/10.1098/rspa.2008.0443.

[33] Michael J. Bremner, Richard Jozsa, and Dan J. Shepherd. «Classical simulation of commuting quantum computations implies collapse of the polynomial hierarchy». In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 467.2126 (Aug. 2010), pp. 459–472. ISSN: 1471-2946. DOI: 10.1098/rspa.2010.0301. URL: http://dx.doi.org/10.1098/rspa.2010.0301.

[34] Gavin E. Crooks. *Gradients of parameterized quantum gates using the parameter-shift rule and gate decomposition*. 2019. arXiv: 1905.13311 [quant-ph]. URL: https://arxiv.org/abs/1905.13311.

[35] James C. Spall. «Multivariate stochastic approximation using a simultaneous perturbation gradient approximation». In: *IEEE Transactions on Automatic Control* 37.3 (1992), pp. 332–341. DOI: 10.1109/9.119632. URL: https://www.jhuapl.edu/spsa/pdf-spsa/spall_tac92.pdf.

[36] Antal Szava and David Wierichs. *SPSA Optimization*. Last updated: 2024-11-06. Xanadu Quantum Technologies. Mar. 19, 2023. URL: https://pennylane.ai/qml/demos/tutorial_spsa.

[37] Chris Heunen and Jamie Vicary. *Notes on: Categorical Quantum Mechanics – an Introduction*. Lecture notes based on lectures by David Reutter and Jamie Vicary, Department of Computer Science, University of Oxford. 2019. URL: https://www.cs.ox.ac.uk/files/10510/notes.pdf.

[38] nLab authors. *rigid monoidal category*. Última actualización: 2025-01-07, Accedido: 2025-06-18. 2025. URL: https://ncatlab.org/nlab/show/rigid+monoidal+category.

[39] John H. Selby, Carlo Maria Scandolo, and Bob Coecke. «Reconstructing quantum theory from diagrammatic postulates». In: *Quantum* 5 (Apr. 2021), p. 445. ISSN: 2521-327X. DOI: 10.22331/q-2021-04-28-445. URL: http://dx.doi.org/10.22331/q-2021-04-28-445.

[40] Mehrnoosh Sadrzadeh. «Pregroup Grammars, Their Syntax and Semantics». In: *Joachim Lambek: The Interplay of Mathematics, Logic, and Linguistics*. Springer International Publishing, 2021, pp. 347–376. ISBN: 9783030665456. DOI: 10.1007/978-3-030-66545-6_10. URL: http://dx.doi.org/10.1007/978-3-030-66545-6_10.

[41] J. Lambek. «Type Grammar Revisited». In: *Logical Aspects of Computational Linguistics*. Springer, 1999, pp. 1–17.

[42] nLab authors. *Pregroup grammar*. Última actualización: 2021-07-22, Accedido: 2025-06-15. 2021. URL: https://ncatlab.org/nlab/show/pregroup+grammar.

[43] Stephen Clark. *Something Old, Something New: Grammar-based CCG Parsing with Transformer Models*. 2021. arXiv: 2109.10044 [cs.CL]. URL: https://arxiv.org/abs/2109.10044.

[44] G. de Felice, A. Toumi, and B. Coecke. «DisCoPy: Monoidal Categories in Python». In: *EPTCS*. Vol. 333. 2021, pp. 183–197. DOI: 10.4204/EPTCS.333.13.

[45] Explosion.ai. *spaCy: Industrial-strength Natural Language Processing in Python*. Accedido: 2025-10-15. 2020. URL: https://spacy.io.

[46] Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. Cambridge, MA: MIT Press, 1999. ISBN: 978-0-262-13360-9. URL: https://nlp.stanford.edu/fsnlp/.

[47] Shervin Le Du, Senaida Hernández Santana, and Giannicola Scarpa. *A gentle introduction to Quantum Natural Language Processing*. 2022. arXiv: 2202.11766 [cs.CL]. URL: https://arxiv.org/abs/2202.11766.

[48] Michael A. Nielsen. *Quantum information theory*. 2000. arXiv: quant-ph/0011036 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/0011036.

[49] David J. Griffiths. *Introduction to Quantum Mechanics*. Prentice Hall, 1995. ISBN: 978-0-13-124405-4.

[50] S. Abramsky and N. Tzevelekos. «Introduction to Categories and Categorical Logic». In: *New Structures for Physics*. Springer Berlin Heidelberg, 2010, pp. 3–94. ISBN: 978-3-642-12821-9. DOI: 10.1007/978-3-642-12821-9_1. URL: http://dx.doi.org/10.1007/978-3-642-12821-9_1.

[51] nLab authors. *category theory*. Última actualización: 2025-10-19, Accedido: 2025-11-05. 2025. URL: https://ncatlab.org/nlab/show/category+theory.

[52] Samson Abramsky and Bob Coecke. *Categorical quantum mechanics*. 2008. arXiv: 0808.1023 [quant-ph]. URL: https://arxiv.org/abs/0808.1023.

[53] Wikipedia contributors. *Compact closed category*. Última actualización: 2024-10-26, Accedido: 2025-06-18. 2024. URL: https://en.wikipedia.org/wiki/Compact_closed_category.